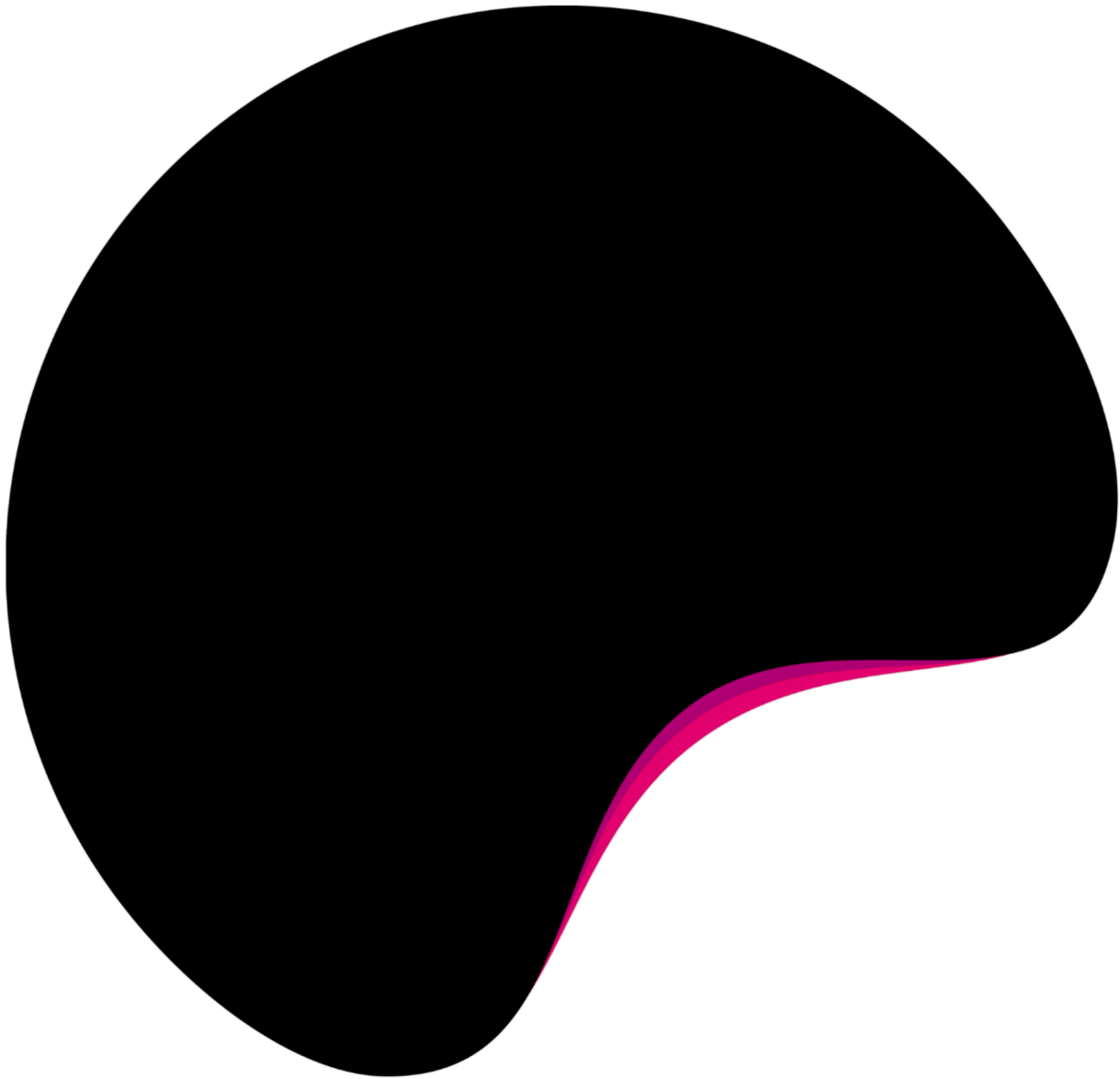




GlowCore

SE Project Documentation

Version: 2.01

Date: 26.05.2026

Git Version: 4038c2f

Project Team: Cedric Cathomas
Yoris Kucera
Dominik Wyss
Nathanael Fässler

Project Advisor: Thomas Bocek

School of Computer Science
OST Eastern Switzerland University of Applied Sciences

Contents

I	Management Summary	1
II	Product Documentation	4
1	Glossary	5
2	Requirements	6
2.1	Functional Requirements	6
2.1.1	Actors / Personas	6
2.1.1.1	Casual Player	6
2.1.1.2	Dedicated Player	6
2.1.2	Use Case Diagram	6
2.1.3	Use Cases	8
2.1.4	UC Overview	8
2.2	Non-Functional Requirements	11
2.2.1	NFR Overview	12
2.2.2	Priority Matrix	13
2.2.3	Performance Efficiency	14
2.2.4	Usability	16
2.2.5	Reliability	17
2.2.6	Maintainability	19
2.2.7	Portability	19
2.2.8	NFR-Architecture Traceability	21
3	Domain Analysis	22
3.1	Enemy State Machine	22
4	Architecture	24
4.1	Frontend Architecture	24
4.1.1	Main Menu	24
4.1.2	New Game	25
4.1.3	Pause Menu	26
4.1.4	Audio Settings	27
4.1.5	Display Settings	28
4.1.6	Unapplied Changes	29
4.1.7	Inventory	30
4.1.8	Crafting Table	31
4.1.9	More Crafting Stations	32
4.1.10	GlowCore Upgrade	33
4.1.11	Chest	34
4.2	C4 Model	35
4.2.1	System Context Diagram	35
4.2.2	Container Diagram	36
4.2.3	Component Diagram	36
4.2.3.1	Gameplay Logic	37
4.2.3.2	Presentation Layer	38
4.3	Interfaces	39
4.3.1	Overview Diagram	39
4.3.2	Interface Contracts	40
4.3.3	Extensibility through Stable Interfaces	40
4.4	Technologies	41
4.4.1	Unity	41
4.4.2	Universal Render Pipeline (URP)	41
4.4.3	.NET & C#	41
4.4.4	GitHub	41
4.4.5	Blender	41
4.4.6	Audacity	41

4.5	Used Design Patterns	42
4.5.1	Component-Based Architecture	42
4.5.2	Singleton	42
4.5.3	Dependency Injection	42
4.5.4	Scriptable Objects	42
4.5.5	Prefabs	42
4.6	Extensibility	43
4.7	External Dependencies	43
4.8	Game Design	44
4.8.1	Core Gameplay Loop	44
4.8.2	Gameplay Progression	44
4.8.2.1	Early Game	44
4.8.2.2	Automation Phase 1	44
4.8.2.3	Automation Phase 2	44
4.8.2.4	Automation Phase 3	44
4.8.2.5	Full Automation	44
4.8.3	Automation Constraints	45
4.8.4	Depth and Complexity	45
4.9	Key Architectural Decisions	46
4.9.1	ScriptableObjects as the Data Layer	46
4.9.2	IInteractable for World Interaction	46
4.9.3	IInventoryService and Event-Driven UI	46
4.9.4	Use Actions and Interact Actions	46
4.9.5	IPlayerInventoryAware via Explicit Injection	46
4.10	Deployment	48
4.10.1	Continuous Deployment - WebGL (GitHub Pages)	48
4.10.2	Release Deployment - Windows & Linux (GitHub Releases)	48
4.10.3	Steam	48
4.10.4	Deployment Diagram	48
5	Quality Measures	49
5.1	Working Environment	49
5.1.1	CI/CD Pipeline	49
5.1.2	Definition of Ready and Definition of Done	50
5.1.2.1	DoR Example	51
5.1.2.2	DoD Example	51
5.2	Test Concept	52
5.2.1	Automated Testing (Unit & Integration)	52
5.2.2	System Testing	52
5.2.3	Usability Testing	52
5.2.4	Non-Functional Requirements (NFR) Verification	52
5.3	Unity Testing	53
5.3.1	Edit Mode Tests	53
5.3.2	Play Mode Tests	53
5.3.3	Coverage Summary	54
5.4	Code metrics	54
5.4.1	Lines of Code	54
5.4.2	Test Coverage	55
5.5	User Testing	55
5.5.1	User Test 1 (UT01)	57
5.5.1.1	Additional Feedback	57
5.5.1.2	Conclusion	57
5.5.2	User Test 2 (UT02)	58
5.5.2.1	Additional Feedback	58
5.5.2.2	Conclusion	58
5.6	Coding Guidelines	58
5.6.1	Naming Conventions	58
5.6.2	Class Member Ordering	59

5.6.3	General Guidelines	59
5.6.4	Formatting Rules	59
5.7	Version Control Workflow	60
5.7.0.1	Branch Naming	60
5.7.0.2	Rebasing Workflow	60
5.7.0.3	Pull Requests	60
6	Savefile	61

III Project Documentation 62

7	Initial Project Proposal	63
7.1	Team Members	63
7.2	Availabilities	63
7.3	Vision	63
7.4	Proposed Realisation	64
8	Project Plan	65
8.1	Resources	65
8.1.1	People	65
8.1.1.1	Project Management Roles	65
8.1.1.2	Developer Roles	66
8.1.2	Time	66
8.1.3	Cost	66
8.2	Organization	66
8.2.1	Meetings	66
8.2.2	Planning Tools	67
8.3	Schedule	68
8.3.1	Work Items	68
8.3.2	Long Term Plan	68
8.3.2.1	Milestones	68
8.3.2.1.1	Initial Milestones	69
8.3.2.1.2	Updated Milestones as of 07.04.2026	69
8.3.2.2	Epics	70
8.3.2.3	Initial Backlog & MVP	71
8.3.3	Short Term Plan	72
8.3.3.1	Sprint 1 (Sprint Zero) 20.02.2026 - 06.03.2026	73
8.3.3.2	Sprint 2 06.03.2026 - 20.03.2026	74
8.3.3.3	Sprint 3 20.03.2026 - 03.04.2026	75
8.3.3.4	Sprint 4 03.04.2026 - 17.04.2026	76
8.3.3.5	Sprint 5 17.04.2026 - 01.05.2026	77
8.3.3.6	Sprint 6 01.05.2026 - 15.05.2026	78
8.4	Risk Management	79
8.4.1	Risk Matrix	79
8.4.2	Risk Matrix (Mitigated)	80
8.4.3	Assessed Risk Matrix (14.04.2026)	80
8.4.4	Adjustment of Current Risk Assessment	80
9	Time Tracking Report	82
9.1	Time Tracking	82
9.2	Time Statistics	82
10	Personal Reports	83
11	Meeting Minutes	84
11.1	Weekly Scrum / 28.02.2026	84
11.1.1	Agenda	84
11.1.2	Todos	85
11.2	Weekly Scrum and Sprint Retrospective / 06.03.2026	86
11.2.1	Agenda	86
11.2.2	Sprint 1 Retrospective	86
11.2.2.1	What Went Well	86

11.2.2.2	What Didn't Go Well	87
11.2.2.3	Improvements	87
11.2.3	Todos	87
11.3	Sprint Planning / 06.03.2026	88
11.3.1	Sprint goal	88
11.4	Sprint Review (Advisor) / 10.03.2026	89
11.4.1	Agenda	89
11.4.2	Todos	89
11.4.3	Next dates	89
11.5	Weekly Scrum / 13.03.2026	90
11.5.1	Agenda	90
11.5.2	Todos	91
11.6	Weekly Scrum and Sprint Retrospective / 20.03.2026	92
11.6.1	Agenda	92
11.6.2	What went well	92
11.6.3	What didn't	92
11.6.4	Improvements	92
11.6.5	Todos	92
11.7	Sprint Planning / 20.03.2026	94
11.7.1	Sprint goal	94
11.8	Sprint Review (Advisor) / 24.03.2026	95
11.8.1	Agenda	95
11.8.2	Todos	95
11.8.3	Next dates	95
11.9	Weekly Scrum / 27.03.2026	96
11.9.1	Agenda	96
11.9.2	Todos	96
11.10	Weekly Scrum and Sprint Retrospective / 03.04.2026	97
11.10.1	Agenda	97
11.10.2	What went well	97
11.10.3	What didn't	97
11.10.4	Todos	97
11.11	Sprint Planning / 03.04.2026	99
11.11.1	Sprint goal	99
11.12	Sprint Review (Advisor) / 14.04.2026	100
11.12.1	Agenda	100
11.12.2	Todos	100
11.12.3	Next dates	101
11.13	Weekly Scrum and Sprint Retrospective / 18.04.2026	102
11.13.1	Agenda	102
11.13.2	What went well	102
11.13.3	What didn't	102
11.13.4	Todos	102
11.14	Sprint Planning / 18.04.2026	103
11.14.1	Sprint goal	103
11.15	Weekly Scrum / 24.04.2026	104
11.15.1	Agenda	104
11.15.2	Todos	104
11.16	Weekly Scrum and Sprint Retrospective / 30.04.2026	105
11.16.1	What went well	105
11.16.2	What didn't	105
11.17	Sprint Planning / 01.05.2026	106
11.17.1	Sprint goal	106
11.18	Sprint Review (Advisor) / 05.05.2026	107
11.18.1	Agenda	107
11.18.2	Todos	107
11.18.3	Next dates	107

11.19	Weekly Scrum / 08.05.2026	108
11.20	Weekly Scrum and Sprint Retrospective / 15.05.2026	109
11.20.1	Agenda	109
11.20.2	What went well	109
11.20.3	What didn't	109
11.20.4	Todos	109
11.21	Sprint Planning / 15.05.2026	110
11.21.1	Sprint goal	110
11.22	Sprint Review (Advisor) / 19.05.2026	111
11.23	Weekly Scrum / 22.05.2026	112
11.24	Weekly Scrum and Sprint Retrospective / 29.05.2026	113
IV	Bibliography	114
V	List of Figures	115
VI	List of Tables	117
VII	Code Listings	120

Part I

Management Summary

Management Summary

Starting Point

GlowCore started as the idea of building a small singleplayer game where the player begins in a tiny lit area surrounded by darkness and has to expand the world by upgrading a central object called the GlowCore. The challenge for us as a team of four students had two sides. We had to design a game that is actually fun to play, and at the same time we had to build it cleanly enough that the code survives growing scope, four people working in parallel, and the usual deadline pressure of a Semester 4 project.

The other half of the goal was applying the software engineering process we learned about in lectures to a real codebase that we cared about. Scrum, CI/CD, automated testing, code reviews, branching, all of it had to work in practice and not just on paper.

Our Approach

We picked Unity 6 with the Universal Render Pipeline as our engine. C# was a language everyone already knew and we used the time before the official project start to get up to speed on Unity, so we could be productive from day one. The code was split into two main layers, gameplay logic and presentation, with clear interfaces between them. ScriptableObjects are used for data like items, recipes and node definitions, which makes it possible to add new content without touching any code.

Work was organised in two-week sprints with a weekly Scrum meeting and a regular review with our advisor. Every change went through a pull request, was reviewed by a teammate, and had to pass our CI pipeline that runs linting, formatting checks and automated tests. Roles were split between the four of us so that everyone had a main area of responsibility but could still help out anywhere in the codebase.

One of the more important decisions we made during the project came after the first few sprints. The original plan was ambitious and included a combat system, multiple automation phases and additional biomes. After looking at our progress in April we decided to drop combat and the later automation phases from the Beta scope on purpose. The reasoning was simple, a smaller game that feels polished beats a bigger game that feels half finished. We documented the updated milestones and went through them with our advisor.

For the game itself we designed a core loop around gathering wood, crafting tools, upgrading the GlowCore and slowly automating the work with the axe machine. The intention was to give the player a steady sense of progress and to make automation feel like a reward rather than something the game hands out from the start.

Result

GlowCore covers the full MVP we defined at the start of the project. The player can install the game, save and load progress, break and place nodes, expand the map by feeding the GlowCore, store items in chests, craft items at the workbench and the furnace, and change settings to their taste. All of this runs on Windows, Linux and in the browser through WebGL, with the WebGL build deployed automatically to GitHub Pages on every push to main or dev.

The numbers back this up. The game runs at around 120 FPS on WebGL during active play, well above the 60 FPS target we set ourselves. Worlds load in roughly 1.5 seconds against a 3 second budget, and save files come in at a few kilobytes despite a 5 MB ceiling. We ran two external user tests with players outside the team. Both testers reached the GlowCore upgrade loop on their own within a few minutes and confirmed that the core loop is fun to play. Their feedback on the parts that confused them or needed polish went straight into the next sprint.

On the engineering side the codebase stays consistent thanks to the automated checks, and the architecture proved its value when we added additional crafting stations late in the project without having to rewrite existing systems. The hours we tracked across the team are running close to the 480 hour budget we planned at the start, which tells us that our storypoint estimates and sprint planning have been reasonably accurate.

Looking Forward

GlowCore is in a good state to keep growing. The architecture was built with extensibility in mind, so adding new items, nodes, recipes or even entire crafting stations comes down to creating a new ScriptableObject asset and wiring it up in the inspector. The later automation phases, combat and farming systems are documented in the design and could be picked up either as a follow up project or as a continuation outside the semester. The foundation, the deployment pipelines, the testing setup and the documentation are all in place to support that, regardless of who continues the work.

Part II

Product Documentation

1 Glossary

Term	Definition
Grid	Refers to the 2D Grid that divides the GlowCore world into rows and columns.
Tile	A Tile is a single unit of space in the grid and has the form of a square.
Node	A Node refers to an object that is occupying a Tile. Not every Tile has a Node, but every Node belongs to a Tile.
World Item	World Items are items that can be picked up by the player and physically exist somewhere in the GlowCore world. In contrast to Nodes they do not belong to a Tile and do not have a hitbox.
Inventory Item	Inventory Items refer to the items that the player possesses. Certain Nodes such as Chests can also hold Inventory Items.
Tool	A Tool is an Inventory Item that can be used to break certain Nodes. For example an Axe can break Trees.
Block	A Block is an Inventory Item that can be placed onto the Grid to build a Node.

Table 1: Glossary

2 Requirements

2.1 Functional Requirements

2.1.1 Actors / Personas

Since a game typically only has one primary actor - **The Player** - it makes sense to define Personas that reflect different types of players. The categorization into Casual- and Dedicated Players covers most player types. In the following, the pronoun **he** is used for simplicity, but the Personas are meant to include all players.

2.1.1.1 Casual Player

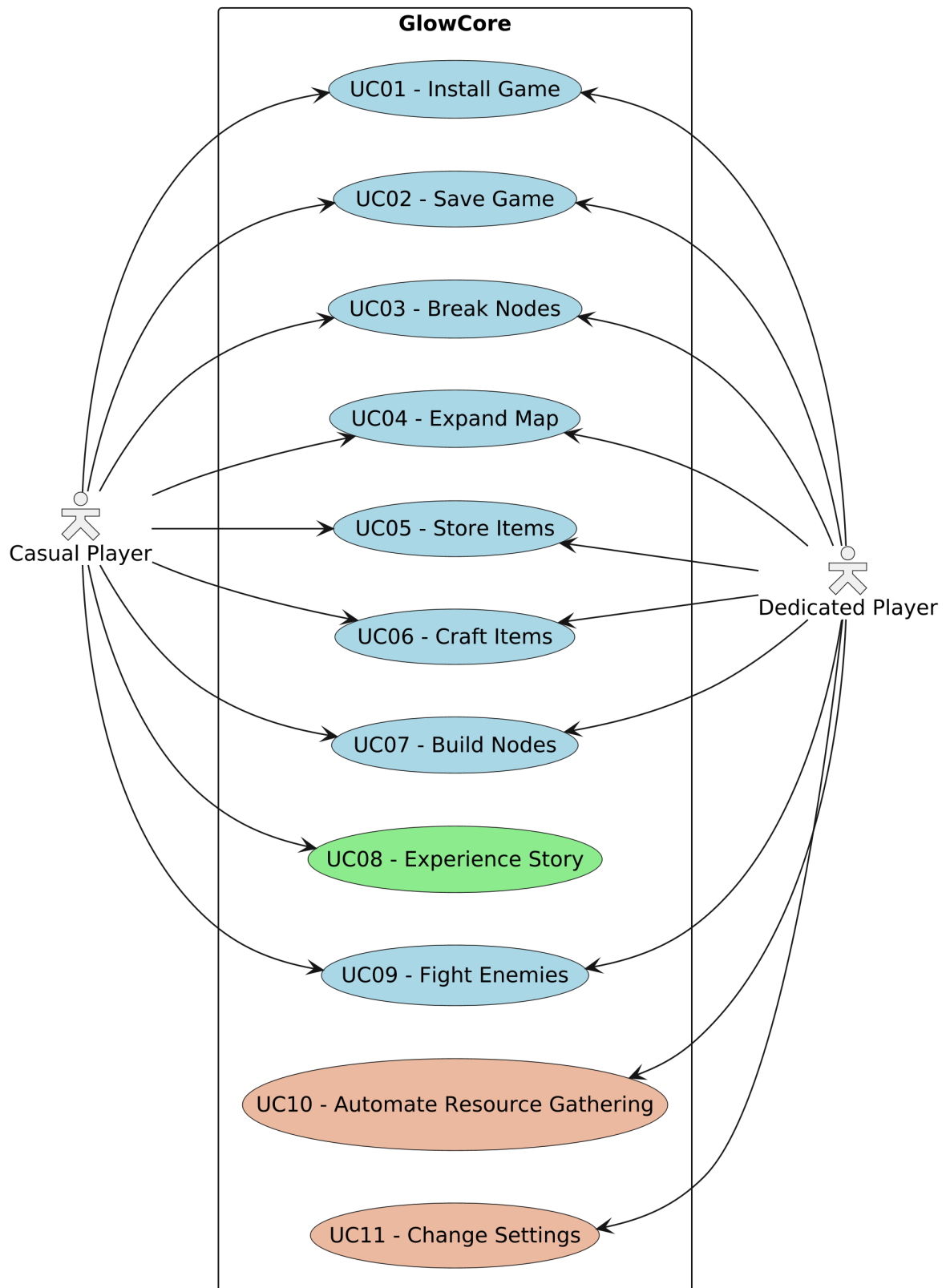
- The Casual Player plays GlowCore infrequently and for short amounts of time. He wants to relax and unwind from his life. He prefers to play the game just for fun without putting effort into it.
- The Casual Player wants to experience the game from beginning to end with a story and explore the world. When he plays the game for the first time, he depends on the game to explain the rules of the game to him without being too overwhelming and tedious. As soon as he finishes the game, he will be done with it, having experienced everything he wanted.
- The Casual Player never changes settings in games, he never feels the need to customize anything. He likes to play the game as it is.

2.1.1.2 Dedicated Player

- The Dedicated Player plays GlowCore in long sessions. He loves games where you build factories and automate everything. He searches for the feeling of progression and finds satisfaction in it. He always searches for a better way to progress and optimize his gameplay.
- The Dedicated Player has already played the game multiple times before, thus he doesn't want to be bored with a long tutorial everytime he starts a new game. He wants to be able to dictate the pacing of the game.
- He doesn't care for the story and lore in games. He doesn't want to read through dialogues and doesn't like long cutscenes.

2.1.2 Use Case Diagram

The diagram shows a modified version of the Use Case Diagram. The Personas take the place of the Actors. An arrow indicates that a Persona uses a feature. A missing arrow means that the Persona is not interested in using a feature, although it remains accessible to him.

**Image 1:** Use Case Diagram

2.1.3 Use Cases

The Use Cases are described in the brief format, with each Use Case including an appropriately detailed description.

It is essential for the project that all Use Cases that define the **MVP** are implemented. The Use Cases marked with **Priority: Middle** are planned to be implemented, but can be reduced if the MVP takes more time than anticipated. The Use Cases marked with **Priority: Low** will be implemented if enough time is left.

Each Use Case table includes a colored status indicator in the top-right corner representing its current verification state:

Color	Status
	Not yet verified (default)
	Implemented – use case fully functional and verified
	Partially implemented – some aspects functional, others pending
	Not implemented – use case not yet functional
	N/A – decided not to implement

Table 2: UC Status Color Legend

2.1.4 UC Overview

The following table provides a compact overview of all Use Cases, their persona, priority, and current verification status.

ID	Name	Persona	Priority	Verify at	Status
UC01	Install Game	Casual & Dedicated Player	High – MVP	M09 Beta	
UC02	Save Game	Casual & Dedicated Player	High – MVP	M09 Beta	
UC03	Break Nodes	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC04	Expand Map	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC05	Store Items	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC06	Craft Items	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC07	Build Nodes	Casual & Dedicated Player	High – MVP	M08 Alpha	
UC08	Experience Story	Casual Player	Low	M10 Release	
UC09	Fight Enemies	Casual & Dedicated Player	Low	M10 Release	
UC10	Automate Resource Gathering	Dedicated Player	Middle	M10 Release	
UC11	Change Settings	Dedicated Player	Middle	M09 Beta	

Table 3: UC Overview – All Use Cases at a Glance

ID	UC01
Name	Install Game
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player can install the game. The installation requires no technical skills.
Result	The game can be installed on Windows by downloading the zip from the GitHub release and launching the .exe directly. For that no technical knowledge is required.

Table 4: UC01 - Install Game

ID	UC02
Name	Save Game
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	When the player stops playing, the game saves his save state in a file and loads it when he resumes the game.
Result	The game autosaves every 20 seconds and the save is loaded correctly the next time the player starts the game. Inventory contents, chest contents, the GlowCore level and any blocks the player built or broke are all restored. The player can also save at any time from the pause menu, and the game saves automatically when returning to the main menu. If the player just closes the window without saving, the most recent autosave is loaded next time.

Table 5: UC02 - Save Game

ID	UC03
Name	Break Nodes
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player can break Nodes in the world with tools. When the Nodes break, they drop items depending on the type of Node. E.g. a Tree drops a Wood-Item. Certain Nodes that are built by the player themselves such as a workbench drop themselves upon breaking.
Result	Verified manually. Trees drop wood when broken with an axe, stone nodes drop stone when broken with a pickaxe, and coal nodes drop both coal and stone. The crafting bench drops itself when broken with an axe. All acceptance criteria met.

Table 6: UC03 - Break Nodes

ID	UC04
Name	Expand Map
Persona	Casual Player & Dedicated Player
Priority	High - Required for MVP
Description	The player is confined to only move on tiles that are unlocked at a given point. The player can drop off items at the GlowCore, whereupon the map expands. The player can then move into and explore the new area.
Result	Verified manually. The player is correctly confined to unlocked tiles and cannot move into locked areas. Feeding resources into the GlowCore triggers a map expansion, and the newly unlocked area becomes accessible immediately. All acceptance criteria met.

Table 7: UC04 - Expand Map

ID	UC05	
Name	Store Items	
Persona	Casual Player & Dedicated Player	
Priority	High - Required for MVP	
Description	Items that the player picks up are stored in his inventory. The player's inventory has a limited size. To organize items, the player can move items into chests and take them out again.	
Result	Verified manually. Items picked up by the player are stored in his inventory, and the inventory has a limited size and refuses additional items when it is full. The player can open a chest by interacting with it, move items between the chest and his inventory by dragging them, and the chest closes automatically when he walks away. Chest contents are saved and restored correctly on the next start, and breaking a chest drops its remaining contents on the ground.	

Table 8: UC05 - Store Items

ID	UC06	
Name	Craft Items	
Persona	Casual Player & Dedicated Player	
Priority	High - Required for MVP	
Description	The player can craft items into other items with predefined recipes. Without a workbench the player is limited to a few simple recipes. The player can interact with a workstation such as a workbench to unlock more crafting recipes. Workstations such as an oven require additional items: e.g. Coal to smelt an Ingot.	
Result	Verified manually. Without a workbench the player has access to a few basic recipes that can be crafted directly from the inventory. Interacting with a crafting table opens its crafting UI and unlocks the full set of standard recipes. A furnace is also available as a second workstation with its own recipe list, where the player can smelt ores into ingots and has other furnace specific recipes.	

Table 9: UC06 - Craft Items

ID	UC07	
Name	Build Nodes	
Persona	Casual Player & Dedicated Player	
Priority	High - Required for MVP	
Description	The player can place certain items onto the world, where they become Nodes. Each Tile can hold only one Node, and the player is not allowed to build a Node on an already occupied Tile.	
Result	Verified manually. The player can place items such as crafting tables and chests onto the world grid, where they become Nodes. Placing a Node on an already occupied tile is correctly prevented. All acceptance criteria met.	

Table 10: UC07 - Build Nodes

ID	UC08	
Name	Experience Story	
Persona	Casual Player	
Priority	Low	
Description	The player can experience a story while playing the game. When the game starts there is an intro sequence. There is an end to the game, at which point a cutscene is played. The game contains small narrative hints that fit into the story conveyed by the GlowCore-World, including the acoustic, the visual art style and the overall gameplay experience.	
Result		

Table 11: UC08 - Experience Story

ID	UC09
Name	Fight Enemies
Persona	Casual Player & Dedicated Player
Priority	Low
Description	The player can engage in combat with enemies. Defeated enemies drop items that are useful for the player. When the enemy hits the player, he loses hitpoints. When the player has no hitpoints left, he loses some of his items and respawns near the center of the map.
Result	

Table 12: UC09 - Fight Enemies

ID	UC10
Name	Automate Resource Gathering
Persona	Dedicated Player
Priority	Middle
Description	The player can automate tedious tasks, such as breaking Nodes. He can craft machines that are placed as Nodes and they automatically perform a task e.g. chopping down trees. There is a selection of machines that can be combined to automate more and more. Example: The player places multiple Replanter-Machines in a row which automatically replant trees on the neighbouring tile. The player places an Axe-Machine onto the start of the line, at which point the Axe-Machine chops one tree after another and then moves onto the next tile. To automate it even further, the player places Rotaters on both which cause the Axe-Machine to turn around on contact.
Result	

Table 13: UC10 - Automate Resource Gathering

ID	UC11
Name	Change Settings
Persona	Dedicated Player
Priority	Middle
Description	The player can change settings of the game, such as the FOV and the Volume.
Result	Verified manually. The player can open the settings from both the title screen and the in-game pause menu. The audio section has separate sliders for master, music and sound-effect volume. The display section has a fullscreen toggle, a resolution dropdown and separate camera sensitivity sliders for the X and Y axes. Changes are kept between sessions, and the player is asked to confirm before leaving the settings with unsaved changes.

Table 14: UC11 - Change Settings

2.2 Non-Functional Requirements

This chapter documents the Non-Functional Requirements (NFRs) for the GlowCore project, categorized according to the ISO/IEC 25010:2011 quality model. The ISO/IEC 25010 standard defines a comprehensive set of software quality characteristics that serve as a framework for specifying and evaluating system quality. The following categories are used to structure our NFRs:

- **Performance Efficiency** – NFR1XX
- **Usability** – NFR2XX
- **Reliability** – NFR3XX
- **Maintainability** – NFR4XX
- **Portability** – NFR5XX

Each NFR is documented with a description, concrete acceptance criteria, a measurement method, a verification approach, and a priority. The priority consists of two parts:

- **MVP Relevance:**
 - **Required** – must be fulfilled for the MVP.
 - **Optional** – planned for a later stage, not required for MVP delivery.
- **Importance:**
 - **High** – critical for the product.
 - **Medium** – important but not blocking.
 - **Low** – nice to have.

Each NFR table includes a colored status indicator in the top-right corner representing its current verification state:

Color	Status
	Not yet tested (default)
	Passed – all acceptance criteria met
	Partially met – some criteria passed, others pending
	Failed – acceptance criteria not met
	N/A – decided not to implement

Table 15: NFR Status Color Legend

2.2.1 NFR Overview

The following table provides a compact overview of all non-functional requirements, their category, priority, verification milestone, and current status. This serves as a quick reference before diving into the detailed NFR descriptions below.

ID	Name	Category	Priority	Verify at	Status
NFR101	World Load Time	Performance	Required / High	M09 Beta	■
NFR102	In-Game Transition Time	Performance	Required / High	M08 Alpha	■
NFR103	Frame Rate	Performance	Required / High	M08 Alpha	■
NFR104	Save File Size	Performance	Required / Medium	M09 Beta	■
NFR105	Minimal Hardware Req.	Performance	Optional / Medium	M09 Beta	■
NFR201	Player Restriction Visibility	Usability	Required / High	M08 Alpha	■
NFR202	Light Upgrade Progress	Usability	Required / High	M08 Alpha	■
NFR203	New Player Learnability	Usability	Required / High	M09 Beta	■
NFR204	Input Support	Usability	Optional / Medium	M09 Beta	■
NFR205	Game Language	Usability	Optional / Low	M10 Release	■
NFR206	Visual Style Consistency	Usability	Optional / Medium	M09 Beta	■
NFR301	Save File Portability	Reliability	Required / High	M09 Beta	■
NFR302	Game Progress Persistence	Reliability	Required / High	M09 Beta	■
NFR303	Save Data Integrity	Reliability	Required / High	M09 Beta	■
NFR401	Extensibility of Game Systems	Maintainability	Optional / Medium	M09 Beta	■
NFR402	Automated Test Coverage	Maintainability	Required / Medium	M09 Beta	■
NFR501	Multi-Platform Support	Portability	Required / Medium	M09 Beta	■
NFR502	Engine and Rendering, Pipeline	Portability	Optional / Low	M10 Release	■
NFR503	Display Resolution Support	Portability	Optional / Medium	M10 Release	■

Table 16: NFR Overview – All Non-Functional Requirements at a Glance

Verification is tied to project milestones (see [UpdatedMilestones](#)) rather than fixed dates to ensure NFRs are checked at meaningful delivery checkpoints:

- **M08 – Alpha Release (10.04.2026):** All Required / High NFRs must pass. This is the MVP gate.
- **M09 – Beta Release (15.05.2026):** Required / Medium and Optional / Medium NFRs are verified. Performance tuning and playtesting rounds.
- **M10 – Official Release (05.06.2026):** Remaining Optional / Low NFRs are verified. Final polish and compliance check.

2.2.2 Priority Matrix

The priority matrix visualizes the distribution of NFRs across the two priority dimensions. This helps identify where the project's quality focus lies and whether the prioritization is balanced.

	Required (MVP)	Optional (Post-MVP)
High	NFR101, NFR102, NFR103 NFR201, NFR202, NFR203 NFR301, NFR302, NFR303	–
Medium	NFR104, NFR402, NFR501	NFR105, NFR204, NFR206 NFR401, NFR503
Low	–	NFR205, NFR502

Table 17: NFR Priority Matrix – MVP Relevance vs. Importance

The matrix shows that the core quality effort is concentrated in the **Required / High** cell (9 NFRs), covering performance, usability, and reliability which are the three pillars that directly impact the player experience in the MVP. The **Optional / Low** cell contains only 2 NFRs that represent project constraints rather than testable quality goals. Priority is on delivering a stable, playable MVP before addressing optional requirements.

2.2.3 Performance Efficiency

Performance Efficiency addresses the amount of resources used under stated conditions. For GlowCore, this primarily concerns load times, frame rates, and resource consumption to ensure a smooth and responsive gameplay experience.

ID	NFR101	
Description	World Load Time	
Acceptance Criteria	Existing and newly created worlds must load within 3 seconds when the player presses “Start”.	
Priority	Required / High	
Measurement	Measure elapsed time from pressing “Start” until the world is fully loaded and interactive. Tested on minimum hardware specification.	
Verification	Automated load-time benchmark test across 10 consecutive loads with varying world states (new and existing). All runs must complete within 3 seconds.	
Result	Tested manually from the title screen. Both NEW GAME (which generates a fresh world) and CONTINUE (which restores an existing save) bring the player into a fully interactive world in approximately 1.5 seconds, easily below the 3-second threshold.	

Table 18: NFR101 – World Load Time

ID	NFR102	
Description	In-Game Transition Time	
Acceptance Criteria	In-game transitions (e.g. walking into a cave) shall not exceed 2 seconds.	
Priority	Required / High	
Measurement	Measure elapsed time from triggering the transition until the new scene is fully loaded and interactive.	
Verification	Trigger each scene transition during playtesting and record the load time. Automated scene-transition benchmark covering all transition points.	
Result	Not applicable. Due to the lack of time the team decided not to implement cave areas or any scene transitions during the project. This requirement no longer applies to the current scope of GlowCore.	

Table 19: NFR102 – In-Game Transition Time

ID	NFR103
Description	Frame Rate
Acceptance Criteria	The game must run at a minimum of 60 FPS with 1% lows not dropping below 50 FPS on recommended hardware.
Priority	Required / High
Measurement	Use Unity Profiler or an external frame-time analysis tool to record FPS over a 5-minute gameplay session. Evaluate average FPS and 1% low values.
Verification	Run a standardized gameplay scenario on minimum and recommended hardware. Capture FPS metrics and verify thresholds are met.
Result	Manually tested on WebGL build using Chrome's Frame Rendering Stats overlay. The game ran at a stable 120 FPS during active gameplay with no significant drops observed. Exceeds the minimum threshold of 60 FPS and the 1% low threshold of 50 FPS. All acceptance criteria met.

Table 20: NFR103 – Frame Rate

ID	NFR104
Description	Save File Size
Acceptance Criteria	Save files should remain compact and use a binary serialization format. Under the currently defined maximum expected game progression and world configuration, a save file should not exceed 5MB.
Priority	Required / Medium
Measurement	Measure the file size of the save file after a full gameplay session under the defined maximum expected game progression and world configuration.
Verification	Create save files at various progression stages and verify the file size remains within the defined limit for the current scope. Confirm that a binary serialization format is used.
Result	Tested by playing the game through several progression stages (fresh world, a few broken nodes, multiple chests filled with items, several built blocks) and measuring the on-disk size of the save file after each save. A fresh save was around 1KB, and a save taken after reaching the maximum GlowCore level peaked at 3KB, well under the 5MB limit. The save file is written in a custom binary format, as required.

Table 21: NFR104 – Save File Size

ID	NFR105
Description	Minimal Hardware Requirements
Acceptance Criteria	The game must run on minimal hardware with the following specifications: • 2 GHz CPU • 8 GB RAM • NVIDIA GeForce GTX 1660
Priority	Optional / Medium
Measurement	Test the game on a defined minimum hardware baseline and verify all performance NFRs (FPS, load times) are met.
Verification	Execute a full gameplay session on minimum-spec hardware. Record FPS, load times, and memory usage to confirm compliance.
Result	Tested on a Microsoft Surface Pro 8 running on integrated Intel Iris Xe graphics with no dedicated GPU. The game ran smoothly throughout the entire session with no frame drops or lag. FPS stayed consistently high and load times were near instant. Memory usage remained stable with no noticeable spikes. For a game of this scope running on integrated graphics only, the minimal hardware requirement is met.

Table 22: NFR105 – Minimal Hardware Requirements

2.2.4 Usability

Usability covers the degree to which the product can be used effectively, efficiently, and satisfactorily. For GlowCore, this includes visual clarity of game mechanics, learnability for new players, and input support.

ID	NFR201	
Description	Player Restriction Visibility	
Acceptance Criteria	Player restrictions in the game must be clearly communicated through visual indicators such as greyed-out or red-tinted elements. Players must understand restrictions without additional explanation.	
Priority	Required / High	
Measurement	Conduct playtesting with at least 3 new players. Each player must correctly identify restricted actions without external guidance.	
Verification	During playtesting sessions, observe whether players attempt restricted actions and whether they understand the visual cues. Document success rate.	
Result	User testing was conducted. The locked area border is visually communicated to the player. There is a progress indicator when breaking nodes and an interaction prompt when hovering over workstations such as the crafting table. All acceptance criteria met.	

Table 23: NFR201 – Player Restriction Visibility

ID	NFR202	
Description	Light Upgrade Progress Indication	
Acceptance Criteria	Light upgrade progress must be clearly indicated to the player at all times during gameplay.	
Priority	Required / High	
Measurement	During playtesting, ask players to describe their current upgrade progress. At least 80% must answer correctly.	
Verification	Conduct playtesting sessions and survey players about their perceived progress. Verify the UI communicates upgrade status unambiguously.	
Result	User Testing was conducted. The GlowCore upgrade UI clearly displays the current level, progress bar with percentage, required materials and amounts, a preview of the next level appearance, and the effect of the next upgrade. All information updates in real time as materials are fed. All acceptance criteria met.	

Table 24: NFR202 – Light Upgrade Progress Indication

ID	NFR203	
Description	New Player Learnability	
Acceptance Criteria	A new player shall understand core mechanics (movement, gathering, upgrading) within 5 minutes without external instructions.	
Priority	Required / High	
Measurement	Time how long a new player takes to perform their first gather, move to a new area, and initiate an upgrade without any external help.	
Verification	Conduct playtesting with at least 3 players who have never seen the game. Measure time to complete core actions. All players must achieve this within 5 minutes.	
Result	User testing was conducted. All participants were able to upgrade the GlowCore multiple times in under 5 minutes. They learned the core mechanic quickly and were able progress without external help.	

Table 25: NFR203 – New Player Learnability

ID	NFR204	
Description	Input Support	
Acceptance Criteria	The game must support both controller and keyboard input methods.	
Priority	Optional / Medium	
Measurement	Test all core gameplay actions with both a standard controller (e.g. Xbox controller) and keyboard. All actions must be executable with both input methods.	
Verification	Perform a full gameplay session using only controller, then only keyboard. Verify all interactions, menus, and gameplay mechanics are fully accessible with each input method.	
Result	Keyboard input works perfectly for all core gameplay actions. Controller support is not implemented yet, but since the game uses Unity's new Input System, adding controller bindings to the existing input actions would not require much effort.	

Table 26: NFR204 – Input Support

ID	NFR205	
Description	Game Language	
Acceptance Criteria	All in-game text, UI elements, and instructions must be in English.	
Priority	Optional / Low	
Measurement	Review all in-game text and UI elements for non-English content.	
Verification	Manual review of all text-containing assets and UI screens to confirm English language throughout.	
Result	Verified by walking through every screen of the game (title screen, new game, pause menu, settings, inventory, crafting and chest UIs) and reviewing all visible text. All labels, buttons, item names, recipe names and tooltips are in English. No localized or translated strings are present.	

Table 27: NFR205 – Game Language

ID	NFR206	
Description	Visual Style Consistency	
Acceptance Criteria	All visual assets must follow the low-poly pastel art style consistently throughout the game.	
Priority	Optional / Medium	
Measurement	Visual review of all assets against the defined art style guide. No asset shall deviate from the low-poly pastel aesthetic.	
Verification	Conduct an art review session where all in-game assets are compared against reference material. Flag and correct any deviations.	
Result	Went through all in-game assets during a gameplay session and compared them against our reference material. Tiles, trees, rocks, the player character, placed nodes, and UI elements all follow the low-poly pastel aesthetic consistently. No asset stood out as visually inconsistent. The color palette stays coherent throughout and nothing looks out of place.	

Table 28: NFR206 – Visual Style Consistency

2.2.5 Reliability

Reliability addresses the degree to which a system performs specified functions under specified conditions for a specified period of time. For GlowCore, this is critical for save data integrity and game stability.

ID	NFR301
Description	Save File Portability
Acceptance Criteria	Each world save must be stored as a single file that can be copied and moved. When placed into the game's save file folder, the save file must be recognized and function without issues.
Priority	Required / High
Measurement	Copy a world save file to a different machine or location, place it in the save folder, and verify the game loads it correctly with all progress intact.
Verification	Create a world save file with significant progress. Copy it to a new installation. Load the game and verify all progress, items, and world state for that world are preserved.
Result	Tested manually by copying the save file (glowcore.bin) out of the save folder on one machine and dropping it into the save folder on a different installation. The save was checked after every scenario (moving the player, filling inventory and chests, breaking and building blocks, upgrading the GlowCore) and in each case the game recognized the file and restored everything correctly: player position, inventory contents, chest contents, GlowCore level and all blocks the player had built or broken. The save is a single self-contained file, so copying and moving it works without any additional steps.

Table 29: NFR301 – Save File Portability

ID	NFR302
Description	Game Progress Persistence
Acceptance Criteria	Game progress shall not be lost during normal shutdown. Auto-save shall occur at least every 2 minutes or immediately after a light upgrade.
Priority	Required / High
Measurement	Verify auto-save triggers by monitoring save file timestamps during gameplay. Confirm no data loss after normal shutdown.
Verification	Play for several minutes, perform a light upgrade, then shut down the game normally. Restart and verify all progress is retained. Additionally, verify auto-save timestamps occur within 2-minute intervals.
Result	Tested manually by playing several sessions and watching the save file timestamp. The autosave now fires every 20 seconds and keeps firing for the entire session, comfortably below the 2-minute limit required. After a normal shutdown and restart, the player position, inventory contents, chest contents, GlowCore level and all built or broken blocks were preserved. The worst case for unsaved progress is the 20 seconds since the last autosave, which is well within what the requirement allows.

Table 30: NFR302 – Game Progress Persistence

ID	NFR303
Description	Save Data Integrity
Acceptance Criteria	Save data must persist correctly across sessions without corruption.
Priority	Required / High
Measurement	Perform several manual save/load cycles and verify that the game state after loading matches the state at the time of saving.
Verification	Play the game, note the current state (inventory, player position, placed blocks), let the autosave trigger, restart the game, and verify the loaded state matches what was saved.
Result	Tested manually by playing the game, noting the current state (player position, inventory contents, chest contents, GlowCore level and all placed or broken blocks), letting the autosave trigger, restarting the game, and comparing the loaded state to what was noted before. Across multiple save and load cycles the loaded state matched the saved state exactly in every case, with no missing items, misplaced blocks or corrupted data.

Table 31: NFR303 – Save Data Integrity

2.2.6 Maintainability

Maintainability represents the degree of effectiveness and efficiency with which a product can be modified. For GlowCore, this covers extensibility of game systems and test coverage.

ID	NFR401	
Description	Extensibility of Game Systems	
Acceptance Criteria	The code must allow adding new LightCore upgrades and new items with their resource nodes without requiring code restructuring.	
Priority	Optional / Medium	
Measurement	A developer unfamiliar with the codebase must be able to add a new upgrade or item by following existing patterns, without modifying core system code.	
Verification	Task a team member with adding a new test upgrade and resource node. Measure the time required and verify no structural code changes were necessary.	
Result	Verified manually by a team member adding a new resource node. The architecture supports adding new nodes and items without modifying core system code.	

Table 32: NFR401 – Extensibility of Game Systems

ID	NFR402	
Description	Automated Test Coverage	
Acceptance Criteria	All systems must be covered by automated tests. The project targets a minimum of 40% code coverage. (This coverage target is relatively common for video games)	
Priority	Required / Medium	
Measurement	Measure code coverage using the Unity Code Coverage package. Report overall and per-system coverage percentages.	
Verification	Run the full automated test suite and generate a coverage report. Verify that overall coverage meets or exceeds 40%.	
Result	Coverage was measured using the Unity Code Coverage package. As of 16.05.2026 the project sits at 54.7% line coverage (1817 of 3320 coverable lines) and 60.5% method coverage (374 of 618 methods) across 4 assemblies, 72 classes and 70 files. Both numbers comfortably exceed the 40% target.	

Table 33: NFR402 – Automated Test Coverage

2.2.7 Portability

Portability addresses the degree of effectiveness and efficiency with which a system can be transferred from one environment to another. For GlowCore, this covers platform support and display resolution compatibility.

ID	NFR501	
Description	Multi-Platform Support	
Acceptance Criteria	The game must run on Windows, Linux, and Web platforms.	
Priority	Required / Medium	
Measurement	Build and deploy the game on all three platforms. Launch and complete a full gameplay session on each.	
Verification	Execute the full test suite and a manual gameplay session on Windows, Linux, and a Web browser. Verify all features work identically across platforms.	
Result	Manually verified on WebGL (browser), Windows (native build via GitHub release zip), and Linux (native build tested in a Linux VM). The game launched and ran correctly on all three platforms. All three platform builds are generated automatically via the CI/CD pipeline. All acceptance criteria met.	

Table 34: NFR501 – Multi-Platform Support

ID	NFR502	
Description	Engine and Rendering Pipeline	
Acceptance Criteria	The game must run on the Unity engine using the PhysX physics engine and the Universal Render Pipeline (URP).	
Priority	Optional / Low	
Measurement	Verify project settings confirm Unity with PhysX and URP are configured as the active engine and rendering pipeline.	
Verification	Review Unity project settings and confirm PhysX is the active physics engine and URP is the active render pipeline. Verify no fallback to built-in pipeline occurs during gameplay.	
Result		

Table 35: NFR502 – Engine and Rendering Pipeline

ID	NFR503	
Description	Display Resolution Support	
Acceptance Criteria	The game must support common display resolutions with a 16:9 aspect ratio. Minimum: 1280×720. Recommended: 1920×1080, 2560×1440, and 3840×2160 (4K).	
Priority	Optional / Medium	
Measurement	Launch the game at each supported resolution and verify the UI scales correctly, no elements are clipped, and the game renders properly.	
Verification	Test the game at all specified resolutions. Take screenshots and verify correct rendering, UI scaling, and absence of visual artifacts.	
Result		

Table 36: NFR503 – Display Resolution Support

2.2.8 NFR-Architecture Traceability

The table below links NFRs to the specific technical solution in the architecture that is primarily responsible for satisfying it. The purpose of this table is to make the connection between quality requirements and design decisions explicit and verifiable.

NFR	Quality Attribute	Technical Solution	How the Architecture Addresses It
NFR401	Maintainability (Extensibility)	IInteractive, IHandItem, ScriptableObject	New nodes implement IInteractive; new usable items implement IHandItem; new content is added as ScriptableObject assets. No existing class needs to change.
NFR402	Maintainability (Test Coverage)	Plain C# classes; interface-based design	Inventory and CraftingSystem are plain C# classes, not MonoBehaviours, so NUnit can test them without a Unity scene. The service interfaces enable mock injection in integration tests.
NFR501	Portability (Multi-Platform)	Unity, URP, New Input System	Unity builds natively to Windows, Linux, and WebGL from one codebase. URP renders consistently across all three. The New Input System abstracts platform-specific input, requiring no per-platform code.
NFR502	Portability (Rendering Pipeline)	Universal Render Pipeline (URP)	URP was chosen over the Built-in Pipeline (deprecated) and HDRP (too heavy). It supports cross-platform shaders and custom post-processing (e.g. LowResViewport) without HDRP's hardware requirements.
NFR103	Performance (Frame Rate)	URP; event-driven UI; Component-Based Architecture	URP has a lower per-frame GPU cost than HDRP. The event-driven UI only runs when events like OnSlotChanged fire, never polling each frame. Component-Based Architecture avoids monolithic Update() methods.
NFR301 NFR302 NFR303	Reliability (Save System)	Binary serialization; ScriptableObject asset refs	Binary serialization keeps file sizes small (NFR104). Items are stored as ScriptableObject references rather than copies, ensuring consistency across sessions. Auto-save triggers on key game events, not only on shutdown.
NFR201 NFR202 NFR203	Usability (Feedback & Learnability)	GetActionPromptText(); event-driven UI	The Use/Interact abstraction reduces controls to two input types, lowering cognitive load (NFR203). Each node provides its own prompt via GetActionPromptText() (NFR201). The event-driven UI updates instantly on state changes, keeping upgrade indicators accurate (NFR202).

Table 37: NFR-to-Architecture Traceability

3 Domain Analysis

Below is the domain model diagram for the problem domain of our project. Due to the nature of this project, the domain is modeled using an entity-relationship diagram, which captures the key entities and their relationships within the system. The diagram provides a high-level overview of the main components and their interactions, which is essential for understanding the structure of the problem domain.

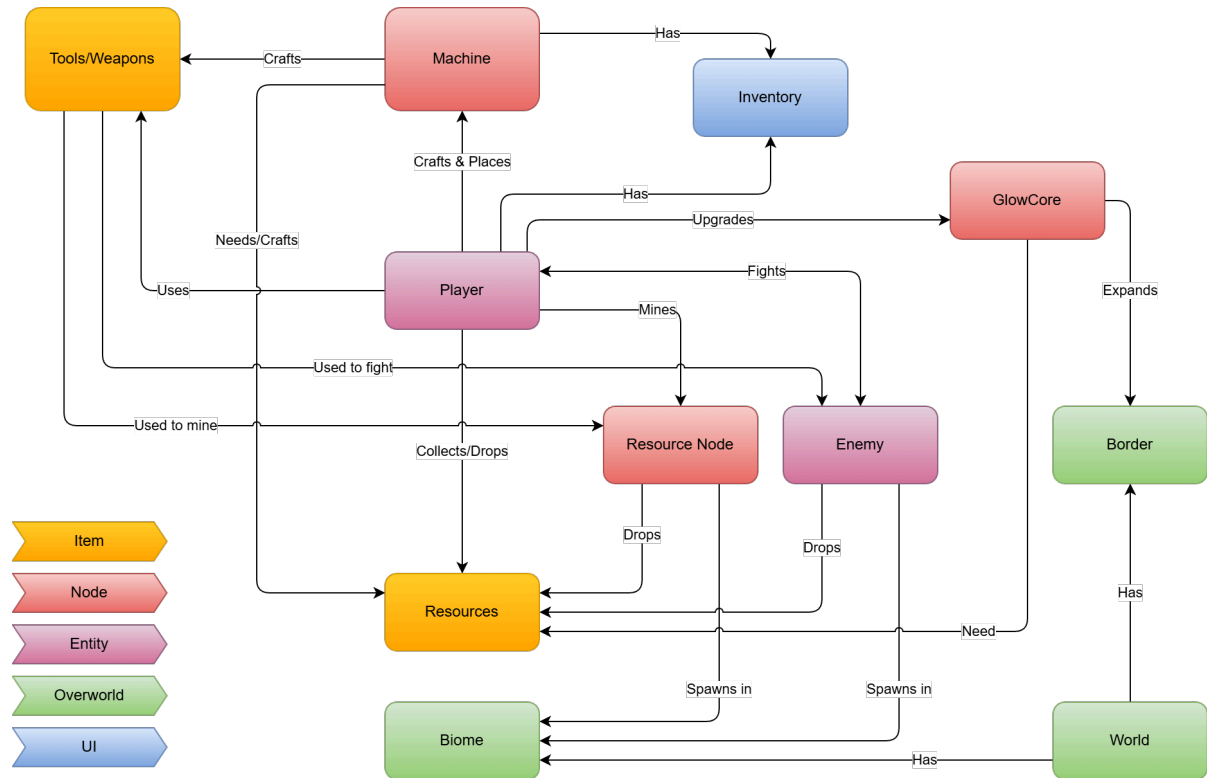
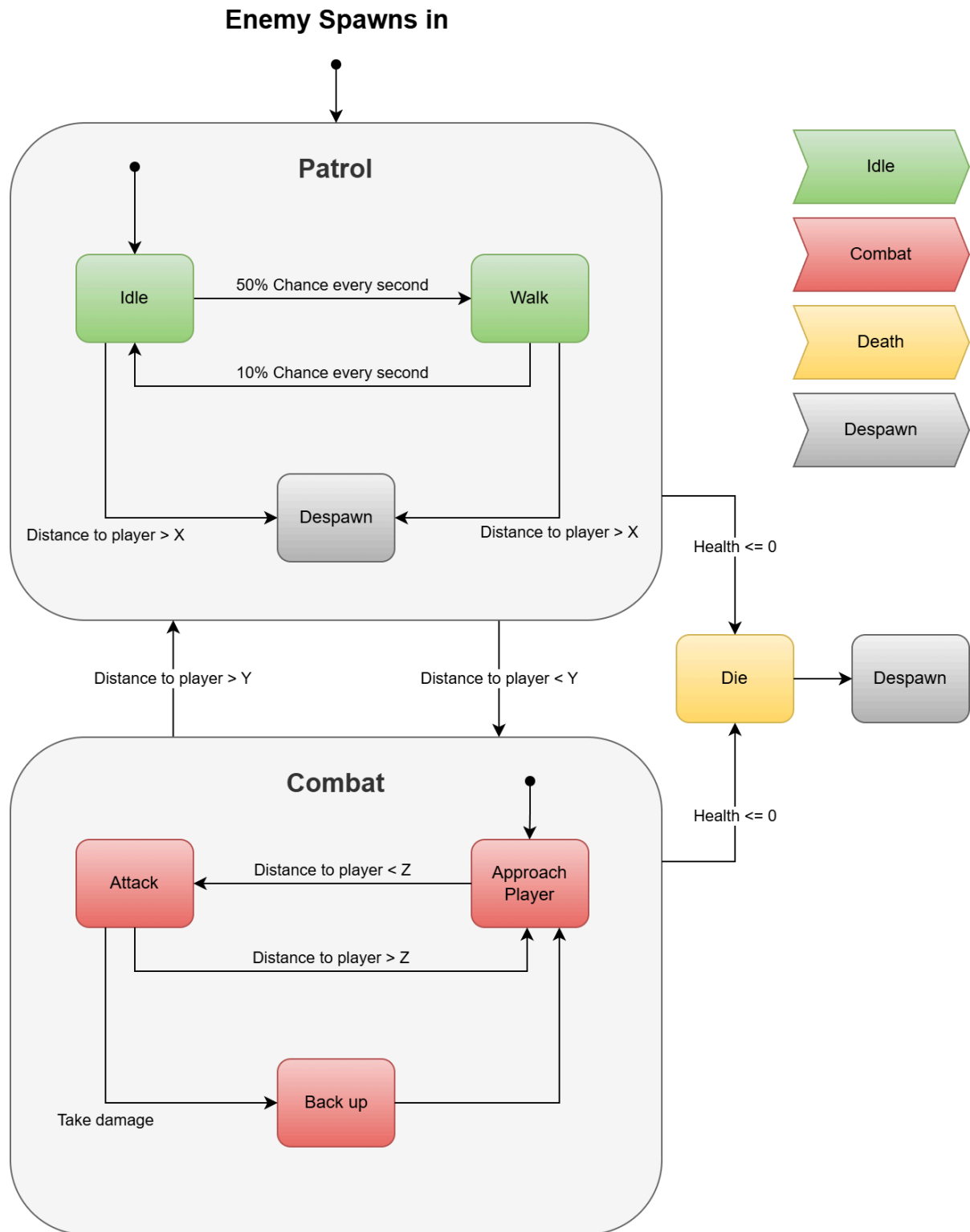


Image 2: Domain Model Diagram

One of the key aspects of the domain is the relationship between “Nodes” and “Items”. Some items can be placed in the world which turns them into Nodes. This means that the same concept can have different representations and behaviors depending on the context.

3.1 Enemy State Machine

The below diagram represents the state machine for the enemy behavior in our project. It captures the various states an enemy can be in as well as the transitions between those states and the conditions that trigger those transitions.

**Image 3:** Enemy State Machine Diagram

4 Architecture

4.1 Frontend Architecture

The user interface of GlowCore is designed to be self-explanatory. Therefore, only selected screens are explained in detail below.

4.1.1 Main Menu



Image 4: Main Menu

The Main Menu is the entry point of the game. The player can continue with an existing save of the game or create a new one. The player can only have one save at a time.

4.1.2 New Game



Image 5: New Game

4.1.3 Pause Menu



Image 6: Pause Menu

In the Pause Menu the player can save the game manually. The game is also automatically saved when the player exits to the Main Menu.

4.1.4 Audio Settings

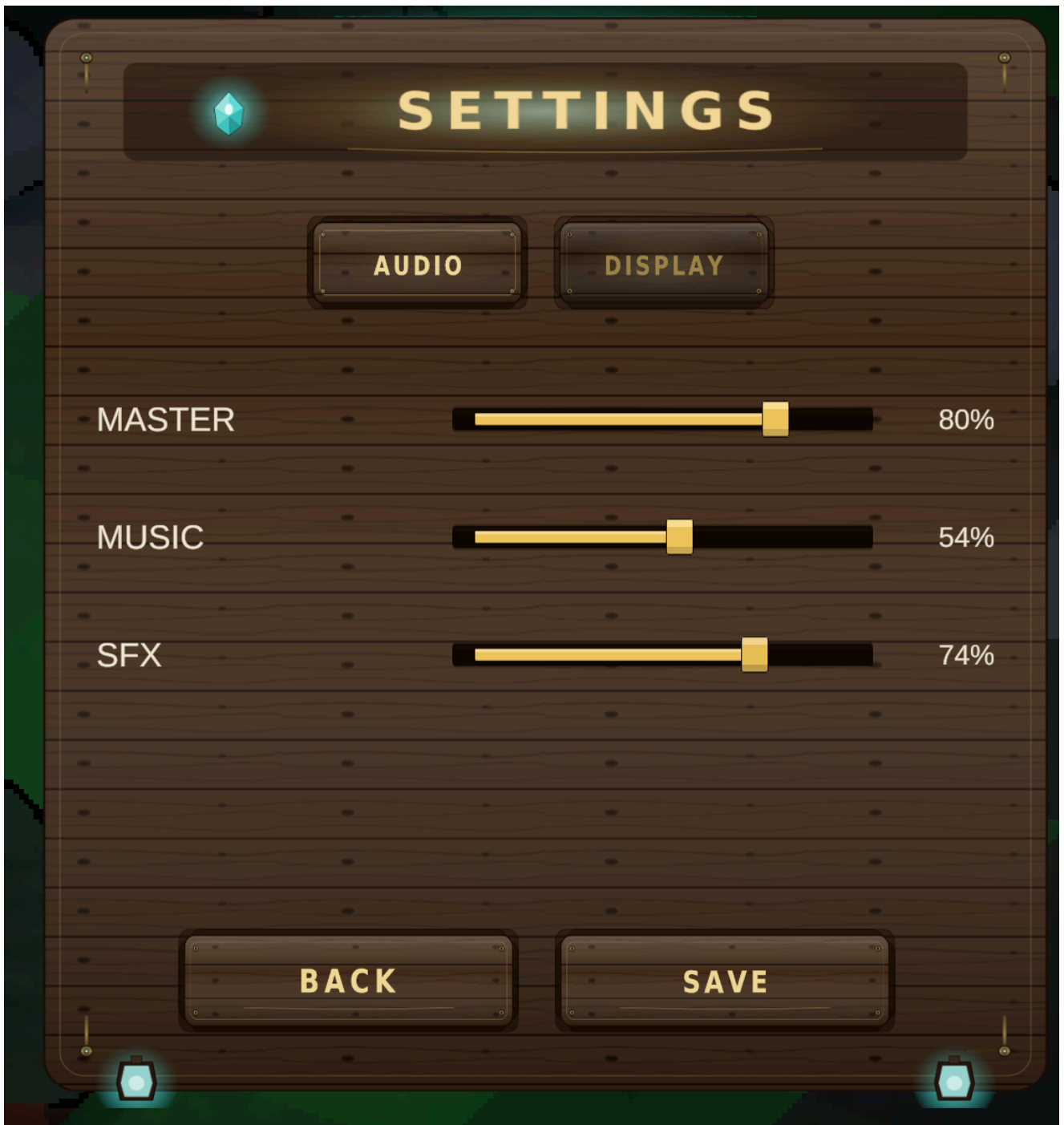


Image 7: Audio Settings

4.1.5 Display Settings

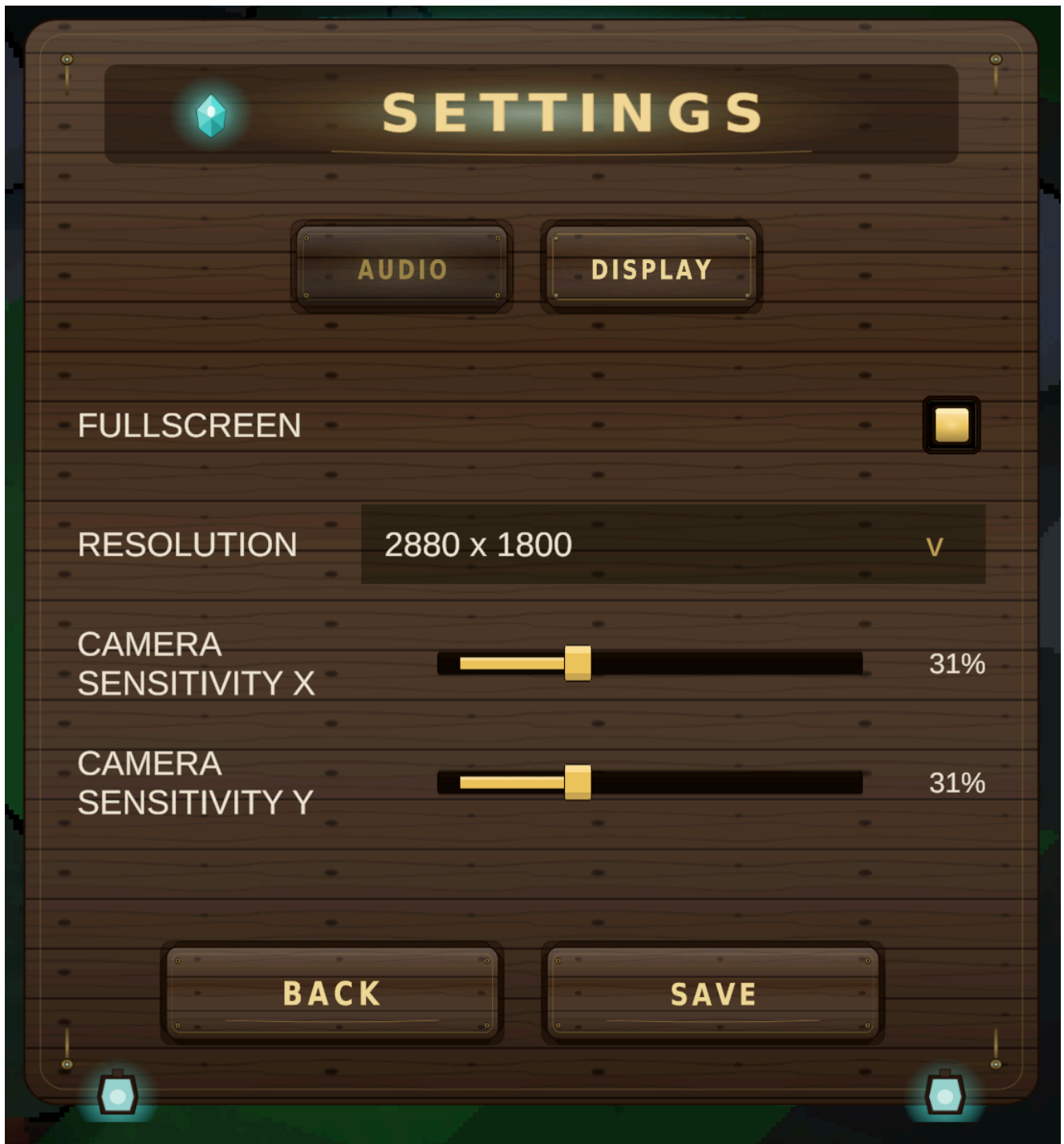


Image 8: Display Settings

The appropriate screen size is automatically detected and selected when the game starts.

4.1.6 Unapplied Changes



Image 9: Unapplied Changes

When the player changes settings without applying them, this confirmation dialog is displayed. It prevents confusion by ensuring that unsaved changes are clearly communicated.

4.1.7 Inventory



Image 10: Inventory

The item stacks can be drag-and-dropped within the inventory. By dragging an item outside the inventory panel, the player drops it into the world. The player can also craft certain items in the inventory without the need for a crafting station.

4.1.8 Crafting Table



Image 11: Crafting Table

4.1.9 More Crafting Stations



Image 12: More Crafting Stations

Crafting Tables, Furnaces, Anvils etc. are abstracted into crafting stations that share a common UI but with different backgrounds and different available crafting recipes. This design follows our extensibility principle and allows new crafting stations to be added easily.

4.1.10 GlowCore Upgrade



Image 13: GlowCore Upgrade

4.1.11 Chest



Image 14: Chest

4.2 C4 Model

To model the architecture of GlowCore we decided to use the C4 Model. The model is designed to provide stakeholders new to the project a high-level overview of GlowCore with different levels of abstraction. We decided against arc42 because it is too extensive for this project.

We intentionally omitted the Code Diagram (Layer 4 of C4) in this documentation because of how much complexity it would add without providing much value. This is common in project management to omit the Code Diagram.

4.2.1 System Context Diagram

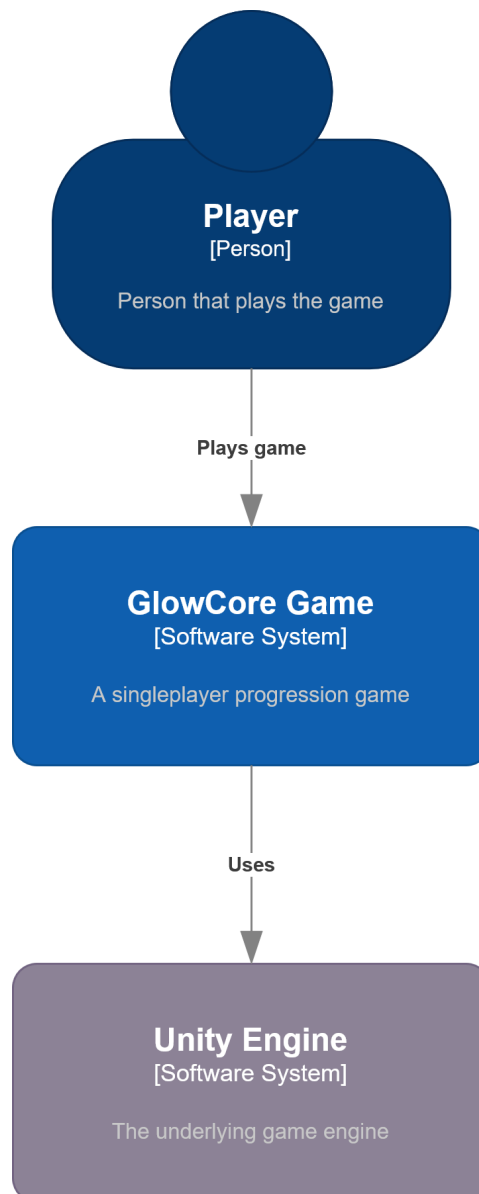


Image 15: C4 System Context Diagram

4.2.2 Container Diagram

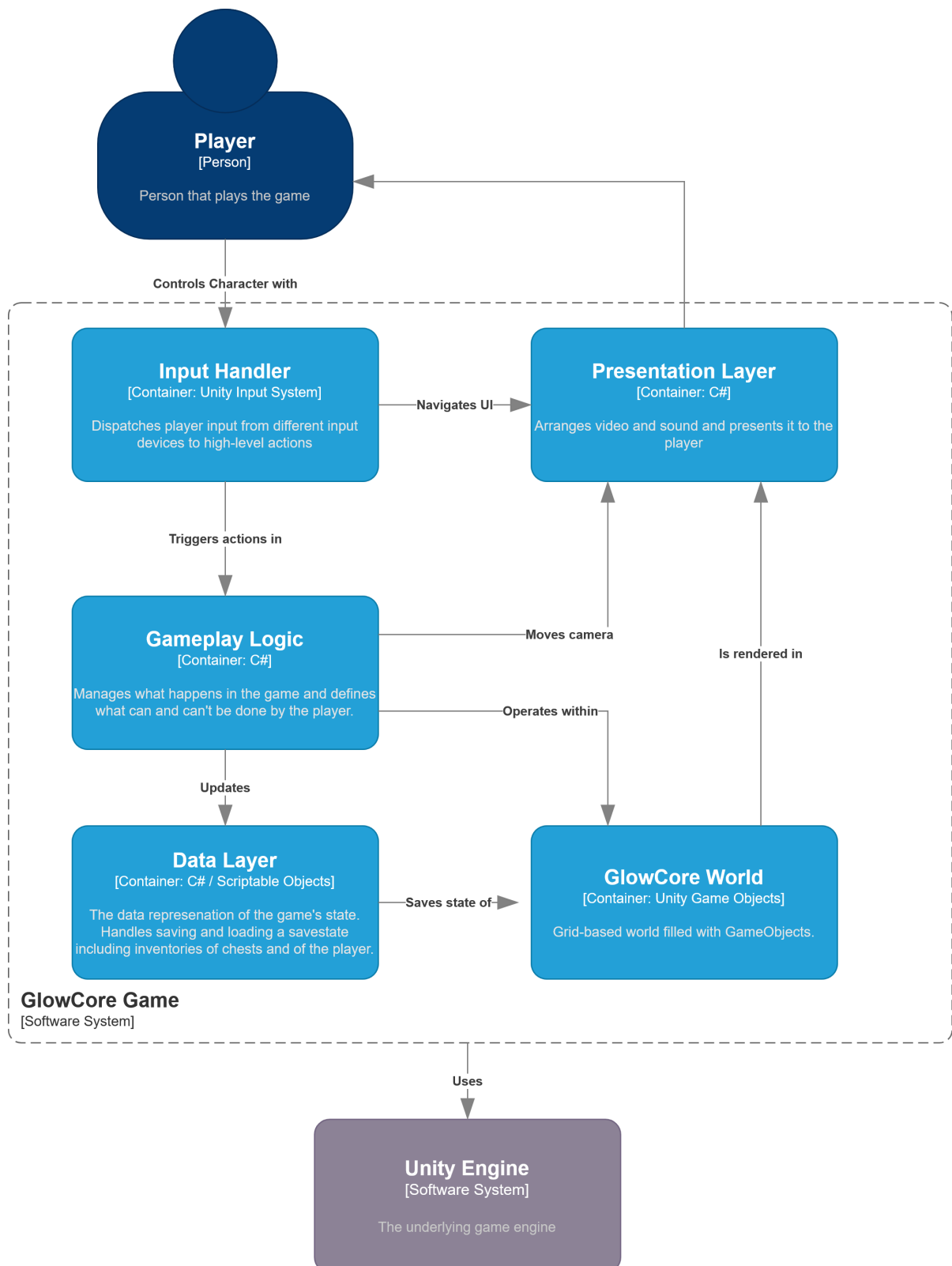


Image 16: C4 Container Diagram

4.2.3 Component Diagram

The Component Diagram zooms into selected Containers to provide a deeper overview.

4.2.3.1 Gameplay Logic

A naive approach to implement features like Breaking Nodes, Building, Inventories and Crafting would be to just implement them separately, each with its own isolated input handling and execution logic. This would result in duplicated code and poorly maintainable code.

Instead, we analyzed the features and abstracted them into two groups: **Use Actions** and **Interact Actions**.

Use Actions are responsible for using the Item in the players hand (The item that is selected in the hotbar and visually displayed in the players hand). The player can only break Nodes when he holds a tool in his hand and he can only place Nodes if he holds a Block in his hand.

Interact Actions define the behaviour of hovered over Nodes. For example the player opens a chest by hovering over it with the cursor and pressing the corresponding button. The GlowCore and all crafting stations share the same input to avoid code duplication. This approach also makes our game extensible.

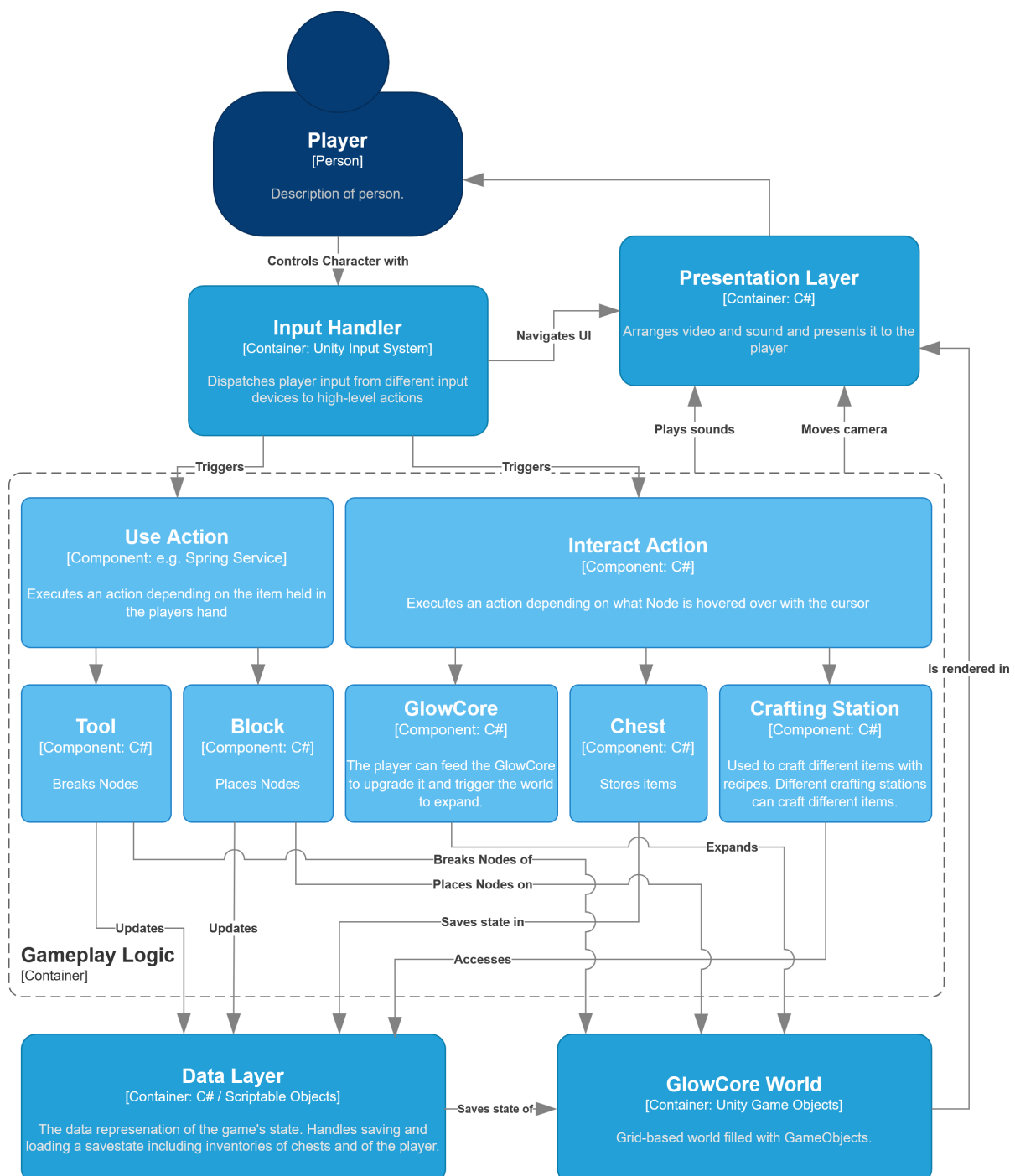


Image 17: C4 Component Diagram - Gameplay Logic

4.2.3.2 Presentation Layer

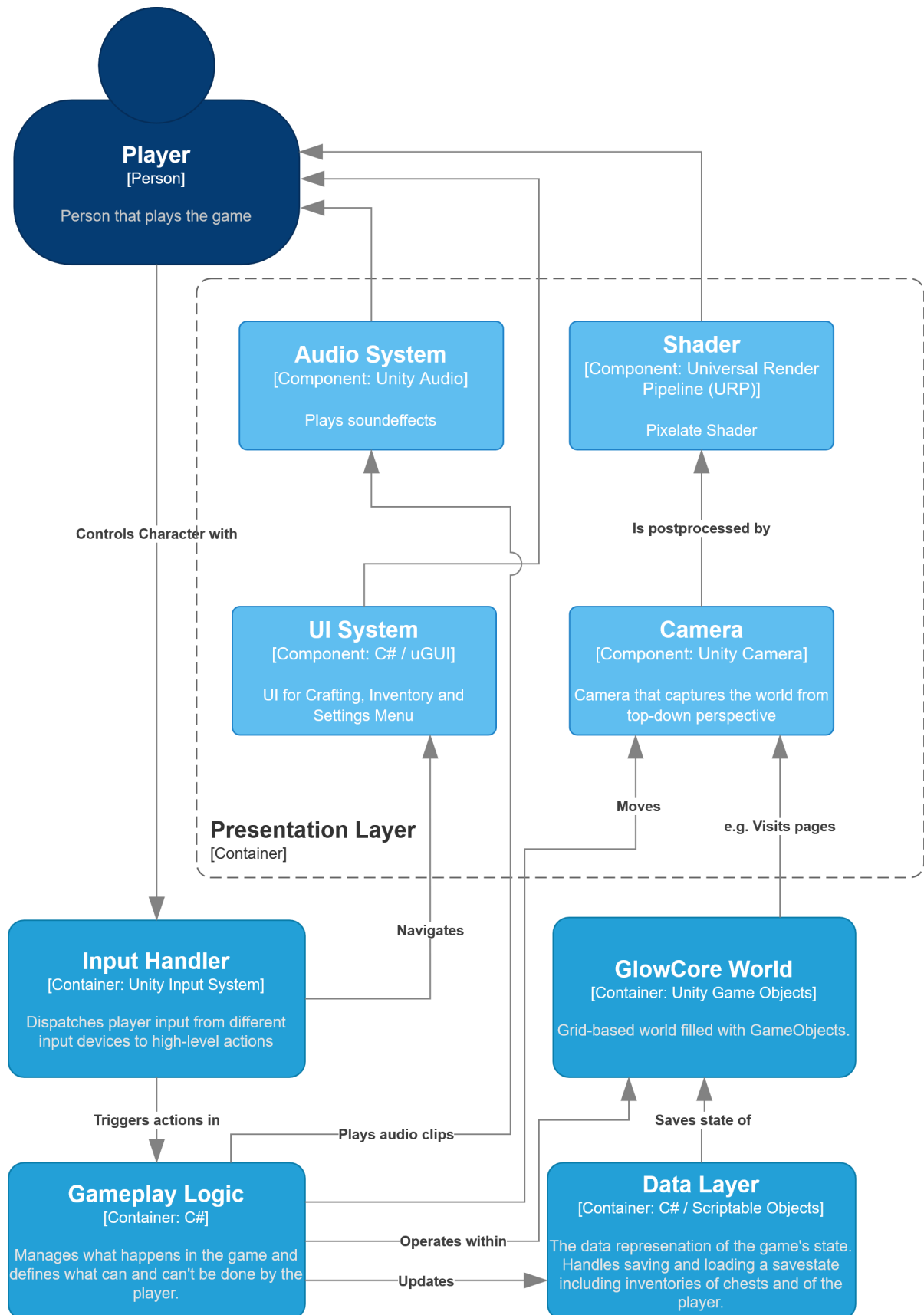


Image 18: C4 Component Diagram - Presentation Layer

4.3 Interfaces

GlowCore's container boundaries are expressed through either a C# interface or a ScriptableObject asset reference, with the exception of the input boundary where Unity's own message dispatch handles the connection directly.

4.3.1 Overview Diagram

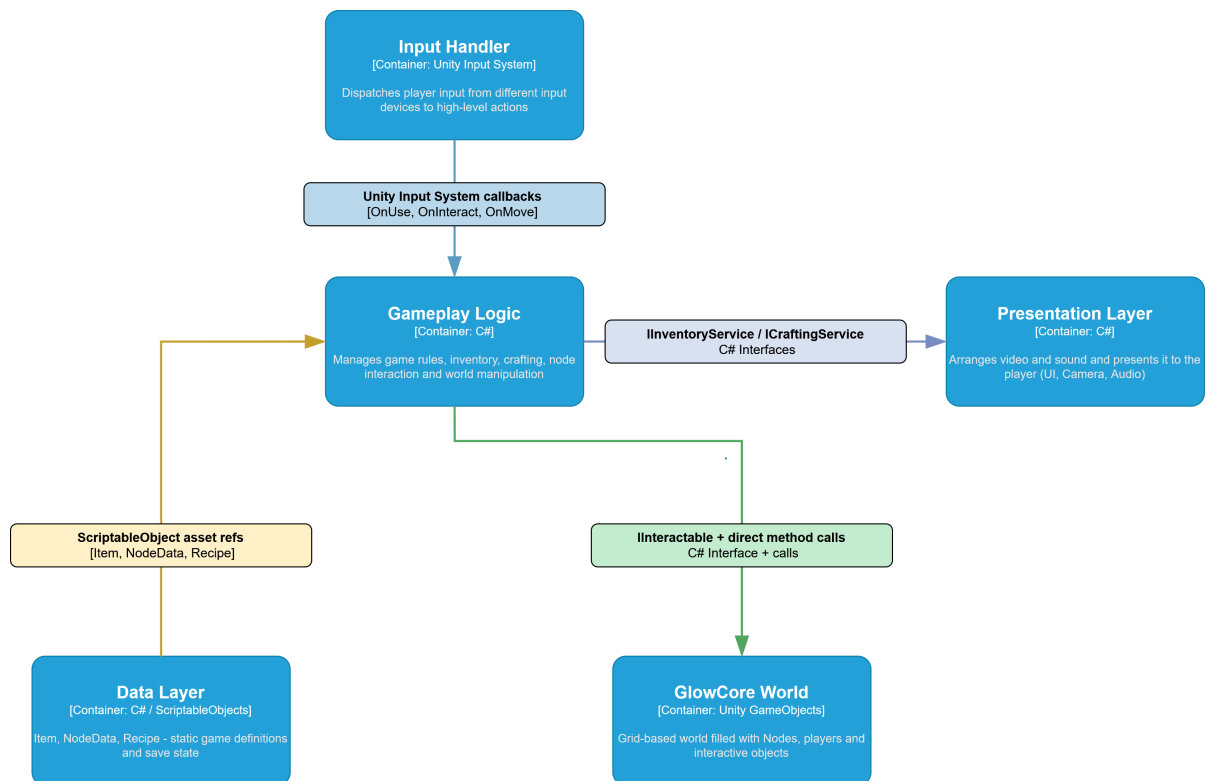


Image 19: Interface Overview Diagram

4.3.2 Interface Contracts

Interface <i>Boundary Mechanism</i>	Purpose
Container Boundaries	
Unity Input callbacks <i>Input Handler → Gameplay Logic</i> Unity Input System (no C# interface)	Unity calls methods like <code>OnUse</code> , <code>OnInteract</code> and <code>OnMove</code> directly on <code>MonoBehaviour</code> components such as <code>PlayerHand</code> and <code>NodeActionSystem</code> . There is no C# interface at this boundary. Unity's own message dispatch handles the connection.
IInteractable <i>Gameplay Logic → GlowCore World</i> C# interface	Nodes that do something when the player interacts with them implement this interface. A Chest opens its inventory, a <code>CraftingTableInteractable</code> shows the crafting UI, and <code>GlowCoreObject</code> runs its own logic, all through the same single-method contract.
IInventoryService <i>Gameplay Logic → Presentation Layer</i> C# interface + events (<code>Action<T></code>)	The main contract between the inventory system and the UI. Components like <code>InventoryUI</code> and <code>HotbarUI</code> subscribe to events such as <code>OnSlotChanged</code> and <code>OnHotbarSelectionChanged</code> to stay in sync with the game state, without ever referencing <code>PlayerInventory</code> directly.
ICraftingService <i>Gameplay Logic → Presentation Layer</i> C# interface + event (<code>Action</code>)	Gives the crafting UI everything it needs: the current recipe list, availability checks, the <code>Craft()</code> command to actually execute a recipe, and the <code>OnRecipesRefreshed</code> event that triggers a refresh whenever the player's inventory changes.
ScriptableObject asset refs <i>Data Layer → Gameplay Logic</i> Unity serialised reference (<code>[SerializeField]</code>)	<code>Item</code> , <code>NodeData</code> and <code>Recipe</code> assets are set up in the Unity Inspector and read at runtime by the components that need them. There is no code dependency; the connection is entirely through serialised Unity references.
Gameplay Logic - Internal Component Boundaries	
IHandItem <i>PlayerHand → Item Prefab (GL internal)</i> C# interface	Any item that can be used while held in the player's hand implements this interface. When the player presses the use button, <code>PlayerHand</code> calls <code>Use()</code> on whichever component is currently active, without caring whether it is a <code>ToolBehaviour</code> , a <code>BlockBehaviour</code> , or anything else.
IPlayerInventoryAware <i>PlayerHand → Item Prefab (GL internal)</i> C# interface	Some item prefab components need access to the player's inventory to function correctly. This interface lets <code>PlayerHand</code> pass them a reference when the item is equipped, avoiding any need for a singleton lookup.

Table 38: Interface Contracts

4.3.3 Extensibility through Stable Interfaces

These interfaces are what makes NFR401 achievable in practice. Each interface was designed so that adding something new only requires work in one place.

Adding a new usable item means implementing `IHandItem` on a prefab component. Adding a new interactive node means implementing `IInteractable`. In both cases nothing else in the system needs to change: the Input Handler, the Presentation Layer, and the Data Layer are completely unaware of the new addition.

Extending the item and crafting catalogue is just as simple. A new `Item` or `NodeData` asset can be created as a `ScriptableObject`, filled in through the Unity Inspector, and referenced wherever it is needed, without touching any C# code at all. The same applies to crafting recipes: a new `Recipe` asset is all that is needed to make a new combination available in the crafting UI.

The inventory and crafting UI are equally easy to extend. Because `InventoryUI` and `HotbarUI` depend only on `IInventoryService`, and `CraftingTableUI` only on `ICraftingService`, a new UI component for either system can be added by subscribing to the same events and use given methods, without modifying any existing class.

4.4 Technologies

4.4.1 Unity

The most important decision was the selection of the game engine. We decided to use Unity because we had the most experience in it and all members have already learned C#. Had our game been 2D and not 3D, Godot would have been our preferred choice.

4.4.2 Universal Render Pipeline (URP)

Unity provides 3 rendering pipelines:

- **Built-in Render Pipeline:** Legacy render pipeline
- **Universal Render Pipeline (URP):** Lightweight scriptable render pipeline.
- **High Definition Render Pipeline (HDRP):** For realistic surfaces.

We chose URP because the HDRP would be overkill for our project and the Built-in Render Pipeline is deprecated and new projects should use URP instead. URP is perfectly suitable to create custom shaders and viewports.

4.4.3 .NET & C#

Unity's default scripting language is C#, so this decision came bundled with the selection of the game engine. Although it is possible to use Plugins to run other languages such as e.g. C++, there are no significant benefits, so we settled on C#.

Unity uses Mono, a .NET runtime to create cross-platform applications. We use NUnit via the Unity Testing Framework for automated testing.

4.4.4 GitHub

We use GitHub for version control. At the beginning of the project we used GitLab, but we switched to GitHub because we experienced problems setting up the CI/CD Pipeline to work. The documentation on GitLab was limited, since most tutorials and examples are focused on GitHub Actions.

4.4.5 Blender

To create our 3D Models we use Blender. Blender is free and open-source and offers all features of its competitors, so the choice was obvious. The workflow to import 3D Models from Blender into Unity is great and preserves all important informations.

4.4.6 Audacity

We use Audacity to record and edit our soundeffects. Audacity is free, easy to use and provides all necessary tools for simple sound design, so there was no need to choose a more sophisticated audio editing software.

4.5 Used Design Patterns

4.5.1 Component-Based Architecture

We use Unity's Component-Based Architecture, where every `GameObject` has Components that define its behaviour. To implement custom Components, we use MonoBehaviour-Scripts to define the logic of a `GameObject`. This approach increases modularity because it is possible to split the logic into different Components and reuse them.

We don't have a real **Entity Component System (ECS)** because we don't separate the logic from the data. To support a real ECS we would need to use Unity's **Data-Oriented Technology Stack (DOTS)**. For our project the additional complexity of DOTS would outweigh its performance benefits.

4.5.2 Singleton

We use the Singleton Pattern where appropriate. In our game it definitely makes sense because we have systems that only exist once. For example there is only one Player and only one World-Grid.

4.5.3 Dependency Injection

Games often have tightly coupled systems. To counter this we use dependency injection wherever appropriate to decrease coupling.

4.5.4 Scriptable Objects

In our game we have to define and store data about Items and Nodes in a organized way. Meant are the Item/Node-Definitions, not the instances at runtime. For example for the Wood-Item we need to store its ID, 3D Model, Max Stack-Size etc..

Scriptable Objects solve this problem. Scriptable Objects are objects that are not bound to a `GameObject`. Instances of ScriptableObjects can be saved as **.asset** files. This improves Version Control for Items/Nodes since they are saved as individual files.

4.5.5 Prefabs

We use Prefabs to save specific `GameObjects` as **.prefab** files. This enables us to dynamically Instantiate Prefabs as `GameObjects` during runtime, which is necessary for e.g. Building.

4.6 Extensibility

Our game is designed to be extensible and exchangeable wherever feasible. Thanks to our ScriptableObject Architecture we can easily add new Items/Nodes just by creating a new Instance of the corresponding ScriptableObject and setting the parameters and referencing the 3D Model and Icon.

Thanks to the Component-Based Architecture we can easily add new Items that perform a specific action when used by the players. It would be easy to add for example Consumables. An additional benefit that it is easy to add Nodes with custom behaviour for when the player interacts with it. For example you could just create a Signpost component and add it to a Node to create a Node where you can write something on it and read it.

We save our 3D Models separate from the ScriptableObjects in the .fbx format to improve exchangeability. It is possible to only exchange the 3D Model without touching the ScriptableObject.

A major limit of our architecture that we are strongly tied to Unity. Switching to another game engine would require a significant effort due to how our systems are relying on Unity's systems.

4.7 External Dependencies

Our game is distributed via Steam, which is our only external dependency. Steam is responsible for installing, launching and updating the game on the end-user side. We also use Steam for marketing.

We create the store page for our game on Steamworks, where we can configure the description, upload marketing material etc. The executable is uploaded using Steamworks SteamPipe, which handles distribution and versioning.

4.8 Game Design

This chapter covers Gameplay Decisions rather than technical solutions, focusing on how the game is structured to create an entertaining experience. Game design can be understood as the application of user experience principles within the domain of game development.

4.8.1 Core Gameplay Loop

The core gameplay loop of GlowCore can be broken down into the following steps:

1. Gather resources
2. Craft items
3. Upgrade tools
4. Build Machines
5. Upgrade the GlowCore, which unlocks new areas and items
6. Repeat loop

4.8.2 Gameplay Progression

The following sections describe the gameplay progression in chronological order and explain the design decisions and their reasons.

4.8.2.1 Early Game

The player starts with a basic wooden axe and gathers wood by manually cutting trees. Early trees can be chopped quickly but only give a small amount of wood. Over time, larger trees appear which require significantly more time to cut but also give more wood.

4.8.2.2 Automation Phase 1

At this point, manual gathering becomes inefficient, encouraging the player to craft their first axe-machine. This machine is placed in front of a tree and automatically cuts it over time. While the machine is operating, the player is free to perform other actions, such as gathering additional resources to craft more machines.

This creates a strong feeling of progress and a passive progression reward, since the player knows that progress is being made even while he is not directly interacting with the machine. This effect is reinforced as the player uses gathered resources to construct more machines, amplifying the loop.

When returning to the machine, the player collects the gathered resources and moves the machine to a new tree. This is acceptable at this stage because repositioning requires significantly less effort than manual tree cutting.

4.8.2.3 Automation Phase 2

Over time, the player reaches a point where moving so many axe-machines becomes tedious. At this stage, a new system is introduced: The replanter block. The replanter automatically places saplings. This allows trees to regrow automatically, removing the need to constantly reposition machines. However, this introduces a trade-off, as machines remain inactive while waiting for trees to regrow. This adds strategic depth to the system.

4.8.2.4 Automation Phase 3

Later in progression, a new tree type is introduced that has short cut time and drops a relatively big amount of wood but has a very long growth time. To improve efficiency the player crafts the newly unlocked axe-machine Tier 2, which moves automatically into the direction its facing and harvest trees sequentially.

At this stage, multiple replanter blocks can be used per machine, enabling more complex automated setups. A key design decision is that replanter blocks must be significantly cheaper than Tier 2 machines, encouraging players to build efficient production lines.

4.8.2.5 Full Automation

However, this introduces a new problem: machine orientation and placement still require manual adjustment. This is resolved by the introduction of rotator blocks. When a machine collides with a rotator block, it automatically rotates by

90, 180, or 270 degrees depending on the block type. This enables fully automated resource gathering systems and allows players to design complex production paths.

4.8.3 Automation Constraints

A core design principle in this system is that not everything should be fully automatable. If all actions were automated, the player would just watch the machines and the player becomes bored. Instead, the player must still actively gather certain resources in order to craft and expand automation systems.

4.8.4 Depth and Complexity

We followed the principle of providing as much depth as possible with as little complexity as possible. The goal of this design is to allow the player to create complex and interesting systems without introducing a large number of different mechanics and complicated rules.

In GlowCore, this is achieved by limiting the core automation system to only three main components: the axe-machine, the replanter, and the rotator. When combined, these simple systems interact in ways that allow for emergent complexity, such as automation lines with different layouts.

4.9 Key Architectural Decisions

The following decisions had the greatest impact on the structure and long-term maintainability of the game. Each one is recorded together with the problem it solved and the reasoning behind the chosen approach.

4.9.1 ScriptableObjects as the Data Layer

Decision: All item, node, and recipe definitions are stored as ScriptableObject .asset files rather than in C# code or external data files (e.g. JSON).

Why: Game content, items, nodes, crafting recipes, must be defined once and consumed from many places without creating code dependencies. ScriptableObjects let non-programmers create and configure new content entirely through the Unity Inspector. Each asset is an individual file, which keeps version control diffs small and conflict-free. Crucially, the connection to gameplay code is a serialised Unity reference, so adding a new item, node, or recipe never requires touching any C# source file.

4.9.2 Interactable for World Interaction

Decision: Every node that responds to player interaction implements the IInteractable interface. NodeActionSystem calls Interact() on whichever node is currently hovered, without knowing its concrete type.

Why: Without a shared interface, NodeActionSystem would require an explicit branch for every interactive object type: chests, crafting stations, the GlowCore, signs, and anything added later. With IInteractable the input handler is permanently closed to modification on this axis, adding a new interactive node is purely additive. The GetActionPromptText() method on the same interface also lets each node control its own UI prompt label, removing another potential switch statement from the core input loop.

4.9.3 InventoryService and Event-Driven UI

Decision: The inventory system is exposed to all UI components exclusively through the IInventoryService interface, which carries both commands and Action<T> events (OnSlotChanged, OnHotbarSelectionChanged, etc.).

Why: Inventory state is modified from many independent sources: item collection, crafting, chest transfers, and GlowCore upgrades. A polling-based UI would either check every frame or miss updates; direct method calls from each source to each UI component would create a tightly coupled web of dependencies. Events mean every UI component reacts exactly when something relevant changes, at zero cost when idle. Because all UI depends only on the interface and not on PlayerInventory directly, the inventory implementation can change freely without any UI class breaking.

4.9.4 Use Actions and Interact Actions

Decision: Instead of implementing each player capability (breaking nodes, placing blocks, opening chests, crafting) as an isolated system with its own input handling, all player inputs are routed through exactly two abstractions: **Use Actions** (what the held item does, via IHandItem) and **Interact Actions** (what the hovered node does, via IInteractable).

Why: The two abstractions map directly to the two roles the player's cursor plays: the held item and the targeted node. Routing them separately through PlayerHand and NodeActionSystem keeps input handling in exactly one place per axis and avoids duplicating raycast logic, inventory checks, and UI-open guards. Adding a new usable item only requires implementing IHandItem on its prefab; adding a new interactive node only requires implementing IInteractable. Neither addition changes any existing class.

4.9.5 IPlayerInventoryAware via Explicit Injection

Decision: Item prefab components that need access to the player's inventory receive it through IPlayerInventoryAware.SetInventory(), called by PlayerHand at equip time, not through a singleton or a FindObjectOfType lookup.

Why: Tools and blocks need to read and modify the inventory at use-time (e.g. consuming the block stack when placing). Fetching `PlayerInventory.Instance` inside a prefab component hides the dependency, makes the coupling invisible to readers, and makes the component untestable in isolation. Passing the reference explicitly at equip time makes the dependency visible, eliminates hidden singleton coupling, and allows the component to be tested with any `PlayerInventory` instance, including a mock.

4.10 Deployment

The game has two deployment paths, both automated through GitHub Actions. Which path runs depends on what is pushed to the repository.

4.10.1 Continuous Deployment - WebGL (GitHub Pages)

Every push to main or dev triggers the WebGL build job. All jobs run on a self-hosted Linux runner with Unity 6000.3.9f1 installed directly on the machine. The Unity Editor is invoked in batch mode via a custom BuildScript, which avoids the overhead of spinning up a Docker container on every run. The license is activated per-run using `buildalot/activate-unity-license`. The resulting build artifact is deployed to the pages branch via `JamesIves/github-pages-deploy-action`. Builds from main land in `prod/webgl`; builds from dev land in `dev/webgl`. This gives stakeholders an always-up-to-date playable version of the game in a browser without any manual steps.

4.10.2 Release Deployment - Windows & Linux (GitHub Releases)

Pushing a version tag (e.g. `v1.0.0`) triggers the Windows and Linux build jobs in parallel, both running on the same self-hosted runner. Each job invokes the Unity Editor directly in batch mode, targeting `StandaloneWindows64` and `StandaloneLinux64` respectively. Once both builds complete, a release job downloads the artifacts, zips them, and publishes them as a GitHub Release with auto-generated release notes. Players can download the zipped executables directly from the GitHub Releases page.

4.10.3 Steam

Once the project is ready to be published to Steam, the release pipeline will be extended with a `SteamPipe` upload step after the Windows and Linux builds. The `steamcmd` CLI can authenticate using Steamworks credentials stored as GitHub Actions secrets and upload the build depot to Steam. Steam would then handle CDN distribution to all players who own the game. No structural change to the existing pipeline would be required.

4.10.4 Deployment Diagram

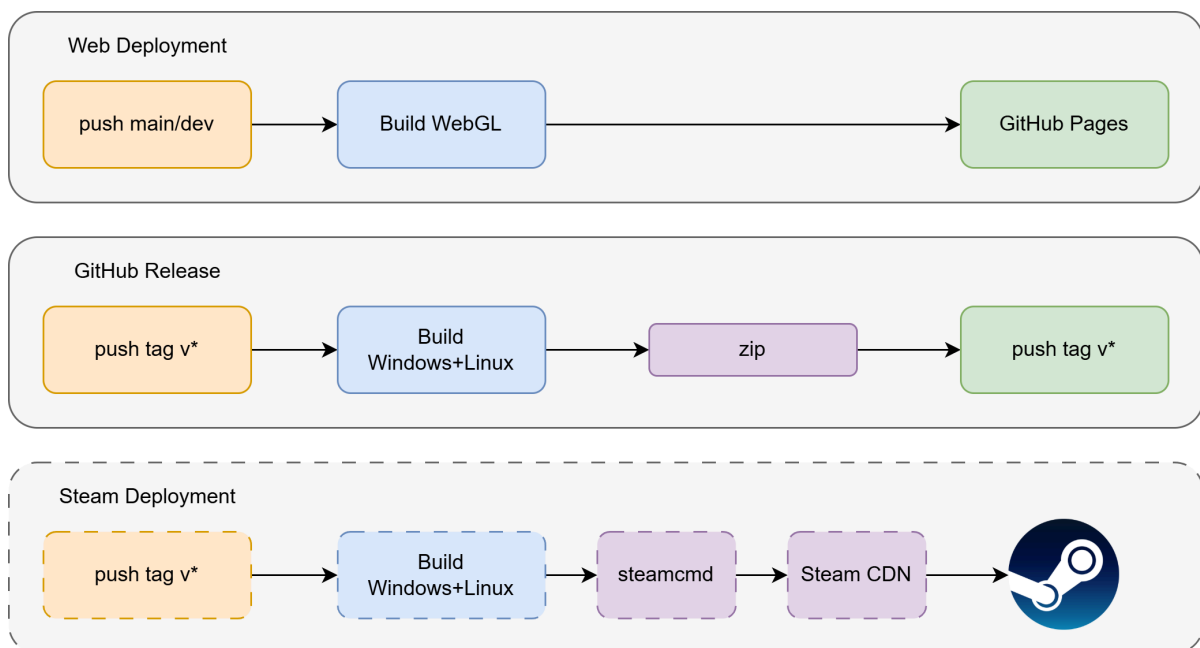


Image 20: Deployment Diagram

5 Quality Measures

5.1 Working Environment

5.1.1 CI/CD Pipeline

The project uses GitHub Actions for all build, test, and deployment automation. Five pipelines are defined, each with a distinct responsibility:

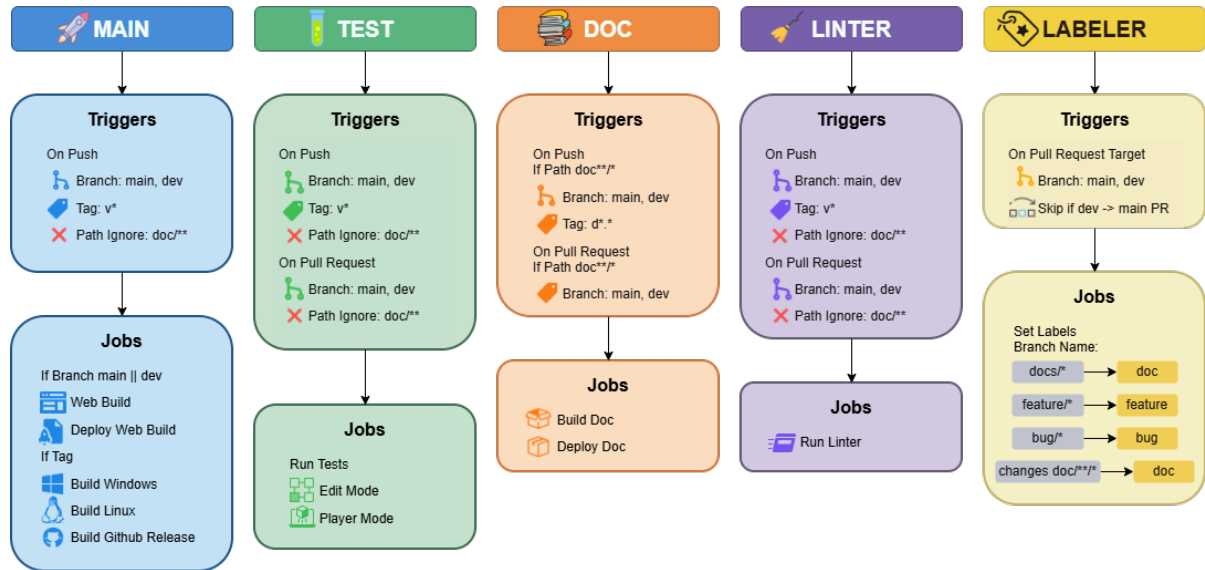


Image 21: CI/CD Pipeline Overview

- **MAIN:** runs on every push to main or dev (excluding doc/** changes) and on version tags (v*). Builds and deploys the WebGL build for pushes to the two main branches. On a version tag it additionally compiles Windows and Linux standalone builds and creates a GitHub Release.
- **TEST:** runs on push to main or dev and on pull requests targeting either branch (both excluding doc/**). Executes the Unity Test Framework suite in both Edit Mode and Play Mode and generates a report on both results and coverage.
- **DOC:** triggered only when doc/** files are modified: on push to main or dev, on doc tags (d*. *), and on pull requests that touch the documentation. Builds the Typst documentation and deploys it to GitHub Pages.
- **LINTER:** runs on the same push and pull-request triggers as TEST (excluding doc/**). Runs the Roslyn code-analysis and dotnet format checks described in the Coding Guidelines section. A failed lint check blocks the merge.
- **LABELER:** fires on pull-request targets to main or dev (skipped for dev → main promotion PRs). Automatically applies labels to each PR based on branch prefix and changed paths (feature/* → feature, bug/* → bug, docs/* or changes under doc/**/* → doc).

5.1.2 Definition of Ready and Definition of Done

Every Jira ticket follows a structured lifecycle governed by our Definition of Ready (DoR) and Definition of Done (DoD). A ticket may only be pulled into a sprint once all DoR criteria are met, including a clear description, defined acceptance criteria, story point estimate, epic assignment, and team sign-off during sprint planning. A ticket is only considered done once all DoD criteria are satisfied, covering code quality, CI/CD pipeline status, test coverage, PR approval and conditionally -> UI consistency and documentation updates.

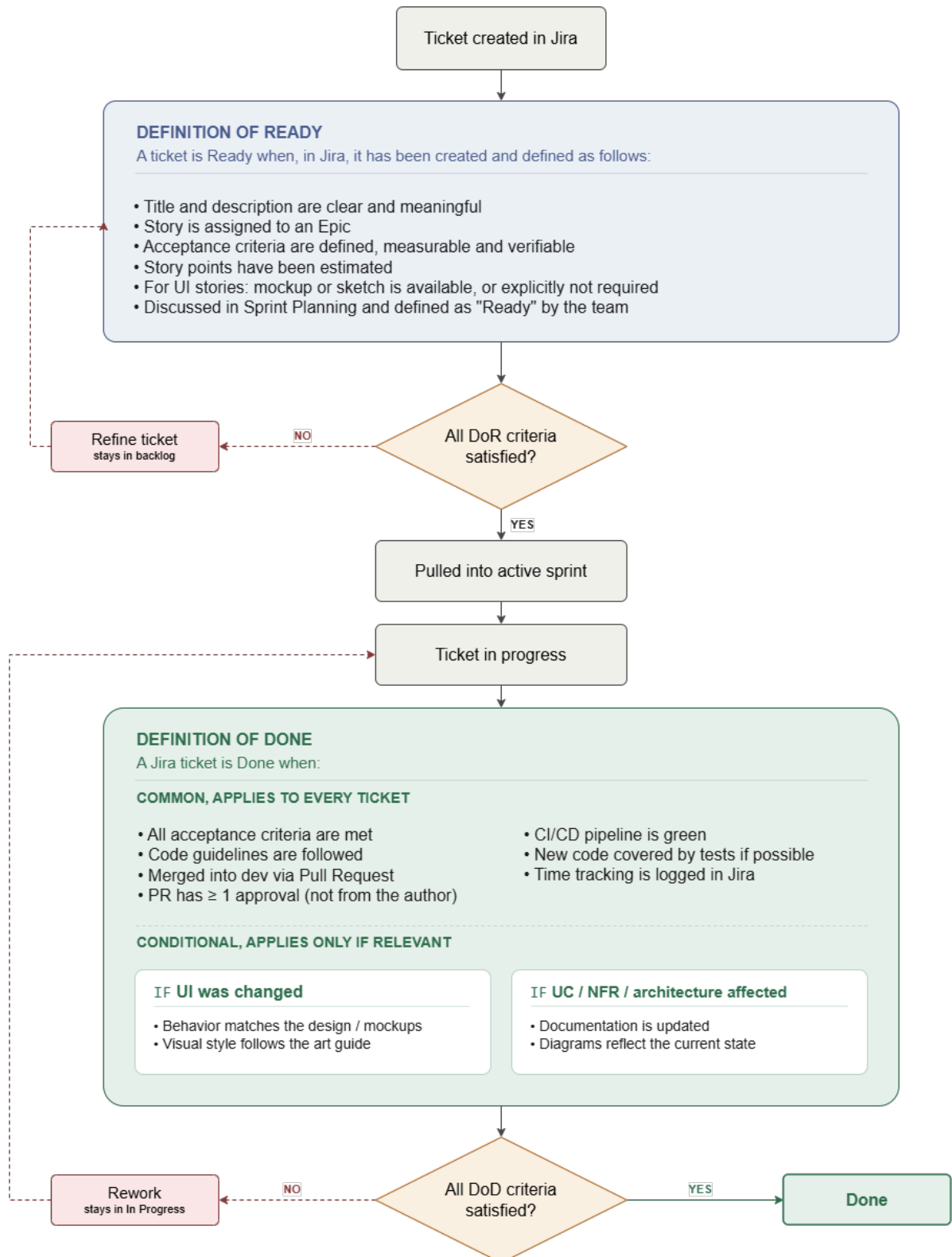
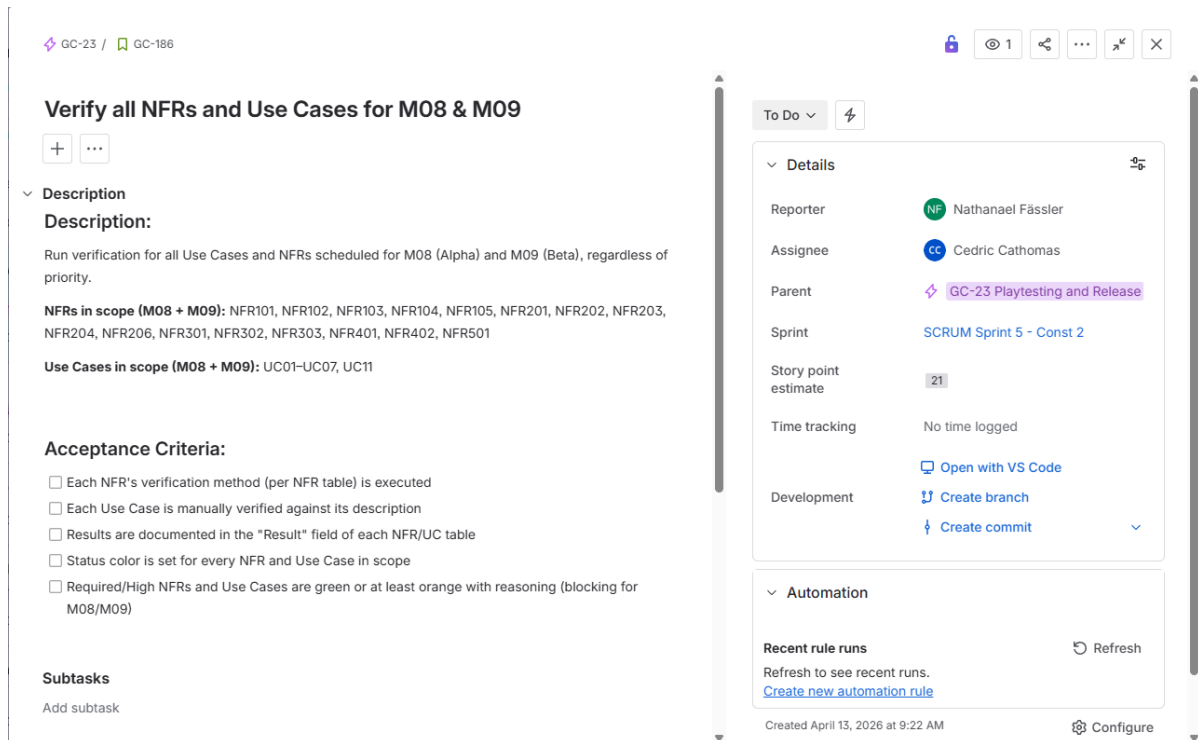


Image 22: Definition of Ready and Definition of Done - ticket lifecycle flowchart

5.1.2.1 DoR Example

The following example shows a real ticket (GC-186) that satisfies all DoR criteria and is ready to be worked on.



GC-23 / GC-186

Verify all NFRs and Use Cases for M08 & M09

+ ...

Description

Description:

Run verification for all Use Cases and NFRs scheduled for M08 (Alpha) and M09 (Beta), regardless of priority.

NFRs in scope (M08 + M09): NFR101, NFR102, NFR103, NFR104, NFR105, NFR201, NFR202, NFR203, NFR204, NFR206, NFR301, NFR302, NFR303, NFR401, NFR402, NFR501

Use Cases in scope (M08 + M09): UC01-UC07, UC11

Acceptance Criteria:

- ☐ Each NFR's verification method (per NFR table) is executed
- ☐ Each Use Case is manually verified against its description
- ☐ Results are documented in the "Result" field of each NFR/UC table
- ☐ Status color is set for every NFR and Use Case in scope
- ☐ Required/High NFRs and Use Cases are green or at least orange with reasoning (blocking for M08/M09)

Subtasks

Add subtask

To Do ⚡

Details

Reporter: Nathanael Fässler

Assignee: Cedric Cathomas

Parent: GC-23 Playtesting and Release

Sprint: SCRUM Sprint 5 - Const 2

Story point estimate: 21

Time tracking: No time logged

Development: [Open with VS Code](#), [Create branch](#), [Create commit](#)

Automation

Recent rule runs Refresh

Refresh to see recent runs.
[Create new automation rule](#)

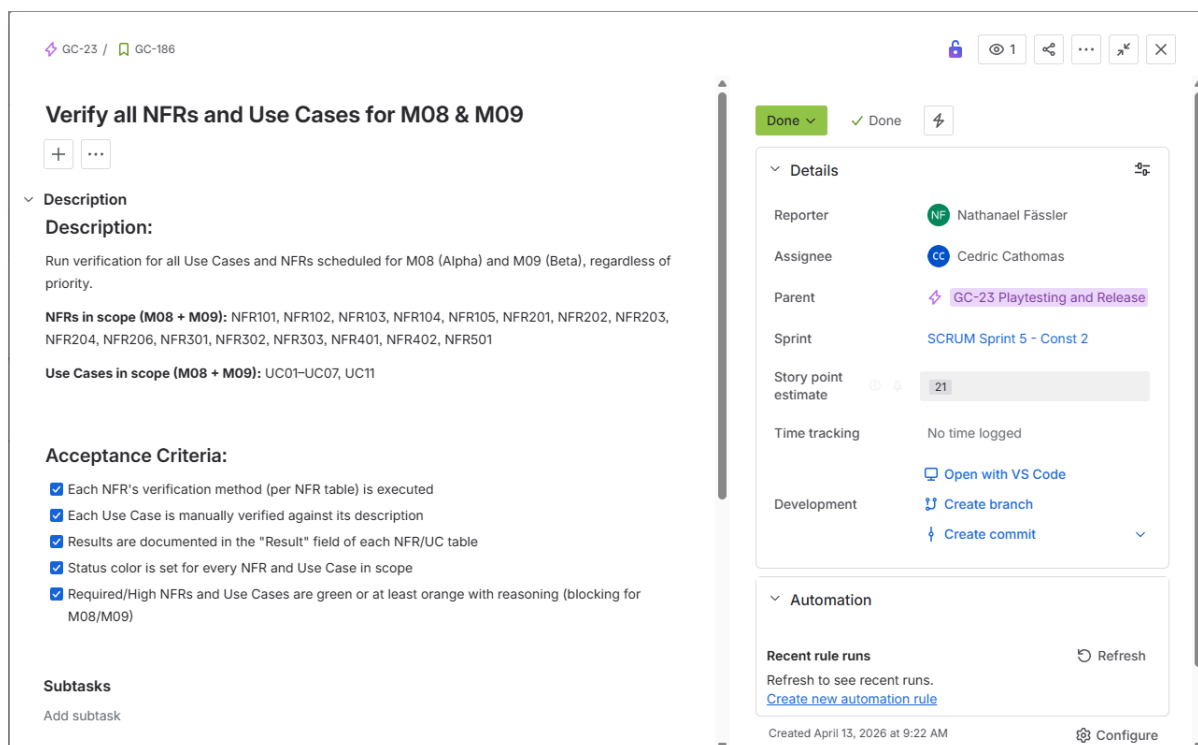
Created April 13, 2026 at 9:22 AM
 Updated 13 seconds ago

Configure

Image 23: GC-186 - Example of a ticket satisfying all Definition of Ready criteria

5.1.2.2 DoD Example

The following example shows the same ticket (GC-186) with all acceptance criteria checked and status set to Done.



GC-23 / GC-186

Verify all NFRs and Use Cases for M08 & M09

+ ...

Description

Description:

Run verification for all Use Cases and NFRs scheduled for M08 (Alpha) and M09 (Beta), regardless of priority.

NFRs in scope (M08 + M09): NFR101, NFR102, NFR103, NFR104, NFR105, NFR201, NFR202, NFR203, NFR204, NFR206, NFR301, NFR302, NFR303, NFR401, NFR402, NFR501

Use Cases in scope (M08 + M09): UC01-UC07, UC11

Acceptance Criteria:

- ☒ Each NFR's verification method (per NFR table) is executed
- ☒ Each Use Case is manually verified against its description
- ☒ Results are documented in the "Result" field of each NFR/UC table
- ☒ Status color is set for every NFR and Use Case in scope
- ☒ Required/High NFRs and Use Cases are green or at least orange with reasoning (blocking for M08/M09)

Subtasks

Add subtask

Done ✓ Done ⚡

Details

Reporter: Nathanael Fässler

Assignee: Cedric Cathomas

Parent: GC-23 Playtesting and Release

Sprint: SCRUM Sprint 5 - Const 2

Story point estimate: 21

Time tracking: No time logged

Development: [Open with VS Code](#), [Create branch](#), [Create commit](#)

Automation

Recent rule runs Refresh

Refresh to see recent runs.
[Create new automation rule](#)

Created April 13, 2026 at 9:22 AM
 Updated 13 seconds ago

Configure

Image 24: GC-186 - Example of a ticket satisfying all Definition of Done criteria

5.2 Test Concept

Our testing strategy for GlowCore is fundamentally shaped by the nature of the project. Since GlowCore is a standalone Unity game built for Windows, Linux, and WebGL, we don't have any server connections, huge amount of data in a database or a frontend beyond simple game-menus. This allows us to focus our entire testing effort directly on the client-side application, ensuring gameplay mechanics, performance, and user experience are as solid as possible.

The test concept is divided into automated testing for our codebase, manual system verification, extensive usability testing, and targeted checks for our non-functional requirements (NFRs).

5.2.1 Automated Testing (Unit & Integration)

At the code level, we rely heavily on the Unity Test Framework (UTF). We use it to write both unit tests and integration tests that verify whether individual functionalities and interconnected systems are working correctly.

A core part of our workflow is our CI/CD pipeline. Our UTF tests are set up to run automatically on the pipeline every time someone pushes new code. Because these tests run continuously on the latest codebase, they effectively act as our regression tests. They give us immediate feedback if a new feature or bug fix accidentally breaks an older, already working part of the game. We are aiming for a minimum of 40% automated test coverage across our systems, which is a pretty standard and realistic target for game development projects.

5.2.2 System Testing

We don't do automated system tests. Fully automating the entire game loop in Unity is just very difficult. It usually results in tests that break way too easily, for example, just because a small visual asset was moved slightly.

Instead, system testing is handled manually. For this, we regularly take the automated WebGL build generated from the current pipeline state and compare it directly against a known, stable release build. This manual side-by-side verification helps us ensure that the game as a whole still compiles, runs, and functions as expected before we consider a new version stable.

5.2.3 Usability Testing

There is a strict limit to what we can test automatically in a game project. A script might tell you if a player's health goes down correctly, but it can't tell you if the game is actually fun or if the controls make sense. Because of this, usability testing is a massive aspect of our QA process.

We will conduct regular playtesting sessions where we let several new users play the game without giving them any prior instructions. Our main goal here is to find out what feels good, what is frustrating, and if the core mechanics are actually understandable. For instance, we want to see if a brand-new player can figure out how to move, gather resources, and upgrade within the first 5 minutes. We will also observe if players naturally understand the visual cues for restricted actions (like red-tinted UI elements) or if they can accurately track their light upgrade progress.

5.2.4 Non-Functional Requirements (NFR) Verification

NFR tests are not done once at the end, but at appropriate points throughout the project. Each NFR is assigned to a milestone at which it must be verified. Additionally, our Definition of Done requires that any change which could affect an NFR is accompanied by a re-verification of that NFR and an updated result in the documentation. This ensures NFR results stay current and are not just a snapshot from a single test session.

A huge portion of our testing isn't just about finding bugs, but verifying our NFRs. We have defined a wide spectrum of NFRs covering different quality attributes. They are sorted by priority, meaning that while our ultimate goal is to fulfill all of them, the ones marked as "Required" are strict targets we have to hit for the final submission.

We cover NFR testing through a mix of automated benchmarks and manual playtesting protocols:

- **Performance Efficiency:** We regularly check our performance manually to make sure the game runs smoothly. We aim to ensure that the world loads quickly, in-game scene transitions are fast, and the game maintains a solid 60 FPS on recommended hardware.
- **Reliability:** Since saving the game correctly is critical, we manually test save and load cycles to ensure data doesn't corrupt. We also verify that save files remain small enough (under 5MB), that they are fully portable between machines, and that auto-saves trigger correctly.
- **Portability & Maintainability:** We manually verify that the game runs identically across Windows, Linux, and WebGL, and that it supports standard 16:9 resolutions. From a code perspective, we constantly test extensibility during normal development. We frequently add new elements like resource nodes, items, or recipes based on existing systems. If our internal architecture was too complicated to extend, we would notice it very quickly.
- **Usability & Visuals:** Beyond the playtesting mentioned earlier, we also manually review the game's language to ensure all text is in English. We test controller and keyboard inputs by simply playing with both, and we hold art reviews to make sure all new assets fit our low-poly pastel style guide.

By combining the automated safety net of our UTF pipeline with heavy manual playtesting and strict NFR benchmarking, we believe this concept will keep the game stable while still allowing us to iterate quickly on the gameplay.

5.3 Unity Testing

Testing a game is a bit different from testing a "standard" software project, like a web-based accounting tool or a simple CRUD application. In a typical software project, you are mostly concerned with whether a specific input leads to a specific output in a database or a UI. You can easily mock external dependencies and call functions in isolation.

In Unity, however, our code doesn't just sit there. It lives inside an engine that is constantly ticking. We have to deal with the "Game Loop," physics calculations, and the complex lifecycle of MonoBehaviours. To test a full cycle in the Unity engine, we are able to use Edit Mode and Play Mode.

5.3.1 Edit Mode Tests

Edit Mode tests are the closest thing we have to "traditional" unit tests. They run entirely within the Unity Editor and do not require the game to actually start. Since they don't have to load scenes or wait for the physics engine to initialize, they are fast.

Edit Mode tests can be used for our "pure" logic things like calculating resource costs, managing inventory math, or utility functions that don't depend on the game's frame rate. If a bug appears in our math, an Edit Mode test will catch it in during our CI/CD run without us ever having to open a game window.

5.3.2 Play Mode Tests

Play Mode tests are where we handle the Unity Engine of the project. Unlike Edit Mode, these tests actually trigger the full game engine. They can load specific test scenes, instantiate Prefabs, and, most importantly, they can run over multiple frames.

This is essential for Games because many mechanics are time dependent. For example, if we want to test if the player really moves on input, we need the game clock to actually run. In Play Mode, we can use `yield return new WaitForSeconds(1);` to let the game simulate for a moment before checking the results. Play Mode tests can be used for:

- Physics interactions
- Component Lifecycle
- Player Input

The downside is that Play Mode tests are significantly slower because they have to "play" the game. However, they give us the confidence that our code isn't just working on paper, but actually performing correctly within the simulated world of the engine.

5.3.3 Coverage Summary

At the current stage of the project (M09 Beta reached, approaching M10 Release), 8 out of 11 Use Cases are fully verified ([UC01](#), [UC02](#), [UC03](#), [UC04](#), [UC05](#), [UC06](#), [UC07](#), [UC11](#)) and 2 have not been verified yet as they are planned for M10 ([UC08](#), [UC10](#)). [UC09](#) (Fight Enemies) is not applicable, since the team decided not to implement combat within the project scope.

For the NFRs, 15 requirements have fully passed ([NFR101](#), [NFR103](#), [NFR104](#), [NFR105](#), [NFR201](#), [NFR202](#), [NFR203](#), [NFR205](#), [NFR206](#), [NFR301](#), [NFR302](#), [NFR303](#), [NFR401](#), [NFR402](#), [NFR501](#)), 1 is partially met ([NFR204](#)), and 2 have not been verified yet as they are scheduled for M10 ([NFR502](#), [NFR503](#)). [NFR102](#) is not applicable since in-game scene transitions were removed from scope. The only remaining partial NFR is input support: keyboard input works for all gameplay actions, but controller bindings are not yet wired up. Adding them is straightforward given the project already uses the new Input System and is planned before the final release.

5.4 Code metrics

5.4.1 Lines of Code

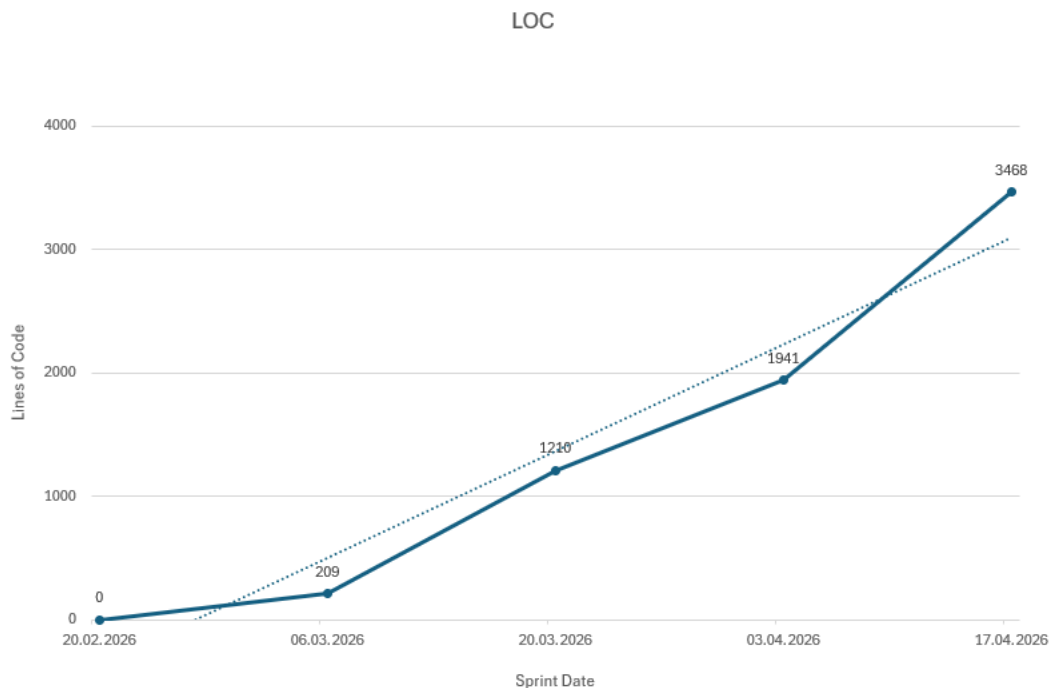


Image 25: Interface Overview Diagram

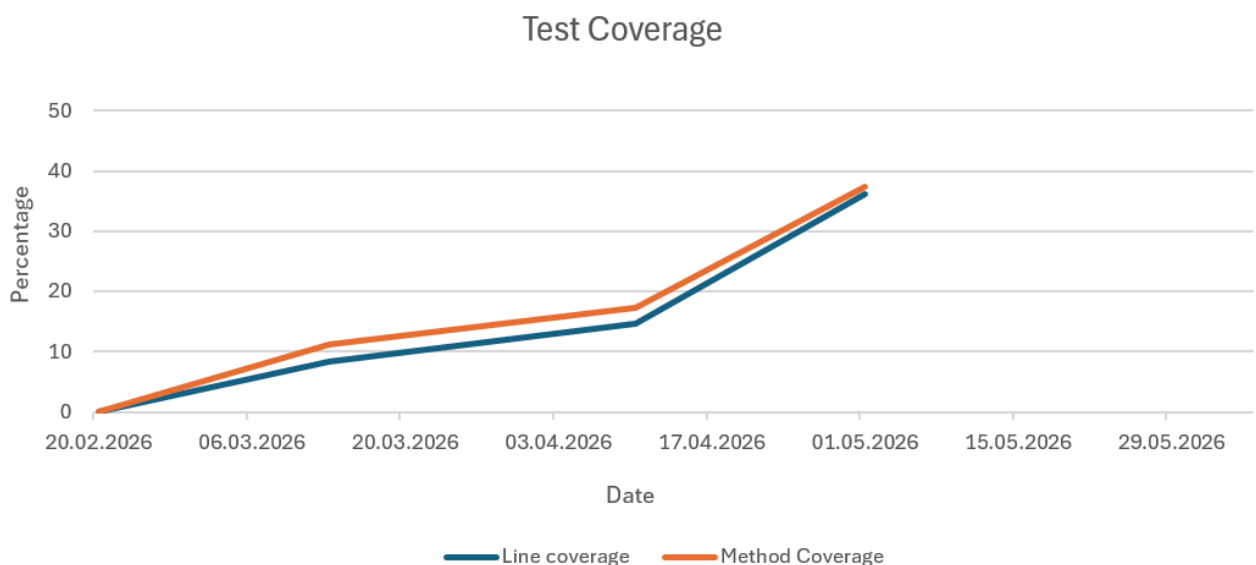
File	Blank	Comment	Code
WorldGrid.cs	87	42	371
InventoryUI.cs	54	0	227
Outline.cs	81	44	217
ReadmeEditor.cs	46	1	195
Inventory.cs	38	0	171
CraftingTableUI.cs	32	5	146
RecipeRowUI.cs	32	0	143
NodeActionSystem.cs	27	1	133
PlayerInventory.cs	32	10	124

File	Blank	Comment	Code
PlayerMovement.cs	8	0	46
CraftingSystem.cs	14	0	45
SlotData.cs	6	0	44
NodeData.cs	6	0	37
LowResViewport.cs	7	0	32
ExampleDamage...	7	0	31
UIColors.cs	7	6	29
InventoryService.cs	4	3	27
ItemIconHelper.cs	4	0	27

File	Blank	Comment	Code
ItemStackDrop.cs	29	1	119
ItemSlotUI.cs	24	1	114
GlowCore.cs	23	4	100
CraftingUI.cs	22	0	91
PlayerIntegra...	21	0	85
Node.cs	13	0	84
HotbarUI.cs	13	0	84
InventoryTests.cs	20	0	83
BlockBehaviour.cs	18	0	77
CraftingTable...	14	3	73
HotbarSlotUI.cs	12	0	71
TooltipUI.cs	16	4	70
ToolBehaviour.cs	9	1	57
Fire.cs	12	4	50
PlayerHand.cs	8	0	50
ItemStack.cs	13	0	48

File	Blank	Comment	Code
PlayerCamera.cs	5	0	23
Recipe.cs	3	0	21
Item.cs	2	0	15
Chest.cs	1	0	14
Readme.cs	2	0	14
ICraftingService.cs	3	3	11
DamageCalculator.cs	2	0	11
Block.cs	1	0	10
RecipeList.cs	2	0	10
TreeNode.cs	1	0	7
NodeActionChild.cs	1	0	6
Tool.cs	1	0	6
IHandleItem.cs	1	0	5
IWeapon.cs	0	0	5
MockWeapon.cs	0	0	5
IPlayerInvent...	0	0	4

5.4.2 Test Coverage



5.5 User Testing

In game development, the most important aspect of the product is that it is entertaining. As developers, we are not representative users for playtesting, since we have spent many hours developing the features and already know how everything works. A first-time player may not immediately understand game rules that are obvious to us. This strongly relates to our usability NFRs, which require the game to be intuitive which is a prerequisite for it to be enjoyable. This is the main purpose of the user tests of GlowCore.

During testing, the participant is given the game on a device with a keyboard and mouse, where no further instructions are given beyond what is provided in the actual game. They are then asked to play the game while commenting on what they

are currently doing. At the end, the participant answers a set of questions to evaluate the corresponding NFRs. The results of the NFRs are documented in their chapter.

5.5.1 User Test 1 (UT01)

- **Participant:** omega-800
- **Gaming Experience / Similar Games:** Has played Factorio
- **Date:** 28.04.2026
- **Tested release:** v0.2.0

Field	Result
NFR201 Player Restriction Visibility. Did you understand the visual clues?	Yes, I especially liked the red and green clues when building.
NFR202 Light Upgrade Progress Indication. Did you understand what causes the map to expand?	Yes, when I feed wood into the fire it grows, and the darkness goes away.
NFR203 New Player Learnability. How long did it take for the first upgrade?	2 minutes.
What was confusing?	I couldn't move the camera vertically.
Did you have fun? What did you like?	Yes, I enjoyed playing it. I liked the visual art style with the outlines.
Was the pacing good, too slow or too fast?	The pacing was good. I had fun and felt motivated to progress. I got bored at level 11 because there were no better tools to craft, nothing to aim for.

Table 39: UT01

5.5.1.1 Additional Feedback

Topic	Explanation
GlowCore UI	He didn't notice that he had to click "Upgrade To Level 2" to progress. At level 14 it was not obvious that he had to scroll down to add coal.
Signs	At first he did not read second sign, assuming that all signs display the same text.
Node hovering	It should be possible to select a Node located behind the player (the raycast should go through the player).
Hotbar	He suggested allowing the player to scroll through the hotbar using the mouse wheel.
Building	He stopped the main task of progressing to build something. This is considered positive, because the game is intended to be open-world to a certain degree.

Table 40: UT01 - Additional Feedback

5.5.1.2 Conclusion

The participant was mainly motivated to progress because he wanted to craft better tools in order to break things faster to progress faster creating a continuous progression loop. This loop is called the core game loop of a game and this is exactly what we wanted to accomplish and what makes the game entertaining ultimately. This user test confirms that the core game loop is engaging.

Based on the feedback we will make improvements to the UI, in particular we will change the GlowCore upgrade UI to be more intuitive.

5.5.2 User Test 2 (UT02)

- **Participant:** Anonymous
- **Gaming Experience / Similar Games:** Has played various game genres including automation games.
- **Date:** 30.04.2026
- **Tested release:** v0.2.0

Field	Result
NFR201 Player Restriction Visibility. Did you understand the visual clues?	Yes, the outlines were obvious.
NFR202 Light Upgrade Progress Indication. Did you understand what causes the map to expand?	Yes, when I upgrade the fire.
NFR203 New Player Learnability. How long did it take for the first upgrade?	3 minutes.
What was confusing?	The border visual is strange.
Did you have fun? What did you like?	It got boring fast, because it's very repetitive. I liked how the crafting table looked. The proportions of the stone-fire and the player were a bit off.
Was the pacing good, too slow or too fast?	The pacing was okay, but I think you should get more wood out of trees.

Table 41: UT02

5.5.2.1 Additional Feedback

Topic	Explanation
Gameplay	The game has potential but there are not enough things to do, not enough content.
Node break prompt	The break prompt should only be displayed for the first few Nodes that the player breaks.
Camera	He suggested that you should be able to zoom in and out a bit with the mouse wheel.
Building bug	When you hold a building block and click somewhere in the UI it still builds.

Table 42: UT02 - Additional Feedback

5.5.2.2 Conclusion

This user test provided valuable feedback on what the gameplay is missing and what it is already doing right. In Sprint 6 we will refine the UI and controls accordingly. We will also add more Items, Nodes and Crafting Recipes. Thanks to our extensible architecture which follows the Open-Closed Principle (OCP), new content can be added without modifying existing code.

5.6 Coding Guidelines

Our coding guidelines are based on the Unity C# scripting conventions and are enforced through Roslyn analyzers and EditorConfig rules. Every C# file must conform to these rules, which are checked automatically on every push to main or dev via the CI pipeline.

5.6.1 Naming Conventions

Element	Convention	Example
Classes / Interfaces	PascalCase; interfaces prefixed with I	PlayerController, ICollectible
Methods	PascalCase	MovePlayer(), TakeDamage()
Bool methods	Question-word prefix	IsAlive(), CanCollect()

Element	Convention	Example
Parameters / Local variables	camelCase	playerSpeed, itemCount
Constants (const)	k prefix + PascalCase	kMaxHealth, kGravity
Private fields	m_ prefix + camelCase	m_playerHealth, m_enemyCount
Public fields	PascalCase (avoid in classes)	PlayerHealth
Properties	PascalCase	Health, Score
Static fields	s_ prefix + camelCase	s_instanceCount
Events	On prefix + PascalCase	OnPlayerDeath, OnItemCollected
Enums / Values	PascalCase	PlayerState.Idle
Namespaces	PascalCase, project-based	GlowCore.Player
Generic type parameters	T prefix + PascalCase	TItem, TState

5.6.2 Class Member Ordering

Members within a class must appear in the following order:

1. Constants
2. Static fields
3. Instance fields
4. Constructors
5. Properties
6. Public methods
7. Private methods

5.6.3 General Guidelines

Properties: For get-only properties, prefer expression-bodied members (e.g. `public int Health => m_health;`). Simple get/set properties may also use expression bodies when the logic fits on one line.

Method parameters: Avoid `in` parameters. The compiler may silently create defensive copies, which is a hidden performance cost.

Access modifiers: Always declare access modifiers explicitly; never rely on C# defaults.

Public fields: Avoid public fields in classes, use properties instead. Public fields in structs are acceptable.

5.6.4 Formatting Rules

These rules are enforced by `.editorconfig` and verified by `dotnet format` in CI. A build with formatting violations is treated as a failure.

Line endings: LF (`\n`) only. CRLF is not permitted in any file.

Indentation: 4 spaces for C# files; 2 spaces for XML/JSON/config files. Tabs are never used.

Brace style (Allman): Opening braces always appear on their own line for all constructs (`if`, `for`, method bodies, class bodies, etc.).

Single-line bodies: `if`, `for`, `while`, and similar statements with a single-line body omit braces; the body is placed on the next indented line.

this. qualifier: Do not qualify field or member access with `this.` unless it is required to resolve an ambiguity.

Language keywords over BCL types: Use `int`, `string`, `bool`, etc. instead of `Int32`, `String`, `Boolean`.

var keyword: Use `var` for built-in types and whenever the type is apparent from the right-hand side of an assignment.

using directives: Placed outside the namespace declaration. `System.*` namespaces are sorted first.

Modifier order: public / private / protected / internal > static > extern / new / virtual / abstract / sealed / override > readonly > unsafe / volatile / async.

Null checks: Prefer null-coalescing (??) and null-conditional (?.) operators over explicit null comparisons where possible.

5.7 Version Control Workflow

The project uses Git with a two-tier branching model:

- **main:** stable production branch. Only receives merges from dev after a release is considered stable.
- **dev:** integration branch. All feature work is merged here first and must pass CI before it is promoted to main.

5.7.0.1 Branch Naming

Every branch must be linked to a Jira issue. Branch names follow the pattern:

`<type>/gc-<issue-id>-short-description`

Where `<type>` is one of:

Type	When to use
feature	New features or significant enhancements
bug	Bug fixes
docs	Documentation updates
task	General tasks that do not fit the above categories

5.7.0.2 Rebasing Workflow

We follow a **rebasing workflow**: before opening a pull request, the feature branch is rebased onto the current tip of dev. This keeps the project history linear and avoids unnecessary merge commits.

If a branch accumulates many small or intermediate commits that do not add meaningful history (e.g. “fix typo”, “WIP”), the branch is squashed before or during merge to keep the log clean.

5.7.0.3 Pull Requests

Pull requests always target dev. Each PR must:

- Reference the corresponding Jira issue in its description.
- Receive **at least one approving review** from a team member before it can be merged.
- Pass all CI checks (linting, formatting, automated tests, and build).

No direct pushes to dev or main are permitted.

6 Savefile

The game periodically saves the player's progress in a savefile. This file contains all the necessary information to restore the game state when the player loads it again. It is stored as a binary file on the player's device. The following diagram illustrates the structure of the savefile:

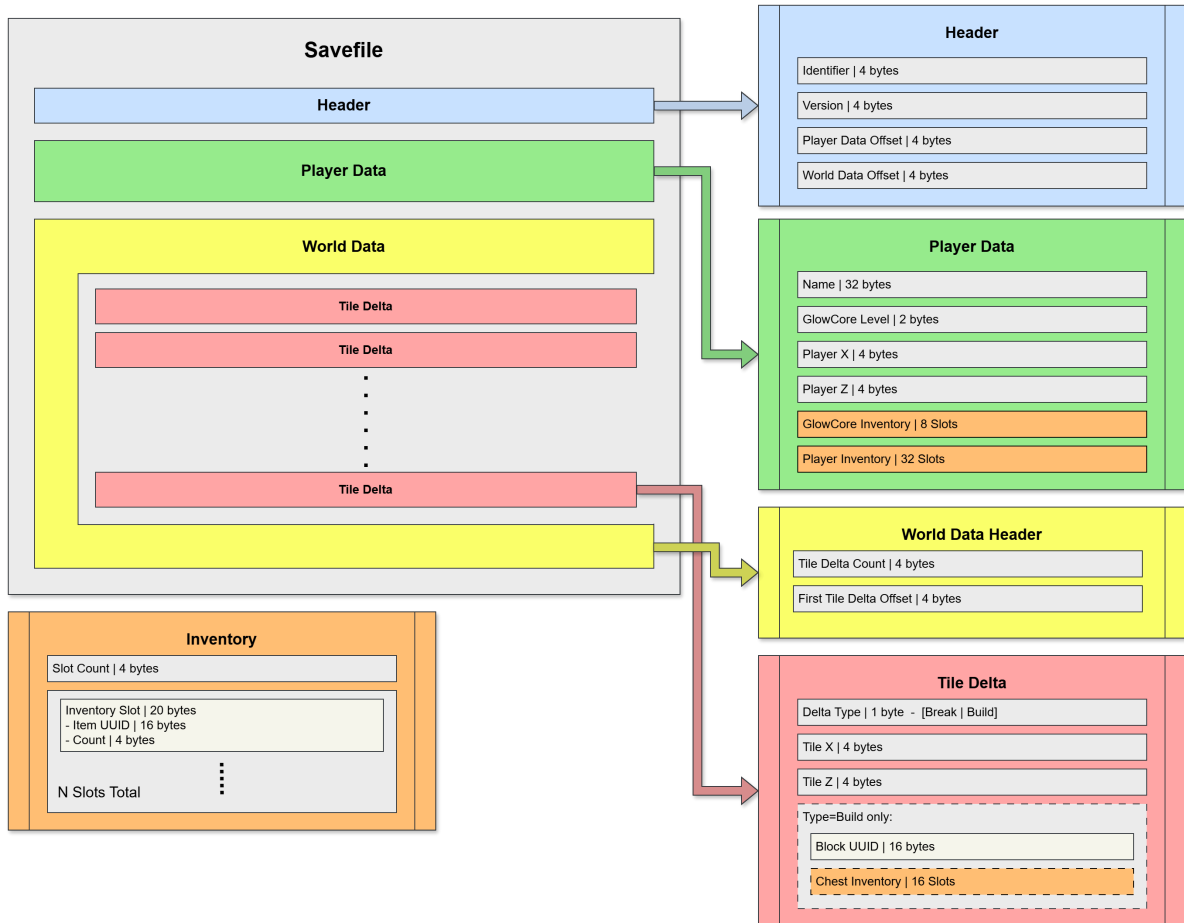


Image 26: Savefile Structure

The savefile is organized into several sections, each containing specific types of data:

- **Header:** Contains metadata about the savefile, such as an identifier, version number, and offsets to subsequent sections.
- **Player Data:** Stores information about the player itself, including name, GlowCore level, inventory, etc.
- **World Data:** Contains information about the game world itself, such as where nodes have been placed or destroyed.

The world data is stored as a list of deltas, instead of a full snapshot of the world state. Each delta represents a change to the world from its initial state, such as placing or destroying a node. This approach allows for more efficient storage and faster loading times, as only the changes need to be recorded rather than the entire world state.

Part III

Project Documentation

7 Initial Project Proposal

Project name: GlowCore

Date: 19.12.2025

7.1 Team Members

1. Cedric Cathomas (Cedric.Cathomas@ost.ch)
2. Dominik Wyss (Dominik.Wyss1@ost.ch)
3. Nathanael Fässler (Nathanael.Faessler@ost.ch)
4. Yoris Kucera (Yoris.Kucera@ost.ch)

7.2 Availabilities

Time slot	Mon	Tue	Wed	Thu	Fri
08h00–09h00	-	(XR)	-	-	(XO)
09h00–10h00	-	(XR)	-	-	(XO)
10h00–11h00	(XR)	-	-	-	(XO)
11h00–12h00	(XR)	-	-	-	(XO)
12h00–13h00	-	-	-	-	(XO)
13h00–14h00	XR	-	-	-	(XO)
14h00–15h00	XR	-	-	-	(XO)
15h00–16h00	(XR)	XR	-	-	(XO)
16h00–17h00	(XR)	XR	-	-	(XO)
17h00–18h00	-	(XR)	-	-	(XO)
18h00–19h00	-	(XR)	-	-	-

Table 43: Team availability for review meetings

7.3 Vision

GlowCore is a singleplayer progression game focused on resource gathering and gradual world expansion. At the start, the player can only access a small, limited area of the game world, while the rest remains hidden and unreachable. At the center of the world is a special entity that can be upgraded using collected materials. Each upgrade expands the accessible area, revealing new parts of the world and enabling further progression. The core gameplay loop is built around exploring newly unlocked areas, gathering resources, and using them to continuously improve the central entity and push the boundaries of the world outward.

As the world expands, additional gameplay systems deepen the experience. The player can obtain and use different materials, interact with tools, craft items, and face enemies within the expanding world. More advanced mechanics such as machines for processing resources, farming systems, and specialized equipment can be introduced over time to increase complexity and strategic depth.

7.4 Proposed Realisation

The project will be implemented using Unity as the primary game engine, using its integrated systems for rendering, physics, input handling, animation, audio, and crossplatform deployment. Development will be done in C# using Unity's component-based architecture (GameObjects, MonoBehaviours, and ScriptableObjects) to structure gameplay logic and systems. Unity's built in physics engine and animation system (Animator) will be used to realise core gameplay and user interaction. Visual assets will be rendered using Unity's Universal Render Pipeline (URP). Individual systems will be tested for robustness using unit tests by method of Unity's own testing framework, based on NUnit. Version control will be handled with Git. The project will be developed within the Unity Editor.

8 Project Plan

8.1 Resources

8.1.1 People

The project team consists of 4 students. Each member contributes approximately 8.6 hours per week and is responsible for tasks of their defined roles.

8.1.1.1 Project Management Roles

Role	Description	Names
Product Owner	• Define product vision • Manage backlog • Prioritize features • Validate requirements	Nathanael Fässler
Scrum Master	• Facilitate meetings • Remove blockers • Ensure Scrum process • Support team workflow	Dominik Wyss
Project Manager	• Schedule planning • Resource management • Risk control • Progress reporting	Cedric Cathomas
DevOps Engineer	• CI/CD maintenance • Build automation • Deployment support • Infrastructure management	Yoris Kucera

Table 44: Project Management Roles

8.1.1.2 Developer Roles

Role	Description	Names
Lead Developer	• Overall technical coordination • Code quality control • Architecture decisions • Integration management	Yoris Kucera
Developer	• Gameplay implementation • Bug fixing • Feature development • System integration • Writing Test Cases	Dominik Wyss Yoris Kucera Nathanael Fässler Cedric Cathomas
Art Director / 3D Artist	• Asset creation • Scene design • Visual consistency • Asset optimization	Nathanael Fässler
UI Architect	• UI/UX design • Menu systems • HUD implementation • User interaction flow	Cedric Cathomas
Graphics Programmer	• Shader development • Particle effects • Performance optimization • Visual effects integration	Yoris Kucera
Audio Engineer	• Sound design • Audio integration • Music management • Audio balancing	Dominik Wyss
Level Designer	• Level layout • Gameplay balancing • Difficulty tuning • Playtesting feedback	Nathanael Fässler Cedric Cathomas

Table 45: Developer Roles

8.1.2 Time

The project runs from 16.02.2026 to 12.06.2026 with an estimated effort of 120 hours per person, resulting in a total workload of 480 hours, structured in milestones and sprints.

8.1.3 Cost

Most required tools and platform are freely available through free or student licenses. We plan on publishing GlowCore on steam for which steam charges a 100 US-Dollar publishing fee. The cost will be shared by the members.

8.2 Organization

8.2.1 Meetings

The project meetings follow mainly the Scrum standard. With weekly Scrum meetings replacing daily stand-ups and biweekly sprint planning including retrospectives. Also the Regular advisor meetings to present and discuss our results so far.

Type	Todos	Frequency
Sprint Review (Advisor)	• Present sprint results • Demonstrate features • Discuss open issues • Collect advisor feedback • Document decisions	Every 2 weeks Tuesday 15:00 - 16:00
Weekly Scrum	• Report progress • Define next steps • Identify blockers • Update tasks • Support team members	Weekly Friday 14:00 - 15:00
Sprint Retrospective	• Analyze sprint process • Identify problems • Define improvements • Document lessons learned	Every 2 weeks Friday 15:00 - 16:00
Sprint Planning	• Define sprint goals • Select backlog items • Estimate effort • Assign tasks • Update schedule	Every 2 weeks Friday 16:00 - 17:00 (after retro)

Table 46: Planned Meetings Overview

8.2.2 Planning Tools

Tool	Use Case
GitHub	• Version control • GitHub Actions (CI/CD) • Pull requests • GitHub notifications via Discord
Typst	• Project documentation • PDF generation
Microsoft Teams	• File sharing • Communication
Jira	• Task management • Sprint planning • Backlog tracking • Issue tracking • Time tracking
Miro	• Brainstorming • Diagrams • Project planning
Discord	• Online meetings • Informal communication • Quick coordination • GitHub activity notifications

Table 47: Planning Tools

8.3 Schedule

The project follows the OST internal SCRUM+ framework for its overall organization. SCRUM+ combines elements of Scrum with the Rational Unified Process (RUP). Long-term planning is based on the RUP approach, while short-term development is organized using Scrum with two-week iterations.

At the start of each iteration, we estimate all tickets with storypoints. The story **GC-54 Make Player** is the reference story which is worth 5 storypoints.

8.3.1 Work Items

We work with 5 types of Work Items

Name	Description
Epic	A group of features that we want to implement.
Story	Describes a feature that is directly related to the game
Task	Everything that isn't directly related to the game
Bug	Bugs are created when a bug is found and fixed in an unplanned matter.
Documentation	Work related to the documentation

Table 48: Work Items

8.3.2 Long Term Plan

The Long Term Planning is separated into two categories:

- Milestones: Serve as checkpoint for releases and review meetings.
- Epics: Provide the structure to organize the stories.

8.3.2.1 Milestones

The milestones for the review meetings and for the releases are interleaved and independent.

In agile project management it is common to reevaluate the Long Term Plan during the project. For this reason there are two Milestone Tables. The first table shows the Initial Milestones that were defined at the beginning of the project and the second table contains updated Milestones as of 07.04.2026.

8.3.2.1.1 Initial Milestones

Nr	Milestone Name	Date	Description
M01	Review 1	10.03.2026	Initial Project Setup
M02	Review 2	24.03.2026	Requirements
M03	Review 3	14.04.2026	End of Elaboration
M04	Review 4	28.04.2026	Quality
M05	Review 5	12.05.2026	Architecture
M06	Project Presentation	12.06.2026	Final Presentation to SEProject-Advisors and fellow students. Completion of SEProject.
M07	Pre-Alpha Release	20.03.2026	First playable prototype is ready. Core gameplay loops are implemented. The player can chop trees, pick up wood and expand the GlowCore.
M08	Alpha Release	10.04.2026	MVP is done and working. Core gameplay Mechanics are implemented. Player can craft, build, expand the map and he has an inventory. The player can save his savestate.
M09	Beta Release	15.05.2026	More Biomes and Materials are added. There is a combat system with enemies and peaceful mobs. There is a title screen and a settings menu. Automation mechanics are implemented. There are more GlowCore Phases.
M10	Official Release and Final Submission	05.06.2026	The game is polished and has no mayor bugs. The game has a beginning and an ending.

Table 49: Initial Milestones

8.3.2.1.2 Updated Milestones as of 07.04.2026

Milestone M08 - Beta Release was updated and the combat system and the automation mechanics were removed. The reduction in scope allows the team to deliver a stable and more polished game at the end. The savestate system was moved from Alpha to Beta. We prioritize quality over quantity. If there is enough time at the end of the project, we can still implement the features as a bonus.

Nr	Milestone Name	Date	Description
M01	Review 1	10.03.2026	Initial Project Setup
M02	Review 2	24.03.2026	Requirements
M03	Review 3	14.04.2026	End of Elaboration
M04	Review 4	28.04.2026	Quality
M05	Review 5	12.05.2026	Architecture
M06	Project Presentation	12.06.2026	Final Presentation to SEProject-Advisors and fellow students. Completion of SEProject.
M07	Pre-Alpha Release	20.03.2026	First playable prototype is ready. Core gameplay loops are implemented. The player can chop trees, pick up wood and expand the GlowCore.
M08	Alpha Release	10.04.2026	MVP is done and working. Core gameplay Mechanics are implemented. Player can craft, build, expand the map and he has an inventory.
M09	Beta Release	15.05.2026	More Biomes and Materials are added. There is a title screen and a settings menu. There are more GlowCore Phases. The player can save his savestate.
M10	Official Release and Final Submission	05.06.2026	The game is polished and has no mayor bugs. The game has a beginning and an ending.

Table 50: Updated Milestones**8.3.2.2 Epics**

Epic Name	Description/Requirements
Documentation	This epic includes all documentation related work.
Project Setup	This epic contains all work that is done in the Inception-Phase.
Core Gameplay Loop Mechanics	- The player can chop down trees, collect wood and feed it into the GlowCore. - The GlowCore levels up and expands the map. - The player is restricted to the unlocked map. The grid system is implemented.
Inventory, Crafting, Building	- Inventory: The player has an inventory and can store a limited amount of items. He can have items in his hands and in his backpack(or similar). - Crafting: The player can craft items to other items on different workstations. - Building: The player can place objects onto the world-grid.
Graphics	- The Artstyle of the game ist set. - The 3D Models follow a consistent Artstyle. - A Shader is implementet.
Game Framework & Completeness	This epic contains everything that makes a game complete and feel like a game. - Intro: When the Game starts the GlowCore Logo is displayed. After the title screen a Cutscene is played that leads into the game. - Ending: A ending of the game is defined. When the player finishes the game, a Cutscene is played and the credits roll. - UI: A Title Screen is available. In-Game there is a Debug Screen, and a Settings Menu. - Gamedata Saving: The player can save his progress.
Big World & GlowCore Phases	- World building: The game has a complete world to play in. - GlowCore Phases: The GlowCore has multiple phases, all of which need different materials to level up. The GlowCore changes visually with every Level.
Biomes & Materials	- The world has multiple biomes: e.g. Forest, Grassland, Mountain, Volcano, Jungle etc. - Each biome has unique ressources. - There are Caves which you can enter.
Combat	- Enemies can spawn and fight with the player. - The player can take damage and can die. - A respawn mechanic is set up. - There are peaceful mobs.
Resource Automation	- The Player can replant trees. - The Player can craft machines that automatically gather resources.
Playtesting & Release	- Playtesting: The game is testet by players that play it for the first time (blind test). - Release: The game is released on steam on its own steam page and has a trailer, screenshots and a description.

Table 51: Epics

The following image shows the Timeline in Jira as of 04.03.2026.

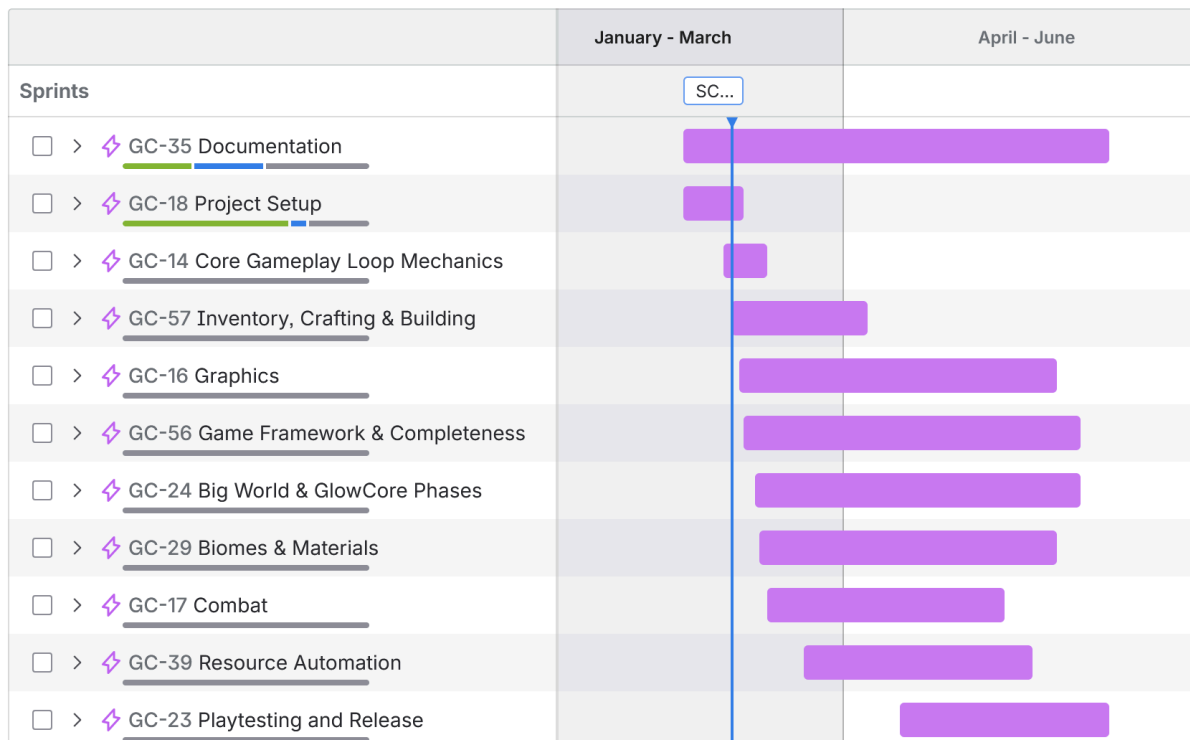


Image 27: Initial Jira Timeline

The Long Term Plan was updated on 07.04.2026. The following image shows the new Timeline in Jira as of 07.04.2026.

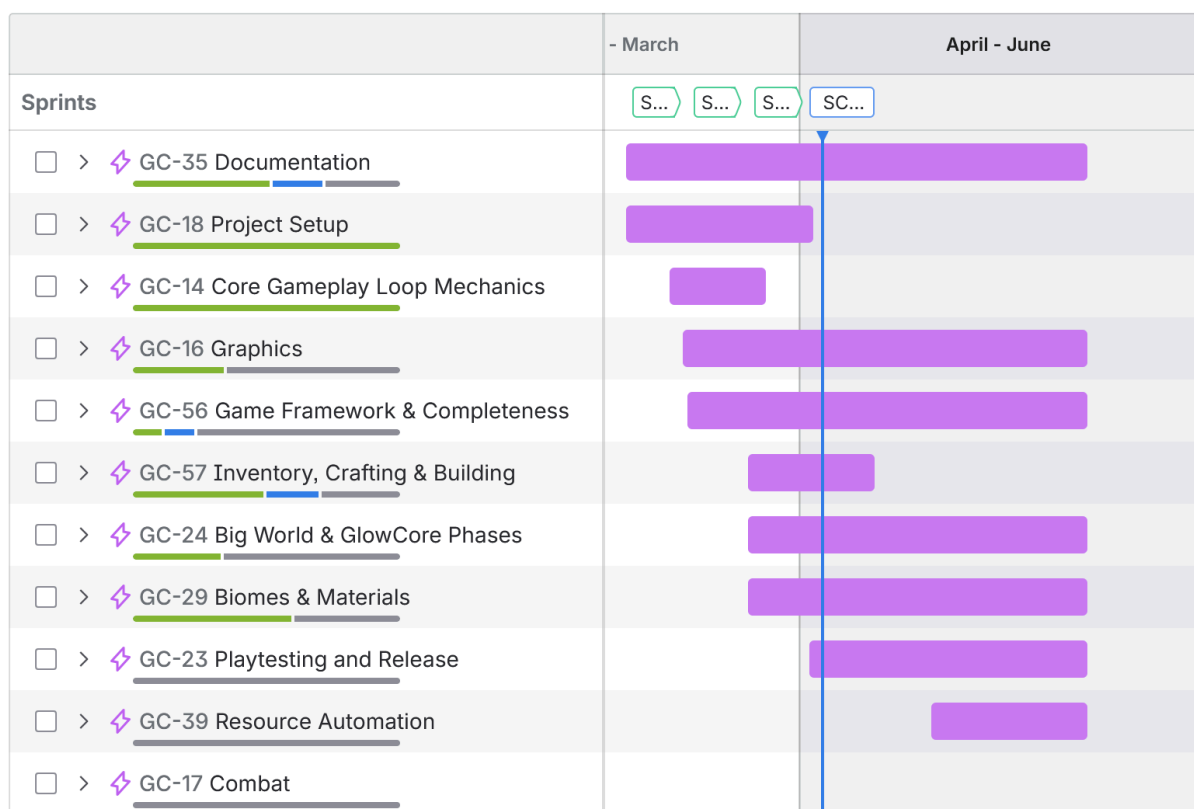


Image 28: Updated Jira Timeline

8.3.2.3 Initial Backlog & MVP

The priority of Stories is defined by the ordering in the backlog. As we progress in Agile, we may divide Stories further and add new Stories. The MVP is reached when every Story up to and including **GC-101 Build World** is completed.

The following is the Initial Backlog as of 04.03.2026.

☐ Backlog (44 work items) 0 0 0 Create sprint				
GC-54	There is a player	CORE GAMEPLAY LO...	TO DO	-
GC-78	Create Tree-Node 3D Model	GRAPHICS	TO DO	-
GC-95	Create Wood-Item 3D Model	GRAPHICS	TO DO	-
GC-47	Trees can be chopped	CORE GAMEPLAY LO...	TO DO	-
GC-109	Make GlowCore Phase 1 3D Model	GRAPHICS	TO DO	-
GC-46	GlowCore expands World	CORE GAMEPLAY LO...	TO DO	-
GC-69	Make Inventory-System	INVENTORY, CRAFTI...	TO DO	-
GC-74	Save Game-Data	GAME FRAMEWORK &...	TO DO	-
GC-108	Create Axe 3D Model	GRAPHICS	TO DO	-
GC-97	Create Coal 3D Model	GRAPHICS	TO DO	-
GC-106	Add Coal Node and Item	BIOMES & MATERIALS	TO DO	-
GC-96	Create Stone 3D Model	GRAPHICS	TO DO	-
GC-93	Add Stone Node and Item	BIOMES & MATERIALS	TO DO	-
GC-107	Make Pickaxe 3D Model	GRAPHICS	TO DO	-
GC-102	Add Pickaxe	BIOMES & MATERIALS	TO DO	-
GC-79	Add Grassland Biome	BIOMES & MATERIALS	TO DO	-
GC-111	Make GlowCore Phase 2 3D Model	GRAPHICS	TO DO	-
GC-100	Add GlowCore Phase 2	BIG WORLD & GLOWC...	TO DO	-
GC-71	Make Crafting-System	INVENTORY, CRAFTI...	TO DO	-
GC-70	Make Building-System	INVENTORY, CRAFTI...	TO DO	-
GC-77	Implement Pixelize-Shader	GRAPHICS	TO DO	-
GC-103	Add Chest	INVENTORY, CRAFTI...	TO DO	-
GC-101	Build World	BIG WORLD & GLOWC...	TO DO	-
GC-99	Add Rock Biome	BIOMES & MATERIALS	TO DO	-
GC-72	Game Intro	GAME FRAMEWORK &...	TO DO	-
GC-73	Game Ending	GAME FRAMEWORK &...	TO DO	-
GC-105	Add Combat Logic	COMBAT	TO DO	-
GC-124	Create Enemy 3D Model	GRAPHICS	TO DO	-
GC-81	Add Enemy	COMBAT	TO DO	-
GC-75	Ingame Debug Settings	GAME FRAMEWORK &...	TO DO	-
GC-76	Player Settings-Menu	GAME FRAMEWORK &...	TO DO	-
GC-125	Create Peaceful-Mob 3D Model	GRAPHICS	TO DO	-
GC-82	Add peaceful Mob	COMBAT	TO DO	-
GC-123	Make Sword 3D Model	GRAPHICS	TO DO	-
GC-83	Add Sword	COMBAT	TO DO	-
GC-120	Make Axe-Machine 3D Model	GRAPHICS	TO DO	-
GC-114	Add Axe-Machine	RESOURCE AUTOMAT...	TO DO	-
GC-121	Make Replant-Machine 3D Model	GRAPHICS	TO DO	-
GC-115	Add Replant-Machine	RESOURCE AUTOMAT...	TO DO	-
GC-122	Make Rotater-Block 3D Model	GRAPHICS	TO DO	-
GC-116	Add Rotater-Block	RESOURCE AUTOMAT...	TO DO	-
GC-117	Conduct Playtesting with User	PLAYTESTING AND R...	TO DO	-
GC-118	Release Game on Steam	PLAYTESTING AND R...	TO DO	-
GC-119	Make Marketing Material for Steam Page	PLAYTESTING AND R...	TO DO	-

Image 29: Initial Backlog

8.3.3 Short Term Plan

In a Sprint the priority of the tasks and stories is defined by the ordering.

8.3.3.1 Sprint 1 (Sprint Zero) 20.02.2026 - 06.03.2026

The following shows the Jira sprint backlog of the first sprint. Since this was Sprint Zero, not all guidelines were followed initially because they had not yet been established.

Work	Assignee	Status
 GG-88 Document Long and Short Term Plan	 Nathanael Fässler	DONE ▾
 GG-135 Document Sprint Planning Meeting Sprint 2	 Nathanael Fässler	DONE ▾
 GG-92 Setup Contract with OST about Ownership	 Nathanael Fässler	DONE ▾
 GG-89 Document Meeting Weekly Scrum Sprint 1	 Nathanael Fässler	DONE ▾
 GG-4 Concept for Code Quality, Linter etc.	 Yoris Kucera	DONE ▾
 GG-3 Unity Setup	 Yoris Kucera	DONE ▾
 GG-126 Documentation list of tables	 Cedric Cathomas	DONE ▾
 GG-1 Risk Specification	 Yoris Kucera	DONE ▾
 GG-91 Document Guidelines for code and documentation	 Yoris Kucera	DONE ▾
 GG-6 GitLab Timetracking Research	 Dominik Wyss	DONE ▾
 GG-5 GitLab Pipeline Setup	 Dominik Wyss	DONE ▾
 GG-2 GitHub Setup	 Yoris Kucera	DONE ▾
 GG-87 Document Meetings Structure	 Cedric Cathomas	DONE ▾
 GG-84 Typst documentation changes	 Cedric Cathomas	DONE ▾
 GG-86 Document Collaboration as a team	 Cedric Cathomas	DONE ▾
 GG-7 Typst Documentation Setup	 Cedric Cathomas	DONE ▾
 GG-8 People, Cost, Time Documentation	 Cedric Cathomas	DONE ▾
 GG-85 Roles Documentation	 Cedric Cathomas	DONE ▾
 GG-12 Longtermplan & Milestones	 Nathanael Fässler	DONE ▾
 GG-9 Jira Setup	 Nathanael Fässler	DONE ▾
 GG-112 Unity Folder Structure	 Nathanael Fässler	DONE ▾
 GG-11 Steamworks Registration	 Nathanael Fässler	DONE ▾
 GG-13 Shorttermplan for first sprint	 Nathanael Fässler	DONE ▾

Image 30: Sprint 1 Backlog

8.3.3.2 Sprint 2 06.03.2026 - 20.03.2026

In this sprint the first actual work on the game was carried out in Unity. The focus was to implement an initial working prototype.

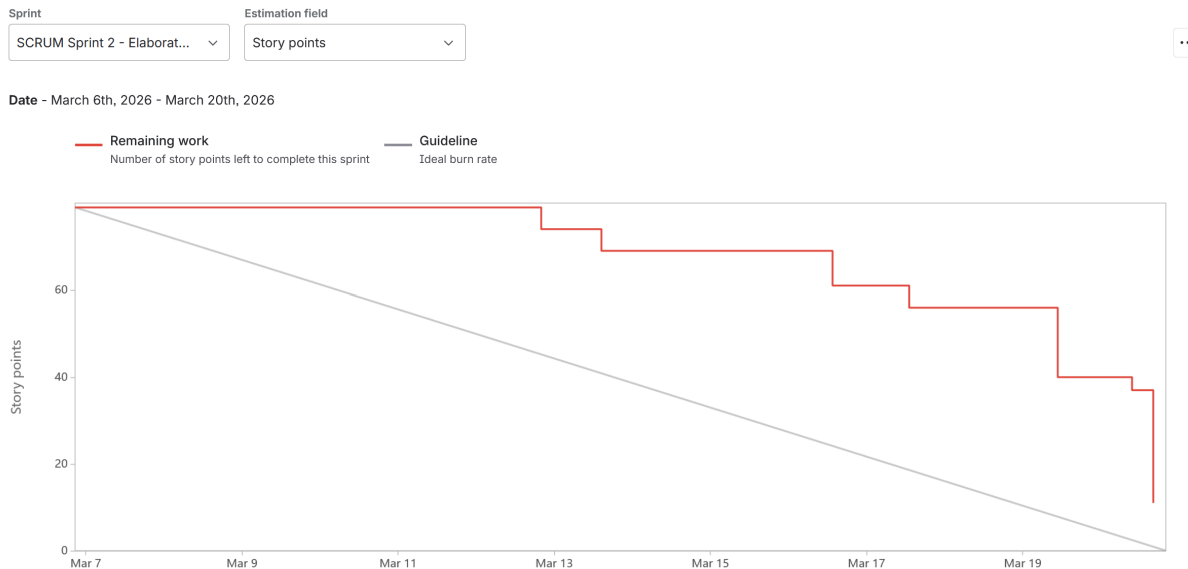


Image 31: Sprint 2 Burndown Chart

Report: SCRUM Sprint 2 - Elaboration 1

Scope changes log

Date	Key	Summary	Work type	Epic	Details of scope change	Change in estimation
2026-03-13	GC-138	GitHub Runner Disk Space usage to high	Bug		Work item added to sprint	-

Incomplete work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-130	Unity Testing Setup and configure Pipeline	Task	PROJECT SETUP	TESTING	NY	3
GC-133	Implement Basic inventory system	Story	INVENTORY, CRAFTING & BUIL...	IN PROGRESS	NY	8

Completed work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-85	Create Wood-Item 3D Model	Story	GRAPHICS	DONE	NY	1
GC-28	Create Tree-Node 3D Model	Story	GRAPHICS	DONE	NY	1
GC-34	Make Player	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NY	5
GC-109	Make GlowCore Phase 1 3D Model	Story	GRAPHICS	DONE	NY	2
GC-41	Trees can be chopped	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NY	5
GC-48	GlowCore expands World	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NY	8
GC-108	Create Axe 3D Model	Story	GRAPHICS	DONE	NY	1
GC-126	Implement Node Hover Picking	Story	CORE GAMEPLAY LOOP MECHA...	DONE	NY	8
GC-130	Document Functional Requirements	Documentation	DOCUMENTATION	DONE	NY	8
GC-131	Document Non-Functional Requirements	Documentation	DOCUMENTATION	DONE	NY	8
GC-132	Document Domain-Model	Documentation	DOCUMENTATION	DONE	NY	13
GC-134	Pipeline for doc build	Task	PROJECT SETUP	DONE	NY	5
GC-136	Make UI Sketches	Task	GAME FRAMEWORK & COMPLET...	DONE	NY	3
GC-138	GitHub Runner Disk Space usage to high	Bug		DONE	NY	-

Image 32: Sprint 2 Backlog

8.3.3.3 Sprint 3 20.03.2026 - 03.04.2026

In Sprint 3 we focused on implementing the MVP. The planned amount of storypoints was too ambitious so we had to move some Tasks into Sprint 4. From this experience we gained a better understanding of our team's capacity.

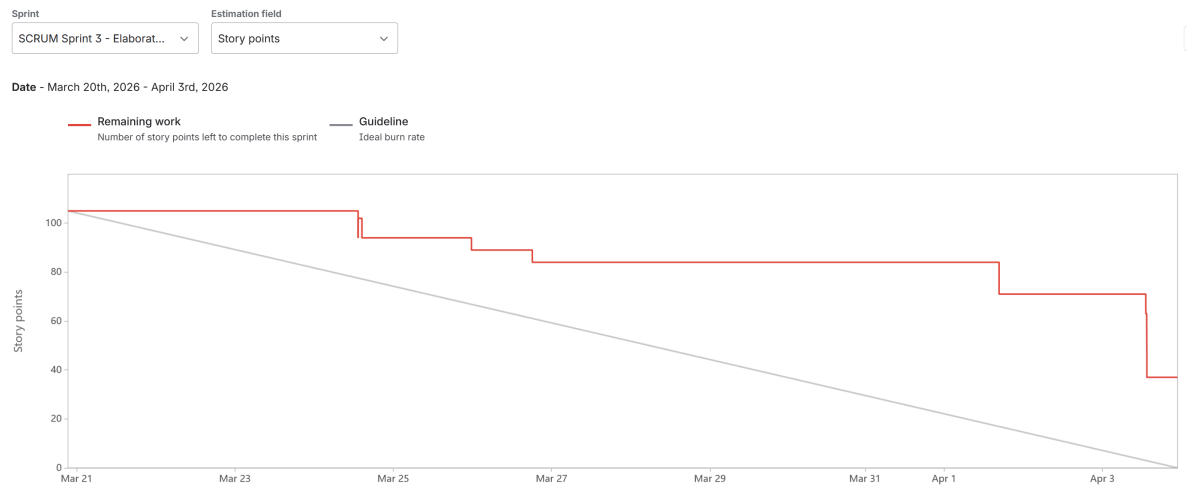


Image 33: Sprint 3 Burndown Chart

Report: SCRUM Sprint 3 - Elaboration 2

Scope changes log

Date	Key	Summary	Work type	Epic	Details of scope change	Change in estimation
2026-04-03	GC-170	Building doesn't work and pickaxe doesn't exist	Bug		Work item added to sprint	0

Incomplete work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-171	Make Crafting-System	Story	INVENTORY, CRAFTING & BUIL...	IN PROGRESS	AS	8
GC-174	Save Game-Data	Story	GAME FRAMEWORK & COMPLET...	IN PROGRESS	AS	13
GC-142	Make Crafting UI	Story	INVENTORY, CRAFTING & BUIL...	IN PROGRESS	AS	8
GC-144	Add GlowCore Upgrade UI	Story	INVENTORY, CRAFTING & BUIL...	TO DO	AS	8

Completed work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-352	Add Pickaxe	Story	MINES & MATERIALS	DONE	AS	3
GC-357	Create Coal 3D Model	Story	GRAPHICS	DONE	AS	1
GC-357	Make Player 3D Model	Story	GRAPHICS	DONE	AS	1
GC-29	Implement Building-System	Story	INVENTORY, CRAFTING & BUIL...	DONE	AS	13
GC-348	Make Inventory UI	Story	INVENTORY, CRAFTING & BUIL...	DONE	AS	13
GC-498	Make Inventory-System	Story	INVENTORY, CRAFTING & BUIL...	DONE	AS	8
GC-348	Create Stone 3D Model	Story	GRAPHICS	DONE	AS	1
GC-302	Make Pickaxe 3D Model	Story	GRAPHICS	DONE	AS	1
GC-308	Add Coal Node and Item	Story	MINES & MATERIALS	DONE	AS	1
GC-333	Add Stone Node and Item	Story	MINES & MATERIALS	DONE	AS	1
GC-373	Implement Pixelator-Shader Prototype	Story	GRAPHICS	DONE	AS	8
GC-303	Build World for Alpha Release	Story	3D WORLD & GLOWCORE PHAS...	DONE	AS	3
GC-343	Implement Tool Logic	Story	INVENTORY, CRAFTING & BUIL...	DONE	AS	5
GC-348	Add Crafting Table 3D Model	Story	GRAPHICS	DONE	AS	1
GC-333	Implement Basic inventory system	Story	INVENTORY, CRAFTING & BUIL...	DONE	AS	8
GC-30	Unity Testing Setup and configure Pipeline	Task	PROJECT SETUP	DONE	AS	3
GC-170	Building doesn't work and pickaxe doesn't exist	Bug		DONE	AS	0

Image 34: Sprint 3 Backlog

8.3.3.4 Sprint 4 03.04.2026 - 17.04.2026

Because in the week of 06.04.2026 to 12.04.2026 was the spring break we had a sprint with reduced work.

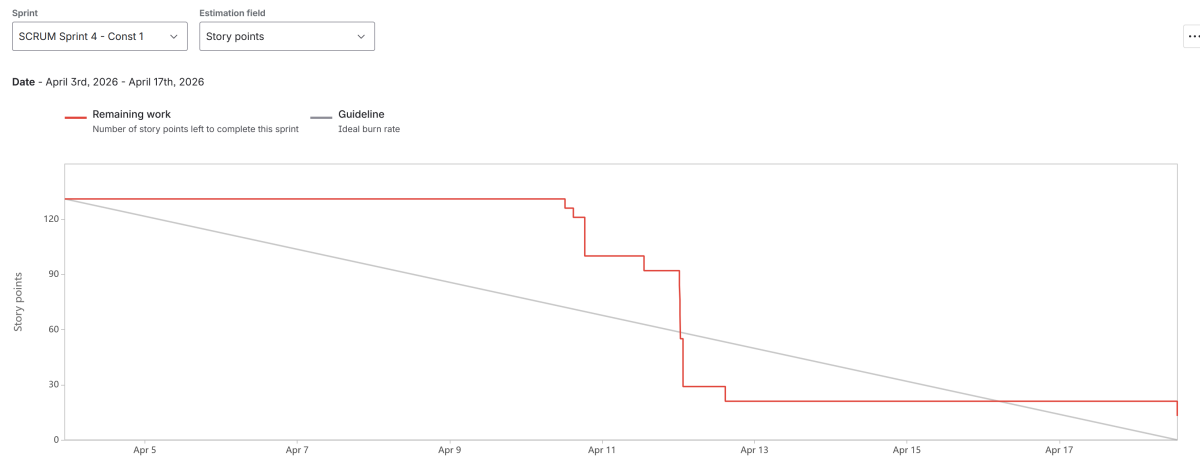


Image 35: Sprint 4 Burndown Chart

Report: SCRUM Sprint 4 - Const 1

Scope changes log

Date	Key	Summary	Work type	Epic	Details of scope change	Change in estimation
2026-04-06	GC-176	GitHub Action failed, free disk space	Bug		Work item added to sprint	-
2026-04-08	GC-177	Test Pipeline run when only doc changes	Bug		Work item added to sprint	-
2026-04-13	GC-186	Test NFIs	Story	PLAYTESTING AND RELEASE	Work item added to sprint	-
2026-04-13	GC-186	Test NFIs	Story	PLAYTESTING AND RELEASE	Work item removed from sprint	-

Incomplete work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-74	Save Game-Data	Story	GAME FRAMEWORK & COMPLET...	IN PROGRESS	NP	13

Completed work items

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-193	Make Risk-Analysis	Documentation	DOCUMENTATION	DONE	NP	5
GC-188	Update Project Plan	Documentation	DOCUMENTATION	DONE	NP	5
GC-187	Create Test Concept	Documentation	DOCUMENTATION	DONE	NP	3
GC-188	Document Architecture	Documentation	DOCUMENTATION	DONE	NP	13
GC-147	Register Steampage	Story	PLAYTESTING AND RELEASE	DONE	NP	5
GC-171	Dev environment setup diagram	Documentation	DOCUMENTATION	DONE	NP	13
GC-172	Document Technologies and patterns	Documentation	DOCUMENTATION	DONE	NP	21
GC-173	AB Document interfaces between system components	Documentation	DOCUMENTATION	DONE	NP	13
GC-174	P1-P3	Documentation	DOCUMENTATION	DONE	NP	8
GC-175	Document Savefile structure	Documentation	DOCUMENTATION	DONE	NP	8
GC-71	Make Crafting-System	Story	INVENTORY, CRAFTING & BUIL...	DONE	NP	8
GC-143	Make Crafting UI	Story	INVENTORY, CRAFTING & BUIL...	DONE	NP	8
GC-144	Add GlowCore Upgrade UI	Story	INVENTORY, CRAFTING & BUIL...	DONE	NP	8
GC-176	GitHub Action failed, free disk space	Bug		DONE	NP	-
GC-177	Test Pipeline run when only doc changes	Bug		DONE	NP	-

Image 36: Sprint 4 Backlog

8.3.3.5 Sprint 5 17.04.2026 - 01.05.2026



Image 37: Sprint 5 Burndown Chart

Report: SCRUM Sprint 5 - Const 2

Scope changes log

Date	Key	Summary	Work type	Spn	Details of scope change	Change in estimation
2026-04-25	SC-38017	Create DoD and Definition of Done	Task	DOCUMENTATION	Work item added to sprint	0
2026-04-25	SC-38017	Q1 UDC and Test Coverage diagrams over time	Task	DOCUMENTATION	Work item added to sprint	0
2026-04-25	SC-38017	Java and GraphQL overview contribution	Task	DOCUMENTATION	Work item added to sprint	0
2026-04-25	SC-38017	T2 TA	Task	DOCUMENTATION	Work item added to sprint	0

Incomplete work items

Key	Summary	Work type	Spn	Status	Assignee	Story points
SC-38017	Improve Camera	Story	IMPROVEMENT	TO DO		8
SC-38017	Anti-Collision Module	Story	NEWLY IDENTIFIED	TO DO		10
SC-38017	Save Game Data	Story	NEWLY IDENTIFIED & IDENTIFIED	TO DO		10
SC-38017	Refined GraphQL Router	Task		IN PROGRESS		10
SC-38017	T2 TA	Task		IN PROGRESS		10

Completed work items

Key	Summary	Work type	Spn	Status	Assignee	Story points
SC-38017	Document Quality Measures	Documentation	DOCUMENTATION	DONE		8
SC-38017	Document Testing	Documentation	DOCUMENTATION	DONE		5
SC-38017	Make it Playable	Story	PLAYTESTING AND RELEASE	DONE		20
SC-38017	Verify all APIs and Use Cases for M2B & M2C	Story	PLAYTESTING AND RELEASE	DONE		20
SC-38017	Make GameCore Logic	Story	PLAYTESTING AND RELEASE	DONE		10
SC-38017	Write Integration Tests	Task		DONE		20
SC-38017	Conduct Field Player Tests	Story	PLAYTESTING AND RELEASE	DONE		5
SC-38017	Create DoD and Definition of Done	Task	DOCUMENTATION	DONE		0
SC-38017	Q1 UDC and Test Coverage diagrams over time	Task		DONE		0
SC-38017	Java and GraphQL overview contribution	Task		DONE		0

Image 38: Sprint 5 Backlog

8.3.3.6 Sprint 6 01.05.2026 - 15.05.2026

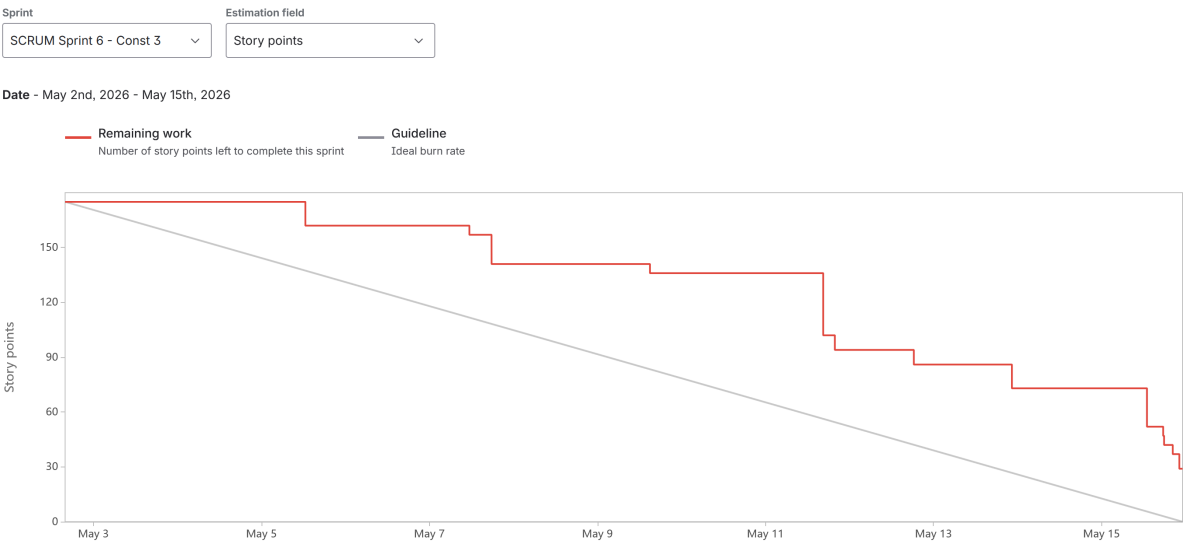


Image 39: Sprint 6 Burndown Chart

Report: SCRUM Sprint 6 - Const 3

Work item added after sprint start

Scope changes log

View in All work navigator

Date	Key	Summary	Work type	Epic	Details of scope change	Change in estimation
2026-05-14	GC-214	Document Game Design	Documentation	DOCUMENTATION	Work item added to sprint	-

Incomplete work items

View in All work navigator

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-213	Game Ending	Story	GAME FRAMEWORK & COMPLET...	TO DO	JP	5
GC-165	Add SoundEffects	Story	GAME FRAMEWORK & COMPLET...	IN PROGRESS	JP	20
GC-88	Add Rock Biome	Story	BIOMES & MATERIALS	IN REVIEW	JP	3

Completed work items

View in All work navigator

Key	Summary	Work type	Epic	Status	Assignee	Story points
GC-161	Add Chest UI	Story	BEST WORLD & GLOWCORE PHAS...	DONE	JP	5
GC-79	Add Game Menus	Story	GAME FRAMEWORK & COMPLET...	DONE	JP	20
GC-168	Extend Crafting Stations	Story	BIOMES & MATERIALS	DONE	JP	5
GC-166	Improve Border Visuals	Story	GRAPHICS	DONE	JP	8
GC-169	Improve GlowCore UI	Story	BEST WORLD & GLOWCORE PHAS...	DONE	JP	5
GC-167	Make new Nodes and Items	Story	GRAPHICS	DONE	JP	34
GC-164	Make better Player 3D Model	Story	GRAPHICS	DONE	JP	3
GC-169	Document Deployment Diagram	Documentation	DOCUMENTATION	DONE	JP	5
GC-208	Document MFR linking to technical solutions	Documentation	DOCUMENTATION	DONE	JP	5
GC-207	Document Design Decisions	Documentation	DOCUMENTATION	DONE	JP	3
GC-202	Document Frontend Architecture	Documentation	DOCUMENTATION	DONE	JP	5
GC-169	Selfhosted Github Runner	Task		DONE	JP	15
GC-74	Save Game-Data	Story	GAME FRAMEWORK & COMPLET...	DONE	JP	15
GC-167	Improve Camera	Story	GRAPHICS	DONE	JP	8
GC-163	T2-16	Task		DONE	JP	-
GC-79	Add Grassland Biome	Story	BIOMES & MATERIALS	DONE	JP	15
GC-214	Document Game Design	Documentation	DOCUMENTATION	DONE	JP	-

Image 40: Sprint 6 Backlog

8.4 Risk Management

This chapter describes potential risks to the project and how we plan to mitigate them. We identified 8 risks, which are listed in the table below. Additionally, these risks are visualized in two risk matrices, one showing the initial risk assessment and the other showing the risk assessment after mitigation strategies are applied.

No.	Risk	Description	Mitigation Strategy
R01	Lack of familiarity with Unity	None of our team members have any significant experience with the Unity Engine. As such, this could heavily impact our development times.	We made ourselves familiar with the engine before the actual start of the project, to get an idea of the workflows.
R02	Unity CI/CD Complexity	We are depending on Unity's build system being robust enough to work in the context of a CI/CD pipeline. We also potentially rely on pre-existing GitHub Actions for Unity, which may not be well-maintained or compatible with our project.	We will set up the CI/CD pipeline very early into the project and did earlier tests. We tried out pre-existing GitHub Actions to make sure they satisfy our needs.
R03	Scope/Feature Creep	As we are developing a game, we run the risk of encountering scope creep, i.e. continuously planning new features, which would result in us eventually not being able to keep up with the workload.	We created a detailed project plan with long-term and short-term planning, along with well defined milestones and a clear MVP.
R04	Bad Code Quality	In large codebases it is easy to end up with badly designed systems that don't fit together with the rest of the code. This would hinder further development.	We do mandatory code reviews before changes can be merged. Additionally, we will set up CI/CD pipelines to automatically run code quality and formatting checks on pull requests.
R05	Team Member is MIA	A team member might be MIA for an extended period of time due to various reasons, such as an illness or damaged hardware. This would reduce our capacity and slow down development.	We can't reduce the possibility of this occurring, but we can distribute work across the remaining team members. We also avoid knowledge monopolies and always aim to share discoveries with the other team members.
R06	Personal Conflicts	There is always the potential for personal conflicts within the team, which would complicate development.	We openly communicate any interpersonal issues and try to resolve conflicts as early as possible.
R07	Data Loss	There is the potential of local storage media failing, resulting in significant data loss.	We mitigate this by frequently committing our changes to GitHub, and keeping untracked files in cloud based locations.
R08	Workload Underestimation	We might be underestimating the amount of work required for specific features and tasks.	We plan with a buffer and try to be conservative with our estimates. We also try to break down large tasks into smaller ones, which are easier to estimate.

Table 52: Risk Analysis

8.4.1 Risk Matrix

The below table shows the risk assessment of the identified risks, before any mitigation strategies are applied. The risks are categorized based on their probability of occurrence and their potential severity.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely			R04	R01
Possible		R02/R06		R03/R08
Unlikely				R05
Rare				R07
None				

Table 53: Risk Matrix

8.4.2 Risk Matrix (Mitigated)

The below table shows the risk assessment of the identified risks, after mitigation strategies have been applied.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely				
Possible				
Unlikely		R06	R01/R04/R05	R08
Rare	R07	R02		
None				R03

Table 54: Risk Matrix (Mitigated)

8.4.3 Assessed Risk Matrix (14.04.2026)

The below table shows the risk assessment of the identified risks, after mitigation strategies have been applied.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely				
Possible			R01	
Unlikely		R06/R02	R04/R05	R08
Rare	R07			
None				R03

Table 55: Assessed Risk Matrix (14.04.2026)

8.4.4 Adjustment of Current Risk Assessment

In the current risk matrix, we increased the probability by one level for both R01 (Lack of familiarity with Unity) and R02 (Unity CI/CD Complexity).

For R01, we observed that working with Unity involves more aspects than initially expected, especially issues that only become visible during actual implementation. The project description already pointed out that Unity could be a risk, so we were aware of this from the beginning. However, during development we realized that some parts are more complex in practice. Despite this, we are still very motivated to use Unity for our project and try to reduce the risk through clear communication and well-defined implementation strategies.

For R02, there are recurring challenges with the CI/CD setup in Unity, which is why we increased the probability here as well. In particular, there is a risk of running out of GitHub Actions minutes. At the moment, we expect the available minutes to be sufficient, but if needed, we already have a plan to set up and use a self-hosted runner on our own machine.

Overall, both risks are now considered slightly more likely, but appropriate mitigation measures are in place.

9 Time Tracking Report

9.1 Time Tracking

Every member tracks time on Jira on the corresponding task/story. Time spent in meetings is documented in the meeting protocols. The time is logged in 15 minute blocks in the format e.g. 3h 15m.

9.2 Time Statistics

The time is grouped by member and by sprint. Per sprint and person the time is rounded to the nearest hour.

Sprint	Cedric Cathomas	Nathanael Fässler	Yoris Kucera	Dominik Wyss	Total
1	23h	29h	21h	24h	97h
2	22h	20h	19h	23h	84h
3	18h	13h	19h	15h	65h
4	25h	13h	15h	17h	70h
5	7h	14h	18h	15h	54h
6	20h	23h	20h	19h	82h
7					
Total	115h	112h	112h	113h	452h

Table 56: Time Statistics Sprint 1

10 Personal Reports

11 Meeting Minutes

11.1 Weekly Scrum / 28.02.2026

Field	Value
Date	28.02.2026
Time	11:00–14:00
Sprint	Sprint 1
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	Cedric Cathomas
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.1.1 Agenda

Agenda item	Owner	Result / Decision
GitHub Setup	Yoris	- We switched from GitLab to GitHub The following rules were discussed: - Pull Request requires one approval that is not the same person as the author. - Merging Methods are Merge and Squash. - Every Jira Task corresponds to one branch.
CICD Pipeline Setup	Dominik	- Web build and .exe build succeeded. - Pages Architecture with main and dev branches decided
Timetracking	Dominik	- We won't use GitLab timetracking because we switched to GitHub - Timetracking is in Jira: Everyone tracks time for individual Tasks/Stories.
Unity/Coding Setup	Yoris	Coding Guidelines decided
Steam Release	Nathanael	- Registered on Steamworks - Up to 30 days waiting time until application is processed - Maybe we need to make a contract with OST.
Testing	Nathanael	Unit-Test and Integration-Test Strategy discussed.
Long/Shorttermplan	Nathanael	- Hierarchy: There are Epics which contain Stories. - Everything that is not a game-feature is a task. - Longtermplan with epics according to the Jira Timeline - Shorttermplan per sprint on the Jira board

		• Priority is defined by the order in the backlog in Jira • MVP is defined • We will estimate Tasks/Stories with Story-points • On Jira we can create Bug-Tickets which we will not estimate with Storypoints
--	--	--

11.1.2 Todos

Todo	Owner	Due
Finish CICD Pipeline	Dominik	06.03.2026
Ask Advisor about contract	Nathanael	04.03.2026
Ask Advisor about project plan	Nathanael	04.03.2026

11.2 Weekly Scrum and Sprint Retrospective / 06.03.2026

Field	Value
Date	06.03.2026
Time	14:00-16:00
Sprint	Sprint 1
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Cedric Cathomas
Links	-

11.2.1 Agenda

Agenda item	Owner	Result / Decision
Board Review	All	- Walked through all current board tasks - Updated status on completed tasks
Done Tasks and New Functionalities	All	- Reviewed completed work from Sprint 1 - Discussed new functionalities and rules
Open Questions	All	Discussed open questions regarding project setup and workflow
Unity Graphics in Repository	Nathanael Fässler	Only .fbx files are to be committed to the repository for 3D graphics in Unity
Documentation Versioning	Cedric Cathomas	- Tags for documentation versions start with "d" (e.g. d1.1) - Version is updated automatically on pull request to main via GitHub Action
How do we manage smaller tasks that are not worth creating a new task on jira. (small documentation changes)	Dominik Wyss	We create ticket for small backlog tasks that do not need an own Jira item so we can reference it on our pull request and use the name of that ticket for the branch
Decide if we need a separate pdf for protocols	Cedric Cathomas	We decided that there's no use in having a separate page just for meeting minutes, so we have one thing less to worry about

11.2.2 Sprint 1 Retrospective

11.2.2.1 What Went Well

- Good and fast communication
- Good planning on Jira
- Good preplanning visible
- Product Owner did a good job
- Good and understandable selection of planning and working tools
- Pipeline and GitHub are working
- Typst is set up
- General setup went very well

11.2.2.2 What Didn't Go Well

- Problems with GitLab
- Didn't have enough time for some tasks

11.2.2.3 Improvements

- Better time management
- Better definitions of done
- Use GitHub from the beginning

11.2.3 Todos

Todo	Owner	Due
Create ticket for small backlog tasks that do not need an own Jira item	Nathanael Fässler	07.03.2026
Create all table references so documentation looks nice and page breaks are clean	Cedric Cathomas	07.03.2026
Adapt milestone dates to single dates (e.g. 16.02.2026 instead of a time frame)	Cedric Cathomas	07.03.2026
clean up documentation (spaces that are too big)	Cedric Cathomas	07.03.2026
Change date format in documentation to use dots	Cedric Cathomas	07.03.2026
Create task for combined build and documentation pipeline	Nathanael Fässler	07.03.2026
Use figures for images so they appear in the list of figures	Nathanael Fässler	07.03.2026
remove seperate meeting minutes pdf typst code	Cedric Cathomas	07.03.2026

11.3 Sprint Planning / 06.03.2026

Field	Value
Date	06.03.2026
Time	16:00–18:00
Sprint	Sprint 2
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.3.1 Sprint goal

- Build first working prototype.
- Define Functional Requirements and Non-Functional Requirements.
- Create Domain Model

11.4 Sprint Review (Advisor) / 10.03.2026

Field	Value
Date	10.03.2026
Time	15:00–16:00
Sprint	Sprint 2
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.4.1 Agenda

Agenda item	Owner	Result / Decision
Status of Work	Yoris	Feedback was good
Project Plan	Yoris	Good impression. Advisor liked that we asked him at 02.03.2026 on our initiative
Showcase Pipeline	Dominik	Positive that Dominik sent the pipeline beforehand
Feedback	Thomas	Overall very good
Questions about Grading & Review Schedule	Dominik & Nathanael	- To get a better grade we have to do more than what is required on the checklist. - The group can choose when the Review Meetings take place. - Meeting 5 has to be before 05.06.2026 (final submission)

11.4.2 Todos

Todo	Owner	Due
Update README on GitLab to indicate that project has moved to GitHub	Yoris	20.03.2026
Configure Pipeline to build documentation	Not assigned	20.03.2026
Add AGENTS.md	Cedric Cathomas	20.03.2026

11.4.3 Next dates

Next date	When
Review 2 - Requirements	24.03.2026 15:00

11.5 Weekly Scrum / 13.03.2026

Field	Value
Date	13.03.2026
Time	14:30-15:30
Sprint	Sprint 2
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Yoris Kucera
Links	-

11.5.1 Agenda

Agenda item	Owner	Result / Decision
Time Tracking	Dominik Wyss	- New time tracking report system integrated in Jira (Worklogs Report) - Task does not need to be assigned for time to be tracked
Documentation Build Pipeline	Yoris Kucera	- Pipeline for building documentation on GitHub Actions is set up and working - Automatically deployed to pages on push to dev and main branches.
Domain Model	Yoris Kucera	- First version of domain model created - State machine for enemy created
Functional Requirements	Nathanael Fässler	- Documented Functional Requirements, Actors and Personas - Documented Use Cases - Use Cases formatting could be improved
Player and Camera	Nathanael Fässler	- Player controller implemented and working, controls with WASD - Camera implemented and working, controls with mouse movement - Camera follows player and rotates around it, fixed height and distance
Non-Functional Requirements	Cedric Cathomas	- Documented Non-Functional Requirements - Priority Matrix for NFRs - NFRs grouped by category and numbered (e.g. NFR101, NFR102, NFR403, etc.)
GitHub CI/CD Failing	Dominik Wyss/Yoris Kucera	- Pipeline fails because of lack of disk space on the runner - Possible solutions: ▸ Clean up even more storage ▸ Make repository public ▸ Use self-hosted runner with more storage ▸ Rent server at OST

UI Sketches	Dominik Wyss/ Cedric Cathomas	• Sketches for Main Menu ▸ Scene from the game shown in the background ▸ Buttons for New Game, Continue, Save Files, Settings, Exit • Sketch for New Game menu ▸ Give name to world • Sketch for Save File Menu ▸ List of save files with name and date ▸ Button to delete save file ▸ Show glowcore level? ▸ Separate list for auto saves and manual saves? • Sketch for Settings Menu ▸ Sliders for music and sound effects volume ▸ Slider for mouse sensitivity? ▸ Graphics options? • Sketch for Pause Menu
-------------	----------------------------------	--

11.5.2 Todos

Todo	Owner	Due
Fix WebGL Build Pipeline	Yoris Kucera	20.03.2026
Unify NFRs, FRs and Domain Model	Yoris Kucera	20.03.2026
Adjust README on GitLab	Dominik Wyss	20.03.2026
Agents.md	Cedric Cathomas	20.03.2026
Explicitly address review feedback in Teams	Dominik Wyss	20.03.2026

11.6 Weekly Scrum and Sprint Retrospective / 20.03.2026

Field	Value
Date	20.03.2026
Time	13:00–15:00
Sprint	Sprint 2
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.6.1 Agenda

Agenda item	Owner	Result / Decision
Planned Inventory System Architecture	Yoris	- ScriptableObject Item represents InventoryItem and WorldItem at the same time. - ItemStack ScriptableObject hold Item and amount.
World Grid & GlowCore expansion	Cedric	- Showed simple prototype for tree generation on border expansion. - Grid represented by 2D Array.
Node Interaction System	Dominik	- Setting for Interaction Range for every Node. - Problem with InputSystem to track how long a button is held.

11.6.2 What went well

- Had fun beginning the work in Unity.
- Good cooperation: Questions by team-members are answered quickly.
- The planning starts to pay off.

11.6.3 What didn't

- Unclear dependencies of tasks.
- CI/CD work can be annoying.

11.6.4 Improvements

- Keep Jira Board up to date.
- Better coordinate the task assignment.

11.6.5 Todos

Todo	Owner	Due
------	-------	-----

Send documentation to Advisor	Dominik	21.03.2026
Make Meeting Minutes	Nathanael	23.03.2026
Finish and merge prototype	Yoris	23.03.2026

11.7 Sprint Planning / 20.03.2026

Field	Value
Date	20.03.2026
Time	15:00–16:00
Sprint	Sprint 3
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

11.7.1 Sprint goal

- Create working MVP.
- Implement the foundational Item and Node Architecture.

11.8 Sprint Review (Advisor) / 24.03.2026

Field	Value
Date	24.03.2026
Time	15:00–16:00
Sprint	Sprint 3
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Cedric Cathomas
Links	GitHub repository + Jira board (internal team links)

11.8.1 Agenda

Agenda item	Owner	Result / Decision
Documentation status (FR, architecture, protocols)	Yoris Kucera	Overall quality was positively received; minor consistency updates required (see Todos)
Current game state demo	Yoris Kucera	Game is playable as planned for pre-alpha release
Mockups	Yoris Kucera	Gave a clear picture of how our UI will look
Code quality (linter/status)	Yoris Kucera	Everything was good
Advisor feedback	Thomas Bocek	Very good feedback, overall impression better than last week

11.8.2 Todos

Todo	Owner	Due
FR: align color definitions and structure as in the NFRs	Cedric Cathomas	27.03.2026
Enemy state machine: add explicit color legend to diagram	Yoris Kucera	27.03.2026

11.8.3 Next dates

Next date	When
Review 3 - End of Elaboration	14.04.2026 15:00

11.9 Weekly Scrum / 27.03.2026

Field	Value
Date	27.03.2026
Time	14:00–14:45
Sprint	Sprint 3
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Nathanael Fässler
Links	-

11.9.1 Agenda

Agenda item	Owner	Result / Decision
New Nodes and Items created with new 3D Models	Dominik Wyss	Scriptable Object Architecture works
Pixelate Shader	Yoris Kucera	- Team thinks it looks good - Showcased parameters of the shader - Camera angle and zoom needs refining
Inventory UI	Cedric Cathomas	Prototype is working
Node and Tile hovering Architecture	Nathanael Fässler	Will be implemented with events

11.9.2 Todos

Todo	Owner	Due
Explicitly address review feedback in Teams	Dominik Wyss	03.04.2026

11.10 Weekly Scrum and Sprint Retrospective / 03.04.2026

Field	Value
Date	03.04.2026
Time	13:00–14:00
Sprint	Sprint 3
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Cedric Cathomas
Minutes by	Dominik Wyss
Links	-

11.10.1 Agenda

Agenda item	Owner	Result / Decision
Add new nodes, items, tools	Dominik Wyss	- New ScriptableObjects and Prefabs - Updated MainWorld to demo new features
Pixelized Shader	Yoris Kucera	Working shader with editable fields
Building System	Nathanael Fässler	- Chest as first building block - Valid/invalid placement indicator
Inventory UI with system architecture changes	Cedric Cathomas	Inventory and handslot
Feedback Review 2: FR coloring indicator same as NFR	Cedric Cathomas	
Feedback Review 2: Enemy state machine: add explicit color legend to diagram	Yoris Kucera	

11.10.2 What went well

- Good communication within the team
- Strong interest and engagement in the project
- Attention to detail
- Review comments were questioned and discussed, not just - blindly accepted
- Everything came together into a working game

11.10.3 What didn't

- Conflicting ideas about how to implement features and resulting dependencies sometimes disrupted the workflow.
- Tickets should be described more clearly to support - conceptual implementation
- Planned too much
- The 3D player model was more difficult than expected

11.10.4 Todos

Todo	Owner	Due
------	-------	-----

Send message to advisor about feedback from last review	Dominik	10.04.2026
Send message to advisor about review 3, point A5. Remove A5 from checklist because we don't have external systems.	Dominik	10.04.2026

11.11 Sprint Planning / 03.04.2026

Field	Value
Date	03.04.2026
Time	14:00–15:30
Sprint	Sprint 4
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Cedric Cathomas
Minutes by	Dominik Wyss
Links	-

11.11.1 Sprint goal

- Finish tickets from old sprint
- Working prototype with all core mechanics
- Focus on diagramm and documentation changes for review 3

11.12 Sprint Review (Advisor) / 14.04.2026

Field	Value
Date	14.04.2026
Time	15:00–16:00
Sprint	Sprint 4
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	GitHub repository + Jira board (internal team links)

11.12.1 Agenda

Agenda item	Owner	Result / Decision
Showcase DEV-environment	Yoris Kucera	GitHub Action minutes are scarce. We should search for a better solution.
Test Concept	Yoris Kucera	Positive: The idea of Integration Tests was adapted to the domain to use Playmode tests.
C4 Architecture	Yoris Kucera	Well-documented and focuses on high-level topics.
Techstack & Patterns	Yoris Kucera	Use of component-based architecture makes sense. Unity inspector variables are not really dependency injection.
External Dependencies	Thomas Bocek	Missing specification of Steam as an external dependency.
Alpha Demo	Nathanael Fässler	Core game loop is working and playable. Player usability has to be improved.
Extensibility Demo	Nathanael Fässler	Demonstration to show how a new Node can be added without writing code.
Savefile Structure	Yoris Kucera	Could easily be changed to support generated worlds by including the seed in the header.

11.12.2 Todos

Todo	Owner	Due
Research solution to reduce GitHub Runner minutes	Yoris Kucera	28.04.2026
Document Steam as a external dependency	Dominik Wyss	28.04.2026

11.12.3 Next dates

Next date	When
Review 4 - Quality	05.05.2026 15:00

11.13 Weekly Scrum and Sprint Retrospective / 18.04.2026

Field	Value
Date	18.04.2026
Time	13:00–14:00
Sprint	Sprint 4
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Yoris Kucera
Links	-

11.13.1 Agenda

Agenda item	Owner	Result / Decision
Discuss GlowCore Upgrade UI	Nathanael Fässler/ Cedric Cathomas	Upgrade system with individual levels and large upgrades
Review TODOs	All	- Steam Interface Documentation - Self-Hosted GitHub Runner ▸ New Task Created
Save-Data	Yoris Kucera	- Almost done - Moved to next sprint

11.13.2 What went well

- Teamwork in Documentation Tasks
- Most tasks were completed this sprint
- Everyone took on tasks as appropriate
- Alpha Release

11.13.3 What didn't

- Save-Data Task still not done
- A little close on time (Spring Break)
- Uneven distribution of work-types (e.g. one person doing more docs)
- PRs with only small doc changes take a long time until review

11.13.4 Todos

Todo	Owner	Due
Think about items to be added to the game and what use they could have	All	24.04.2026

11.14 Sprint Planning / 18.04.2026

Field	Value
Date	18.04.2026
Time	14:00–15:00
Sprint	Sprint 5
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Yoris Kucera
Links	-

11.14.1 Sprint goal

- Document code style and testing
- Improve test coverage
- Focus on playtesting

11.15 Weekly Scrum / 24.04.2026

Field	Value
Date	24.04.2026
Time	11:00–11:30
Sprint	Sprint 5
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Cedric Cathomas
Links	-

11.15.1 Agenda

Agenda item	Owner	Result / Decision
NFR and UC current Testing state	Cedric Cathomas	- partially implemented for certain UC's and NFR's is fine - Need to synchronize deadlines - new color code N/A is needed
Integration Tests	Nathanael Fässler	Not all wished integration tests will be possible to write.
New Tasks in Backlog due to discussion with Advisor	Dominik Wyss	- Dominik explained wished changes of the advisor 4 new tasks got created in jira - further descriptions of the tasks can be found on the corresponding ticket or we can ask dominik.

11.15.2 Todos

Todo	Owner	Due
Create new color code for the verification of UC's and NFR's	Cedric Cathomas	01.05.2026

11.16 Weekly Scrum and Sprint Retrospective / 30.04.2026

Field	Value
Date	30.04.2026
Time	20:30–20:40
Sprint	Sprint 5
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Yoris Kucera
Links	-

11.16.1 What went well

- Time management
- Proper prototype is cool
- Teamwork
- New outline looks good
- Savefile

11.16.2 What didn't

- Time management
- Tasks still not done 1 day before sprint end
- Extra tasks from advisor
- Random features in one PR that were not planned
- BIG PRs

11.17 Sprint Planning / 01.05.2026

Field	Value
Date	01.05.2026
Time	13:00–15:00
Sprint	Sprint 6
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Cedric Cathomas
Excused	Yoris Kucera
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Cedric Cathomas
Links	-

11.17.1 Sprint goal

- Extend game content with phased progression
- Add new resources and mechanics
- Improve game playability and tutorial pacing
- Implement main menu, refine GlowCore UI and crafting system

11.18 Sprint Review (Advisor) / 05.05.2026

Field	Value
Date	05.05.2026
Time	15:00–16:00
Sprint	Sprint 6
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Nathanael Fässler
Links	GitHub repository + Jira board (internal team links)

11.18.1 Agenda

Agenda item	Owner	Result / Decision
GlowCore v0.2.0 demo	Dominik Wyss	Works, but still needs improvements.
GitHub selfhosted runner	Yoris Kucera	Is working. We have to be careful if we make the repository public.
Quality measures	Dominik Wyss	There are files with less than 10 lines of code. In our case it makes sense to keep them separate.
User Tests	Nathanael Fässler	The first two user tests provided valuable feedback.
Code Review	Dominik Wyss	WorldGrid.cs was reviewed. No unclean code found.
Steampage	Dominik Wyss	Looks good.

11.18.2 Todos

Todo	Owner	Due
Document deployment on steam	not assigned	19.05.2026

11.18.3 Next dates

Next date	When
Review 5 - Architecture	19.05.2026 15:00

11.19 Weekly Scrum / 08.05.2026

Field	Value
Date	08.05.2026
Time	20:00–20:30
Sprint	Sprint 6
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Yoris Kucera
Links	-

11.20 Weekly Scrum and Sprint Retrospective / 15.05.2026

Field	Value
Date	15.05.2026
Time	13:00-14:00
Sprint	Sprint 6
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Cedric Cathomas
Links	-

11.20.1 Agenda

Agenda item	Owner	Result / Decision
UI design and title scene	Cedric Cathomas	• Showed how the implemented UI looks and works
Documentation	Nathanael Fässler and Yoris Kucera	• Presented Documentation changes • Deployment diagram • Design decisions • Game design • Frontend Architecture
Audio system	Dominik Wyss	• Demo of how to implement new audio effects • Presented Audio manager singleton
Biome and Environment	Dominik Wyss	• Demo of how to implement biomes and environment objects

11.20.2 What went well

- Game looks much better, overall impression improved alot, looks like a real game now
- Border looks nice
- Good teamwork

11.20.3 What didn't

- Discord communication tone
- Non documented documentation tasks

11.20.4 Todos

Todo	Owner	Due
NFR Verification	Cedric Cathomas	17.05.2026
Time tracking	All	17.05.2026

11.21 Sprint Planning / 15.05.2026

Field	Value
Date	15.05.2026
Time	14:00–15:00
Sprint	Sprint 7
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Cedric Cathomas, Yoris Kucera
Excused	.
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Cedric Cathomas
Links	-

11.21.1 Sprint goal

- Finalise Audio system
- Finalise Game biomes and environment objects
- Presentation preparation
- Final game features

11.22 Sprint Review (Advisor) / 19.05.2026

Field	Value
Date	19.05.2026
Time	15:00–16:00
Sprint	Sprint 7
Location	Building 8
Participants	Thomas Bocek, Dominik Wyss, Yoris Kucera, Nathanael Fässler, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Dominik Wyss
Minutes by	Nathanael Fässler
Links	GitHub repository + Jira board (internal team links)

11.23 Weekly Scrum / 22.05.2026

Field	Value
Date	22.05.2026
Time	20:00–20:30
Sprint	Sprint 7
Location	Online
Participants	Cedric Cathomas, Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Yoris Kucera
Links	-

11.24 Weekly Scrum and Sprint Retrospective / 29.05.2026

Field	Value
Date	29.05.2026
Time	20:30–20:40
Sprint	Sprint 7
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Yoris Kucera
Links	-

Bibliography

List of Figures

Image 1	Use Case Diagram	7
Image 2	Domain Model Diagram	22
Image 3	Enemy State Machine Diagram	23
Image 4	Main Menu	24
Image 5	New Game	25
Image 6	Pause Menu	26
Image 7	Audio Settings	27
Image 8	Display Settings	28
Image 9	Unapplied Changes	29
Image 10	Inventory	30
Image 11	Crafting Table	31
Image 12	More Crafting Stations	32
Image 13	GlowCore Upgrade	33
Image 14	Chest	34
Image 15	C4 System Context Diagram	35
Image 16	C4 Container Diagram	36
Image 17	C4 Component Diagram - Gameplay Logic	37
Image 18	C4 Component Diagram - Presentation Layer	38
Image 19	Interface Overview Diagram	39
Image 20	Deployment Diagram	48
Image 21	CI/CD Pipeline Overview	49
Image 22	Definition of Ready and Definition of Done - ticket lifecycle flowchart	50
Image 23	GC-186 - Example of a ticket satisfying all Definition of Ready criteria	51
Image 24	GC-186 - Example of a ticket satisfying all Definition of Done criteria	51
Image 25	Interface Overview Diagram	54
Image 26	Savefile Structure	61

Image 27 Initial Jira Timeline	71
Image 28 Updated Jira Timeline	71
Image 29 Inital Backlog	72
Image 30 Sprint 1 Backlog	73
Image 31 Sprint 2 Burndown Chart	74
Image 32 Sprint 2 Backlog	74
Image 33 Sprint 3 Burndown Chart	75
Image 34 Sprint 3 Backlog	75
Image 35 Sprint 4 Burndown Chart	76
Image 36 Sprint 4 Backlog	76
Image 37 Sprint 5 Burndown Chart	77
Image 38 Sprint 5 Backlog	77
Image 39 Sprint 6 Burndown Chart	78
Image 40 Sprint 6 Backlog	78

List of Tables

Table 1	Glossary	5
Table 2	UC Status Color Legend	8
Table 3	UC Overview – All Use Cases at a Glance	8
Table 4	UC01 - Install Game	9
Table 5	UC02 - Save Game	9
Table 6	UC03 - Break Nodes	9
Table 7	UC04 - Expand Map	9
Table 8	UC05 - Store Items	10
Table 9	UC06 - Craft Items	10
Table 10	UC07 - Build Nodes	10
Table 11	UC08 - Experience Story	10
Table 12	UC09 - Fight Enemies	11
Table 13	UC10 - Automate Resource Gathering	11
Table 14	UC11 - Change Settings	11
Table 15	NFR Status Color Legend	12
Table 16	NFR Overview – All Non-Functional Requirements at a Glance	13
Table 17	NFR Priority Matrix – MVP Relevance vs. Importance	14
Table 18	NFR101 – World Load Time	14
Table 19	NFR102 – In-Game Transition Time	14
Table 20	NFR103 – Frame Rate	15
Table 21	NFR104 – Save File Size	15
Table 22	NFR105 – Minimal Hardware Requirements	15
Table 23	NFR201 – Player Restriction Visibility	16
Table 24	NFR202 – Light Upgrade Progress Indication	16
Table 25	NFR203 – New Player Learnability	16
Table 26	NFR204 – Input Support	17

Table 27 NFR205 – Game Language	17
Table 28 NFR206 – Visual Style Consistency	17
Table 29 NFR301 – Save File Portability	18
Table 30 NFR302 – Game Progress Persistence	18
Table 31 NFR303 – Save Data Integrity	18
Table 32 NFR401 – Extensibility of Game Systems	19
Table 33 NFR402 – Automated Test Coverage	19
Table 34 NFR501 – Multi-Platform Support	20
Table 35 NFR502 – Engine and Rendering Pipeline	20
Table 36 NFR503 – Display Resolution Support	20
Table 37 NFR-to-Architecture Traceability	21
Table 38 Interface Contracts	40
Table 39 UT01	57
Table 40 UT01 - Additional Feedback	57
Table 41 UT02	58
Table 42 UT02 - Additional Feedback	58
Table 43 Team availability for review meetings	63
Table 44 Project Management Roles	65
Table 45 Developer Roles	66
Table 46 Planned Meetings Overview	67
Table 47 Planning Tools	67
Table 48 Work Items	68
Table 49 Initial Milestones	69
Table 50 Updated Milestones	69
Table 51 Epics	70
Table 52 Risk Analysis	79
Table 53 Risk Matrix	80
Table 54 Risk Matrix (Mitigated)	80
Table 55 Assessed Risk Matrix (14.04.2026)	80

Table 56 Time Statistics Sprint 1 82

Code Listings